

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Actualización del núcleo de Multimodal Observer (MO)

Matías Jacop Figueroa Contreras

Profesor guía: Roberto González Ibáñez

Tesis para optar al título de Ingeniero
Civil en Informática

Santiago – Chile
2025

RESUMEN

En un contexto donde la investigación científica exige herramientas capaces de integrar diversas fuentes de información, la captura y análisis de datos multimodales se vuelve esencial para obtener una comprensión holística del comportamiento humano. No obstante, muchas de las soluciones disponibles en el mercado presentan costos elevados, escasa flexibilidad y baja interoperabilidad con distintos dispositivos, lo que refuerza la relevancia de MO, una aplicación de escritorio modular, extensible y multiplataforma desarrollada para la captura, visualización y análisis de datos multimodales. MO fue concebido como un software de código abierto y adaptable, sin embargo, la evolución tecnológica y las múltiples versiones de sus módulos derivaron en problemas de compatibilidad entre su núcleo y los plugins, afectando la incorporación de nuevas funcionalidades y su mantenimiento.

Frente a esta problemática, se desarrolló una nueva versión de MO mediante una reimplementación completa de su núcleo, basada en tecnologías modernas que respetan su arquitectura original y fortalecen su extensibilidad a través de plugins. El proyecto fue guiado por el ciclo PDCA (Plan-Do-Check-Act) y apoyado por metodologías RAD y KanBan, permitiendo iteraciones continuas que abarcaron el diagnóstico del estado actual, rediseño, implementación y validación de la solución. En línea con este enfoque metodológico, se construyeron múltiples prototipos funcionales, los cuales sirvieron como base para validar progresivamente los componentes desarrollados y completar los requerimientos del software. Las distintas fases del proyecto fueron acompañadas de pruebas unitarias, de integración, rendimiento y aceptación, las cuales permitieron verificar la calidad técnica del sistema desarrollado. Finalmente, se alcanzaron todos los objetivos planteados, obteniendo una nueva versión operativa de MO, con un núcleo actualizado y una base sólida para su evolución continua como una herramienta clave para la investigación multimodal.

Palabras Claves: multimodal, plugin, extensibilidad, modular, reimplementación

AGRADECIMIENTOS

Quiero comenzar agradeciendo profundamente a mi mamá, cuyo esfuerzo y apoyo constante han sido fundamentales durante todos estos años. Junto a mis hermanos y abuelos, su compañía y cariño han sido siempre el impulso que me ha motivado a seguir adelante. También quiero agradecer a mis amigos, por su compañía y por esos momentos de distracción tan necesarios, que hicieron este camino mucho más llevadero. Finalmente, agradezco a mi profesor guía, Roberto González, por su disposición, orientación y apoyo a lo largo de todo este proceso.

TABLA DE CONTENIDO

1	Introducción	1
1.1	Antecedentes y motivación	1
1.2	Descripción del problema	2
1.3	Solución propuesta	3
1.3.1	Características de la solución	3
1.3.2	Propósito de la solución	4
1.4	Objetivos y alcance del proyecto	4
1.4.1	Objetivo general	4
1.4.2	Objetivos específicos	4
1.4.3	Alcances	5
1.5	Metodología y herramientas utilizadas	5
1.5.1	Metodología	5
1.5.1.1	Ciclo PDCA (Plan-Do-Check-Act)	6
1.5.1.2	Rapid Application Development (RAD)	7
1.5.1.3	Kanban	8
1.5.2	Herramientas de desarrollo	8
1.5.3	Ambiente de desarrollo	8
1.6	Organización del documento	9
2	Marco Teórico	10
2.1	Marco Conceptual	10
2.1.1	Estado actual de MO	10
2.1.2	Actualización de software	13
2.1.3	Extensibilidad	14
2.1.4	Modularidad	14
2.1.5	Plugins	15
2.2	Estado del Arte	15
2.2.1	Software para el trabajo multimodal	15
2.2.1.1	NoldusHub	15
2.2.1.2	iMotions	16
2.2.1.3	The Observer XT	16
2.2.1.4	OpenSignals	17
2.2.2	Software con implementación de plugins	17
2.2.2.1	OBS Studio	17
2.2.2.2	Visual Studio Code	18
2.2.2.3	WordPress	18
2.3	Resumen	19
3	Análisis	21
3.1	Diagnóstico	21
3.1.1	Revisión evolución de MO	21
3.1.2	Revisión de compatibilidad con Plugins	22
3.1.2.1	Plugins de Captura	23
3.1.2.2	Plugins de Visualización	23
3.1.2.3	Plugins de Análisis	24
3.1.3	Conclusión	24
3.2	Captura de Requerimientos	25
3.2.1	Requerimientos funcionales	25
3.2.2	Requerimientos no funcionales	26
3.3	Factibilidad técnica	27

3.3.1	Criterios para la Selección Tecnológica	27
3.3.2	Tecnologías Evaluadas y Comparación	27
3.3.3	Tecnologías Seleccionadas y Justificación	28
3.4	Prototipado	29
3.4.1	Primer prototipo: Replicación de funcionalidades críticas	30
3.4.2	Segundo prototipo: Implementación del módulo de organización	34
3.4.3	Tercer prototipo: Implementación del módulo de captura y gestión de plugins	42
3.4.4	Cuarto prototipo: Implementación del módulo de visualización y gestión de plugins en la interfaz	51
3.4.5	Quinto prototipo: Preferencias de usuario y ajustes de la aplicación	57
3.4.6	Sexto prototipo: Instalador de MO y conjunto de plugins	61
3.5	Resumen	63
4	Diseño y aspectos de Implementación	65
4.1	Vista física	65
4.1.1	Aplicación Escritorio (Frontend)	66
4.1.2	Plugins del Frontend	67
4.1.3	Backend (Servidor FastAPI)	68
4.1.4	Plugins del Backend	68
4.1.5	Almacenamiento Local	68
4.2	Vista de desarrollo	69
4.2.1	Frontend	70
4.2.2	Backend	71
4.2.3	Plugins	72
4.3	Vista lógica	73
4.3.1	Núcleo	73
4.3.2	Módulo de Organización	80
4.3.3	Módulo de Captura	84
4.3.4	Módulo de Visualización	88
4.4	Vista de procesos	92
4.4.1	Configuración inicial de un proyecto	92
4.4.2	Ejecución de un protocolo de investigación	95
4.4.3	Proceso de captura de datos	98
4.4.4	Reproducción de una sesión	108
4.5	Escenarios	111
4.5.1	Configuración inicial de un proyecto	111
4.5.2	Ejecución de un protocolo	113
4.5.3	Captura de datos	115
4.5.4	Reproducción de una sesión	117
4.6	Resumen	119
5	Evaluación	120
5.1	Ambiente de ejecución de pruebas	120
5.2	Pruebas unitarias, de integración y de análisis estático	121
5.3	Pruebas de rendimiento	124
5.4	Pruebas de validez y confiabilidad	132
5.4.1	Evaluación del plugin de captura de mouse	133
5.4.2	Evaluación del plugin de captura de teclado	135
5.4.3	Evaluación del plugin de captura de pantalla	137
5.5	Pruebas de aceptación	140
6	Conclusiones	141
6.1	Objetivos	141
6.1.1	Objetivos específicos	141
6.1.2	Objetivo general	142

6.2	Implicancias y alcances	142
6.2.1	Implicancias	142
6.2.2	Alcances	143
6.3	Trabajo futuro	143
6.4	Reflexiones personales	144
Glosario		146
Referencias bibliográficas		147
Anexos		151
A	Licencia	151
Apéndices		152
A	Requerimientos Funcionales	152
B	Diagramas de clases	158
C	Pruebas de aceptación	162

ÍNDICE DE TABLAS

Tabla 2.1	Software para el trabajo multimodal.	20
Tabla 2.2	Software con implementación de plugins.	20
Tabla 3.1	Resumen de requerimientos funcionales por módulo.	26
Tabla 3.2	Requerimientos NO Funcionales.	26
Tabla 3.3	Primer prototipo - Replicación de funcionalidades críticas.	30
Tabla 3.4	Segundo prototipo - Implementación del módulo de organización.	34
Tabla 3.5	Tercer prototipo - Implementación del módulo de captura y gestión de plugins.	43
Tabla 3.6	Cuarto prototipo - Implementación del módulo de visualización y gestión de plugins en la interfaz.	51
Tabla 3.7	Quinto prototipo - Preferencias de usuario y ajustes de la aplicación.	57
Tabla 3.8	Sexto prototipo - Instalador de MO y el conjunto de plugins.	63
Tabla 3.9	Resumen de requerimientos funcionales cubiertos por prototipo.	63
Tabla 3.10	Resumen de requerimientos no funcionales cubiertos por prototipo.	64
Tabla 4.1	Componentes del núcleo en el frontend.	71
Tabla 4.2	Módulos específicos del frontend.	71
Tabla 4.3	Componentes del núcleo en el backend.	72
Tabla 4.4	Módulos específicos del backend.	72
Tabla 4.5	Descripción de clases del núcleo (backend).	75
Tabla 4.6	Descripción de clases del núcleo (frontend).	76
Tabla 4.7	Descripción de clases del módulo de organización.	81
Tabla 4.8	Descripción de clases del módulo de captura.	84
Tabla 4.9	Descripción de clases del módulo de visualización (backend).	89
Tabla 4.10	Descripción de clases del módulo de visualización (frontend).	90
Tabla 4.11	Casos de uso del escenario de configuración inicial de un proyecto.	112
Tabla 4.12	Casos de uso del escenario de ejecución de un protocolo.	114
Tabla 4.13	Casos de uso del escenario de captura de datos.	116
Tabla 4.14	Casos de uso del escenario de reproducción de una sesión.	118
Tabla 5.1	Criterios de calificación en SonarQube.	122
Tabla 5.2	Resultados de pruebas unitarias, de integración y SCA para el backend.	123
Tabla 5.3	Resultados de pruebas unitarias, de integración y SCA para el frontend.	124
Tabla 5.4	Resumen de resultados de las pruebas de rendimiento (Ambiente 1).	125
Tabla 5.5	Resultados prueba de rendimiento (Ambiente 1): captura multimodal.	126
Tabla 5.6	Resultados prueba de rendimiento (Ambiente 1): captura con plugin de teclado.126	
Tabla 5.7	Resultados prueba de rendimiento (Ambiente 1): captura con plugin del mouse.127	
Tabla 5.8	Resultados prueba de rendimiento (Ambiente 1): captura con plugin de pantalla.127	
Tabla 5.9	Resumen de resultados de las pruebas de rendimiento (Ambiente 2).	128
Tabla 5.10	Resultados prueba de rendimiento (Ambiente 2): captura multimodal.	128
Tabla 5.11	Resultados prueba de rendimiento (Ambiente 2): captura con plugin de teclado.129	
Tabla 5.12	Resultados prueba de rendimiento (Ambiente 2): captura con plugin del mouse.129	
Tabla 5.13	Resultados prueba de rendimiento (Ambiente 2): captura con plugin de pantalla.129	
Tabla 5.14	Resumen de resultados de las pruebas de rendimiento (Ambiente 3).	130
Tabla 5.15	Resultados prueba de rendimiento (Ambiente 3): captura multimodal.	131
Tabla 5.16	Resultados prueba de rendimiento (Ambiente 3): plugin de captura del teclado. 131	
Tabla 5.17	Resultados prueba de rendimiento (Ambiente 3): plugin de captura del mouse. 132	
Tabla 5.18	Resultados prueba de rendimiento (Ambiente 3): plugin de captura de pantalla. 132	
Tabla 5.19	Mediciones de las métricas en la captura de mouse con MO y Nut.js.	134
Tabla 5.20	Coeficiente de varianza por métrica de mouse.	134

Tabla 5.21 Mediciones de las métricas en la captura de teclado con MO y Nut.js.	136
Tabla 5.22 Coeficiente de variación por métrica de teclado.	136
Tabla 5.23 Mediciones de las métricas en la captura de pantalla con MO y OBS.	138
Tabla 5.24 Resumen y errores relativos entre MO y OBS para distintas tasas de FPS. . .	138
Tabla 5.25 Coeficiente de variación por métrica de pantalla.	139
Tabla 5.26 Categorías de evaluación de las pruebas de aceptación.	140
Tabla 5.27 Resultados generales de las pruebas de aceptación.	140
 Tabla A.1 Requerimientos Funcionales con Historias de Usuario asociadas.	 152
 Tabla C.1 Prueba de aceptación PA-01 - Operaciones con plugins.	 162
Tabla C.2 Prueba de aceptación PA-02 - Configuración inicial de un proyecto.	162
Tabla C.3 Prueba de aceptación PA-03 - Proceso de captura y ejecución de protocolo. . .	163
Tabla C.4 Prueba de aceptación PA-04 - Reproducción de sesión capturada.	164
Tabla C.5 Prueba de aceptación PA-05 - Operaciones extra participante.	165
Tabla C.6 Prueba de aceptación PA-06 - Operaciones extra protocolo.	165
Tabla C.7 Prueba de aceptación PA-07 - Operaciones extra plugin configurable.	166
Tabla C.8 Prueba de aceptación PA-08 - Operaciones extra proyecto.	167

ÍNDICE DE ILUSTRACIONES

Figura 2.1	Interfaz original de MO en sus primeras versiones.	11
Figura 2.2	Interfaz de MO luego del rediseño del modelo de interacción.	12
Figura 2.3	Arquitectura en capas de MO: Núcleo, Módulos y Extensiones.	12
Figura 2.4	Línea de tiempo con los principales hitos del desarrollo de MO.	13
Figura 3.1	Primer prototipo - Plugins de teclado y mouse.	31
Figura 3.2	Primer prototipo - Plugin de video.	32
Figura 3.3	Primer prototipo - Procesos.	32
Figura 3.4	Primer prototipo - Comunicación.	33
Figura 3.5	Primer prototipo - System Tray.	33
Figura 3.6	Segundo prototipo - Vista general.	35
Figura 3.7	Segundo prototipo - Acciones para un proyecto.	35
Figura 3.8	Segundo prototipo - Crear proyecto.	36
Figura 3.9	Segundo prototipo - Actualizar proyecto.	36
Figura 3.10	Segundo prototipo - Ver información de un proyecto.	36
Figura 3.11	Segundo prototipo - Eliminar proyecto.	37
Figura 3.12	Segundo prototipo - Eliminar proyecto bloqueado.	37
Figura 3.13	Segundo prototipo - Agregar participante a un proyecto.	37
Figura 3.14	Segundo prototipo - Actualizar participante.	38
Figura 3.15	Segundo prototipo - Ver información de un participante.	38
Figura 3.16	Segundo prototipo - Agregar protocolo a un proyecto.	39
Figura 3.17	Segundo prototipo - Agregar actividad a un protocolo.	39
Figura 3.18	Segundo prototipo - Actualizar protocolo.	40
Figura 3.19	Segundo prototipo - Información de un protocolo.	40
Figura 3.20	Segundo prototipo - Protocolo en ejecución 1.	41
Figura 3.21	Segundo prototipo - Protocolo en ejecución 2.	41
Figura 3.22	Segundo prototipo - Protocolo en ejecución 3.	42
Figura 3.23	Tercer prototipo - Agregar plugin.	43
Figura 3.24	Tercer prototipo - Plugins instalados.	44
Figura 3.25	Tercer prototipo - Información de un Plugin.	44
Figura 3.26	Tercer prototipo - Vista general.	45
Figura 3.27	Tercer prototipo - Capturando.	45
Figura 3.28	Tercer prototipo - Fuentes de captura.	46
Figura 3.29	Tercer prototipo - Listado de sesiones.	46
Figura 3.30	Tercer prototipo - Eliminar sesión.	46
Figura 3.31	Tercer prototipo - Información de sesión.	47
Figura 3.32	Tercer prototipo - Archivos capturados.	48
Figura 3.33	Tercer prototipo - Configuración de plugin de captura de pantalla.	48
Figura 3.34	Tercer prototipo - Configuración de plugin de captura de teclado.	49
Figura 3.35	Tercer prototipo - Archivo resultante captura de teclado.	49
Figura 3.36	Tercer prototipo - Configuración de plugin de captura del mouse.	50
Figura 3.37	Tercer prototipo - Archivo resultante captura del mouse.	50
Figura 3.38	Cuarto prototipo - Plugins instalados.	52
Figura 3.39	Cuarto prototipo - Cambio de tipo de configuración.	52
Figura 3.40	Cuarto prototipo - Acciones configuración de "Playback Views".	53
Figura 3.41	Cuarto prototipo - Vista de reproducción de sesiones capturadas.	54
Figura 3.42	Cuarto prototipo - Redimensión y cambio de espacio de trabajo.	55
Figura 3.43	Cuarto prototipo - Ventana independiente en la reproducción de sesiones.	55
Figura 3.44	Cuarto prototipo - Sesiones disponibles.	56
Figura 3.45	Cuarto prototipo - Sección de controles.	56

Figura 3.46	Quinto prototipo - System tray 1.	58
Figura 3.47	Quinto prototipo - System tray 2.	58
Figura 3.48	Quinto prototipo - Sección de ajustes.	59
Figura 3.49	Quinto prototipo - Vista principal en español.	59
Figura 3.50	Quinto prototipo - Archivo para almacenar preferencias.	60
Figura 3.51	Sexto prototipo - Selección de plugins a instalar.	61
Figura 3.52	Sexto prototipo - Elección del directorio de instalación.	62
Figura 3.53	Sexto prototipo - Instalación finalizada.	62
Figura 4.1	Diagrama de despliegue.	65
Figura 4.2	Diagrama de componentes.	70
Figura 4.3	Diagrama de clases del Núcleo (backend).	74
Figura 4.4	Diagrama de clases del Núcleo (frontend).	74
Figura 4.5	Diagrama de clases del módulo de Organización.	81
Figura 4.6	Diagrama de clases del módulo de Captura.	84
Figura 4.7	Diagrama de clases del módulo de Visualización (backend).	88
Figura 4.8	Diagrama de clases del módulo de Visualización (frontend).	89
Figura 4.9	Diagrama de secuencia - Configuración inicial de un proyecto.	93
Figura 4.10	Diagrama de secuencia - Agregar configuraciones de plugins de captura.	94
Figura 4.11	Diagrama de secuencia - Agregar configuraciones de plugins de reproduc- ción.	94
Figura 4.12	Diagrama de secuencia - Ejecución de un protocolo de investigación.	96
Figura 4.13	Diagrama de secuencia - Captura de datos.	99
Figura 4.14	Diagrama de secuencia - Preparar captura de datos.	100
Figura 4.15	Diagrama de secuencia - Comenzar captura de datos.	102
Figura 4.16	Diagrama de secuencia - Comenzar proceso de buffering y almace- namiento de datos.	104
Figura 4.17	Diagrama de secuencia - Pausar captura de datos.	105
Figura 4.18	Diagrama de secuencia - Reanudar captura de datos.	106
Figura 4.19	Diagrama de secuencia - Detener captura de datos.	108
Figura 4.20	Diagrama de secuencia - Reproducción de una sesión.	109
Figura 4.21	Diagrama de secuencia - Montar layout de reproducción.	110
Figura 4.22	Diagrama de casos de uso - Configuración inicial de un proyecto.	112
Figura 4.23	Diagrama de casos de uso - Ejecución de un protocolo.	114
Figura 4.24	Diagrama de casos de uso - Captura de datos.	116
Figura 4.25	Diagrama de casos de uso - Reproducción de una sesión.	118
Figura 5.1	Gráfico de diana para la captura de mouse.	135
Figura 5.2	Gráfico de diana para la captura de teclado.	137
Figura 5.3	Gráfico de diana para la captura de pantalla.	139
Figura B.1	Diagramas de clases del núcleo (backend).	158
Figura B.2	Diagramas de clases del núcleo (frontend).	158
Figura B.3	Diagramas de clases del módulo de Organización (backend).	159
Figura B.4	Diagramas de clases del módulo de Captura (backend).	160
Figura B.5	Diagramas de clases del módulo de Visualización (backend).	160
Figura B.6	Diagramas de clases del módulo de Visualización (frontend).	161

CAPÍTULO 1. INTRODUCCIÓN

1.1 ANTECEDENTES Y MOTIVACIÓN

La recopilación y el análisis de datos multimodales son fundamentales para el avance en diversas áreas de investigación, incluyendo la medicina, la informática, las ciencias sociales y el desarrollo de inteligencia artificial. Su relevancia radica en que permiten superar las limitaciones propias de los estudios unimodales, que ofrecen una visión parcial del fenómeno estudiado. Al combinar e integrar múltiples fuentes de datos, se obtiene una comprensión más profunda y holística del comportamiento humano (Bogers, 2024). Esta integración no solo proporciona una imagen más completa y precisa de procesos complejos, como los cognitivos y emocionales, sino que también permite identificar patrones, y relaciones que podrían permanecer ocultos si se analizara cada dato por separado. Además, el enfoque multimodal puede ofrecer distintos conocimientos y perspectivas sobre los mecanismos subyacentes al comportamiento observado, ayudando a entender el "por qué" detrás de este comportamiento, y aumentando la objetividad y precisión de las mediciones (Bogers, 2024; Spink, 2023).

Para llevar a cabo este tipo de análisis de manera efectiva, es indispensable contar con herramientas especializadas diseñadas explícitamente para facilitar la captura, integración, visualización y análisis de datos multimodales. Estas herramientas deben ser capaces de manejar la complejidad que conlleva la combinación de múltiples fuentes de datos, proporcionando un entorno robusto que permita a los investigadores centrarse en el análisis y la interpretación científica, en lugar de verse involucrado en obstáculos técnicos.

En este contexto, Multimodal Observer (MO) surge como una herramienta clave para abordar estas necesidades. MO es una aplicación de escritorio desarrollada en Java, creada bajo la autoría del laboratorio de investigación y desarrollo *InTeractiOn-Lab* (InTeractiOn, s.f.). La aplicación fue diseñada por el director del laboratorio, Roberto González, y su primera implementación fue realizada por Celso Gutiérrez, memorista del Departamento de Ingeniería Informática de la Universidad de Santiago de Chile. MO está diseñada para capturar, visualizar y analizar datos multimodales, permitiendo la integración de diversas fuentes de entrada como teclado, mouse y sensores fisiológicos. Asimismo, sigue un enfoque flexible y extensible, que permite la incorporación de nuevas características a través de un sistema de plugins, facilitando su evolución sin necesidad de modificar el núcleo del sistema (Gutiérrez Gálvez, 2016).

Sin embargo, debido a la evolución tecnológica y a las constantes actualizaciones de MO, la aplicación ha experimentado problemas de compatibilidad entre su núcleo y los plugins desarrollados. Por ello, surge la necesidad de actualizar el núcleo de la aplicación y sus

módulos esenciales para asegurar su correcto funcionamiento y mejorar su compatibilidad con las tecnologías actuales. Es importante destacar que los motivos originales para la creación de MO (Gutiérrez Gálvez, 2016) siguen siendo relevantes, ya que muchas de las soluciones existentes en el mercado tienen un costo elevado, están diseñadas para áreas de estudio específicas, carecen de integración flexible con diversos dispositivos de captura (generalmente limitándose a productos de la misma marca) y no siempre son multiplataforma, restringiendo su uso a un solo sistema operativo. Además, estas soluciones no son extensibles, lo que impide agregar nuevas funcionalidades, como la incorporación de nuevos dispositivos de captura o formas alternativas de visualizar los datos.

Considerando que los motivos originales para la creación de MO permanecen vigentes, y que aún se requiere una herramienta con las características de esta para satisfacer las necesidades de investigaciones que manejan datos multimodales, la actualización de MO está fundamentada en la necesidad de seguir respondiendo a estos mismos desafíos. La actualización busca garantizar la continuidad de la herramienta, asegurando su relevancia en el contexto actual y mejorando su capacidad para integrar nuevas tecnologías y dispositivos de captura, manteniendo la flexibilidad, extensibilidad y compatibilidad con múltiples plataformas que la hicieron útil desde su creación.

1.2 DESCRIPCIÓN DEL PROBLEMA

La vigencia de MO enfrenta desafíos debido a problemas de compatibilidad entre el núcleo de la aplicación y los plugins desarrollados para extender sus funcionalidades. A lo largo del tiempo, la evolución tecnológica y los cambios en la interfaz de MO, así como la implementación de nuevos módulos como los de análisis y comunicación, han dado lugar a diversas versiones del núcleo, generando inconsistencias en la conexión y funcionamiento con los plugins existentes. Esto ha dificultado la integración de nuevas características y ha complicado el mantenimiento y uso de la aplicación. Como resultado, la capacidad de MO para adaptarse a las necesidades actuales y futuras de los usuarios se ve limitada, lo que pone en riesgo su relevancia frente a tecnologías y productos alternativos, comprometiendo el propósito original para el cual fue creado.

Con lo anterior, surge la pregunta: ¿Cómo se pueden mitigar los problemas de compatibilidad entre el núcleo de MO y los plugins, permitiendo la integración de nuevas características, para mantener la vigencia de la aplicación?

1.3 SOLUCIÓN PROPUESTA

1.3.1 Características de la solución

El producto desarrollado en este proyecto consiste en una nueva versión de la aplicación de escritorio Multimodal Observer (MO). Esta versión respeta los principios fundacionales de MO, manteniéndose fiel a una arquitectura modular y extensible que permite la integración de plugins para añadir funcionalidades específicas en distintos módulos del sistema, como la captura y visualización de datos multimodales. Como parte de esta nueva versión, se llevó a cabo una migración tecnológica completa, que implicó la reimplementación del núcleo de la aplicación utilizando tecnologías actuales, más robustas y con mayor respaldo de la comunidad. Esta renovación tecnológica permitió mejorar la organización interna del sistema, reducir el acoplamiento entre módulos, y facilitar el mantenimiento y la evolución futura de la herramienta. La actualización del núcleo se enfocó en consolidar una estructura robusta que facilite la expansión futura del sistema, permitiendo la incorporación de nuevos módulos y plugins sin necesidad de modificar el núcleo principal. En particular, se mantuvieron y reforzaron las siguientes características distintivas de la versión original (Gutiérrez Gálvez, 2016):

- **Extensible:** La solución debe permitir la integración de plugins desarrollados de forma independiente, que extienden las funcionalidades base de la aplicación. Este enfoque facilita la personalización del software según los requerimientos específicos de distintos usuarios o contextos de investigación.
- **Modular:** La arquitectura de la aplicación se organiza en módulos independientes y bien definidos, cada uno encargado de una funcionalidad específica (por ejemplo, captura, visualización, análisis o comunicación). Esta estructura modular permite desarrollar, mantener y actualizar cada componente de manera aislada, reduciendo el acoplamiento entre partes del sistema y facilitando la incorporación de nuevas funcionalidades sin afectar el funcionamiento del resto de la aplicación.
- **Multiplataforma:** La aplicación es compatible con los principales sistemas operativos: Windows, Linux y macOS. Esto tiene como objetivo permitir una adopción más amplia y facilitar su integración en distintos entornos de trabajo.
- **Soporte multilingüe:** La aplicación incluye la opción de cambiar el idioma de la interfaz, adaptándose a un público internacional. Se ofrecen los idiomas inglés y español, con la posibilidad de expandir a otros idiomas en el futuro.

1.3.2 Propósito de la solución

El propósito de este proyecto es establecer un software y entorno estable que permita garantizar la continuidad de MO. En particular, se busca resolver los problemas de compatibilidad existentes entre el núcleo de la aplicación y el desarrollo de plugins. Al abordar esta problemática, se facilita la incorporación de nuevos plugins y funcionalidades de forma más eficiente, reduciendo el tiempo invertido en resolver errores de integración y evitando dificultades en el uso de la aplicación.

Con estas mejoras, y aunque excede el alcance inmediato de este proyecto, se proyecta a corto y mediano plazo que MO pueda responder de manera efectiva a las necesidades actuales de investigadores y desarrolladores que trabajan con datos multimodales, mejorando así la experiencia de los usuarios con la aplicación.

1.4 OBJETIVOS Y ALCANCE DEL PROYECTO

1.4.1 Objetivo general

El objetivo general es actualizar el núcleo de MO a través de un proceso integral de reimplementación, refactorización y migración tecnológica para permitir la extensibilidad del software en base a plugins que dan soporte a nuevos sensores.

1.4.2 Objetivos específicos

1. Diagnosticar el estado actual de MO en sus distintas versiones identificando componentes críticos y la compatibilidad con los plugins vigentes de MO.
2. Rediseñar el núcleo de MO preservando su estructura modular y extensible.
3. Implementar el núcleo de MO a partir del rediseño, reutilización de componentes y el uso de tecnologías actualizadas
4. Implementar un instalador para el núcleo de MO y el conjunto de plugins existentes.
5. Demostrar la funcionalidad de la nueva versión mediante la creación de al menos 5 plugins.

1.4.3 Alcances

Los alcances de la solución se detallan a continuación:

- La solución se enfoca exclusivamente en la actualización del núcleo de la aplicación y los módulos base iniciales que permitan garantizar su correcto funcionamiento.
- Se proporciona un instalador para el núcleo de MO y el conjunto de plugins desarrollados.
- La aplicación continuará siendo multiplataforma, garantizando su operatividad en diferentes sistemas operativos.
- Se genera documentación técnica para el núcleo, guías de desarrollo y pautas para la creación de plugins.

Por otra parte, la solución NO contempla:

- La actualización de los módulos de análisis, comunicación y actualización.
- La retrocompatibilidad con plugins existentes.
- La transición de datos obtenidos de capturas en versiones anteriores a la desarrollada en este proyecto.
- El trabajo con datos obtenidos fuera de la aplicación.

1.5 METODOLOGÍA Y HERRAMIENTAS UTILIZADAS

1.5.1 Metodología

Para llevar a cabo la actualización del núcleo de MO, se empleó el ciclo PDCA (Plan-Do-Check-Act), complementado con las metodologías **Kanban** y Rapid Application Development (RAD) para la organización, seguimiento de tareas y desarrollo iterativo del software. El ciclo PDCA sigue cuatro pasos esenciales para llevar a cabo la actualización, mientras que Kanban facilita la visualización y gestión eficiente de las actividades, y RAD proporciona un enfoque práctico para iteraciones rápidas mediante prototipos funcionales. A continuación se detallan estas metodologías:

1.5.1.1 Ciclo PDCA (Plan-Do-Check-Act)

Es una metodología iterativa diseñada para la mejora continua de procesos, productos y servicios. El PDCA es un proceso cíclico que permite planificar, ejecutar, verificar y ajustar las acciones de manera sistemática. Esta metodología resulta útil en el contexto del desarrollo y mantenimiento de software, donde las necesidades de mejora y adaptación son constantes (Vergara, 2024).

1. **Plan (Planificar):** La planificación implica identificar problemas, establecer objetivos claros y definir un plan de acción para abordar las necesidades identificadas. Es fundamental para entender el problema principal y establecer metas alcanzables que guíen las actividades posteriores (Vergara, 2024). En el proyecto de actualización del núcleo de MO, esta etapa se centra en diagnosticar el estado actual del software y los problemas de compatibilidad con los plugins desarrollados. Esta fase se relaciona directamente con la etapa de Definición de Requisitos de la metodología RAD, donde se identifican las funcionalidades clave y se realiza un análisis preliminar del sistema, para luego agregar las tareas y subtareas asociadas a los requisitos e historias de usuario en el tablero Kanban.
2. **Do (Hacer):** La ejecución consiste en implementar las acciones planificadas, convirtiendo las ideas y estrategias en resultados tangibles. En un contexto de mejora continua, esta etapa permite aplicar cambios a pequeña escala, probando su efectividad en un entorno controlado antes de ampliarlos (Vergara, 2024). En este proyecto, la etapa de ejecución se enfocó en rediseñar y migrar el núcleo del software manteniendo su estructura modular y extensible. Además, se implementaron cambios necesarios en el núcleo para garantizar la extensibilidad del software mediante la abstracción de interfaces definidas que permiten interactuar al núcleo con los plugins. También se desarrolló un instalador para la aplicación y los plugins. Este proceso se guió por las etapas de prototipado y construcción de RAD, donde se crearon prototipos funcionales para validar y refinar iterativamente.
3. **Check (Verificar):** La verificación consiste en evaluar la efectividad de las acciones implementadas en la etapa anterior. Durante esta fase, se recopilan datos y se analizan los resultados para determinar si los cambios han cumplido con los objetivos establecidos. Esto permite identificar áreas de éxito y oportunidades de mejora antes de proceder a la etapa final (Vergara, 2024). En este proyecto, la verificación incluyó pruebas de integración, unitarias y análisis estático del código, con la finalidad de validar que el producto construido tenga una buena calidad respecto al código y que todos los componentes funcionen correctamente, tanto de forma aislada como en conjunto. En función de los resultados de los análisis de código estático, se realizaron ajustes orientados principalmente a mejorar la calidad del código fuente. Además, se llevaron a cabo pruebas de aceptación lideradas por

el profesor guía, quien actuó como encargado principal de validar el producto. Estas pruebas fueron complementadas con reuniones de seguimiento durante el desarrollo, en las cuales el profesor realizó sugerencias importantes, especialmente en aspectos relacionados con la interfaz del software. Por lo tanto, se analizaron los resultados obtenidos para confirmar que las modificaciones realizadas resolvieron los problemas de compatibilidad y cumplieron con los requerimientos. Este proceso se vinculó con la etapa de evaluación de RAD, donde los prototipos y el núcleo funcional se probaron continuamente para garantizar que las modificaciones cumplieran con los requisitos establecidos.

4. **Act (Actuar):** La etapa de acción implica tomar decisiones basadas en los resultados de la verificación. Si los objetivos se alcanzan, los cambios se integran plenamente en el sistema. Si los resultados no son satisfactorios, se realizan ajustes y se reinicia el ciclo para refinar las soluciones (Vergara, 2024). En el contexto del proyecto, esta fase incluyó la adopción de las modificaciones cuando las pruebas demostraron que eran efectivas y cumplían con lo establecido. En caso de que se identificaran problemas o áreas de mejora, se realizaron ajustes en el diseño y la implementación del núcleo, reiniciando el ciclo PDCA para refinar las soluciones hasta alcanzar los resultados deseados. En esta etapa se aseguró que el núcleo y los plugins estuvieran listos para su uso en un entorno de producción, respaldados por la documentación técnica adecuada.

1.5.1.2 Rapid Application Development (RAD)

Es una metodología de desarrollo ágil que pone énfasis en el desarrollo rápido y la iteración de prototipos funcionales. Esto permite realizar ajustes en las características del software de manera progresiva, probándolo y refinándolo hasta cumplir con las expectativas del usuario y del proyecto (Martin, 1991). En este proyecto, RAD se integró al ciclo PDCA como una metodología operativa que guió las etapas de desarrollo y validación del núcleo rediseñado de MO. Al permitir iteraciones rápidas y retroalimentación continua, RAD garantizó que el desarrollo del software se mantuviera alineado con los objetivos del proyecto.

1.5.1.3 Kanban

Esta metodología permite organizar visual y lógicamente las tareas o actividades a desarrollar, clasificándolas según su estado dentro de un flujo de trabajo. Esto ayuda a realizar un seguimiento claro del progreso e identificar en qué etapa se encuentran las tareas (Martins, 2024).

Kanban se seleccionó por su capacidad para simplificar la gestión de tareas en proyectos donde la organización es crucial para evitar retrasos y garantizar el cumplimiento de los objetivos dentro de los plazos estipulados. Su representación visual ayudó a mantener claridad sobre los avances, permitiendo un monitoreo continuo del flujo de trabajo en las etapas del ciclo PDCA.

1.5.2 Herramientas de desarrollo

Las herramientas utilizadas en este proyecto son:

- **Multimodal Observer (MO)**
- **Python v3.12**
- **Electron v35.1**
- **SonarQube**, para el análisis de calidad del código
- **Git & GitHub**, para el control de versiones y almacenamiento del código
- **Visual Studio Code**, para el desarrollo de la aplicación
- **Overleaf (LaTeX)**, para la redacción del informe de título
- **Notion**, para almacenar, escribir documentación, tomar notas durante el desarrollo de la aplicación y seguir la metodología KanBan
- **Google Drive**, para almacenar y compartir avances durante el desarrollo del proyecto
- **Figma**, para el desarrollo de mockups

1.5.3 Ambiente de desarrollo

El proyecto se llevó a cabo principalmente en el domicilio personal del autor, utilizando un PC de escritorio que cuenta con las siguientes características:

- Sistema operativo Windows 11
- Procesador Intel Core i5-9400F
- Tarjeta gráfica NVIDIA GeForce GTX 1660
- 16 GB de memoria RAM
- SSD 480 GB

1.6 ORGANIZACIÓN DEL DOCUMENTO

El resto del documento se encuentra organizado en cinco capítulos. En el Capítulo 2, "Marco Teórico", se realiza una revisión del estado actual de MO, de herramientas existentes que trabajan con datos multimodales, y de herramientas que implementan arquitecturas de plugins para extender sus funcionalidades. Luego, en el Capítulo 3, "Análisis", se realiza un diagnóstico sobre los problemas que presenta MO, se detallan los antecedentes de la aplicación, los requerimientos funcionales y no funcionales del proyecto, y se describe el proceso de prototipado de la nueva versión de MO y del instalador. A continuación, en el Capítulo 4, "Diseño y aspectos de implementación", se presenta la arquitectura de la aplicación trabajada y se describen los componentes y características de la implementación. En el Capítulo 5, "Evaluación", se analizan los resultados obtenidos en las pruebas aplicadas a la implementación. Por último, en el Capítulo 6, "Conclusiones", se exponen las conclusiones del trabajo realizado y el trabajo futuro.

Cabe destacar que se emplearon las herramientas ChatGPT-4o y Gemini 2.5 Flash como apoyo para la revisión y mejora de la redacción en los capítulos del presente documento.

CAPÍTULO 2. MARCO TEÓRICO

Este capítulo expone los principales fundamentos teóricos y técnicos relacionados a este trabajo. En primer lugar, se realiza una revisión del estado de MO antes de la realización de este trabajo; luego, se abordan conceptos clave para entender el contexto del trabajo y sus objetivos. Posteriormente, se presenta una revisión de los productos de software disponibles que abordan la problemática por la que fue construida MO, y luego productos que implementan arquitecturas de plugins para extender funcionalidades.

2.1 MARCO CONCEPTUAL

2.1.1 Estado actual de MO

Multimodal Observer (MO) fue concebido inicialmente por Gutiérrez Gálvez (2016) como un software de código abierto, modular, extensible y multiplataforma. Desarrollado en el lenguaje de programación Java en el Departamento de Ingeniería Informática de la Universidad de Santiago de Chile, bajo el alero del laboratorio de investigación y desarrollo InTeractiOn (InTeractiOn, s.f.), su propósito fundamental fue facilitar a los investigadores el trabajo con datos multimodales. Esto incluía la captura, visualización y, eventualmente, el análisis de dichos datos, buscando superar las limitaciones de herramientas existentes en términos de alto costo, especificidad de dominio, falta de soporte multiplataforma y carencia de mecanismos para extender sus funcionalidades como nuevos sensores de captura. Desde sus inicios, la arquitectura original de MO se centró en un núcleo robusto con un sistema de plugins que permitía la integración de diversas fuentes de datos, como teclado, mouse y sensores fisiológicos (Gutiérrez Gálvez, 2016). En la Figura 2.1 se puede observar una vista de la interfaz de MO en sus primeras versiones.

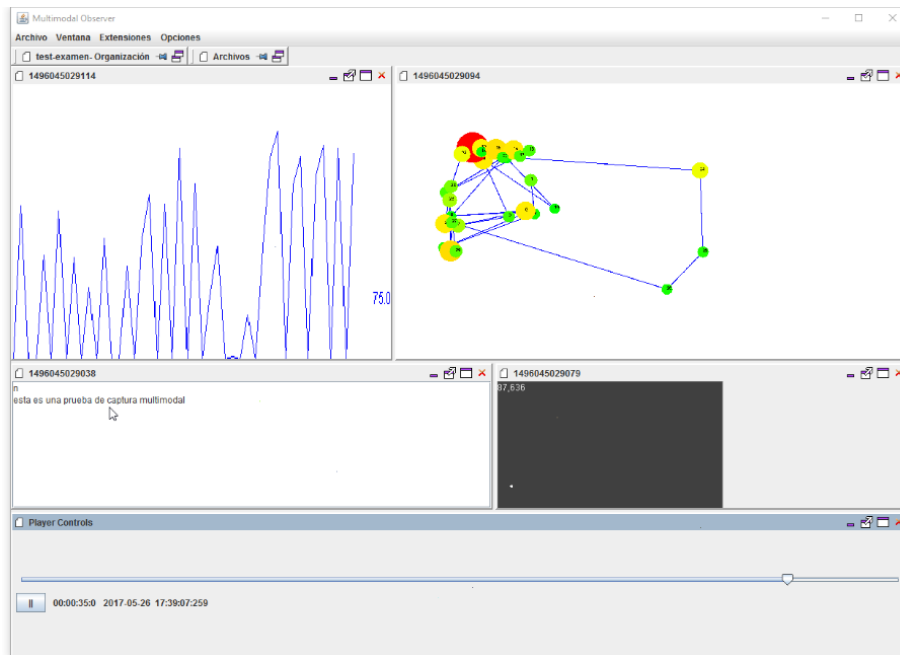


Figura 2.1: Interfaz original de MO en sus primeras versiones.
Fuente: InTeractiOn (s.f.)

Con el tiempo, MO ha incorporado significativos módulos que han expandido sus funcionalidades centrales y, a su vez, han influido en la evolución de su núcleo. Curifuta Poblete (2017) introdujo el módulo de Análisis, permitiendo el procesamiento avanzado de datos. Vilches Céspedes (2017) desarrolló el módulo de Actualización, enfocado en la gestión de extensiones, facilitando su búsqueda, descarga e instalación. Posteriormente, Cáceres Rodríguez (2018) implementó el módulo de Comunicación, que habilitó la interacción, control y monitoreo remoto en MO. La experiencia de usuario también fue un foco de desarrollo, con Polanco Berrios (2019) creando un módulo asistente (wizard) para la gestión y despliegue de protocolos de investigación, y Díaz Sandoval (2021) llevando a cabo un rediseño integral de la interfaz de usuario y el modelo de interacción. La Figura 2.2 muestra una vista de la interfaz luego del rediseño, reflejando las mejoras introducidas en términos de usabilidad e interacción.

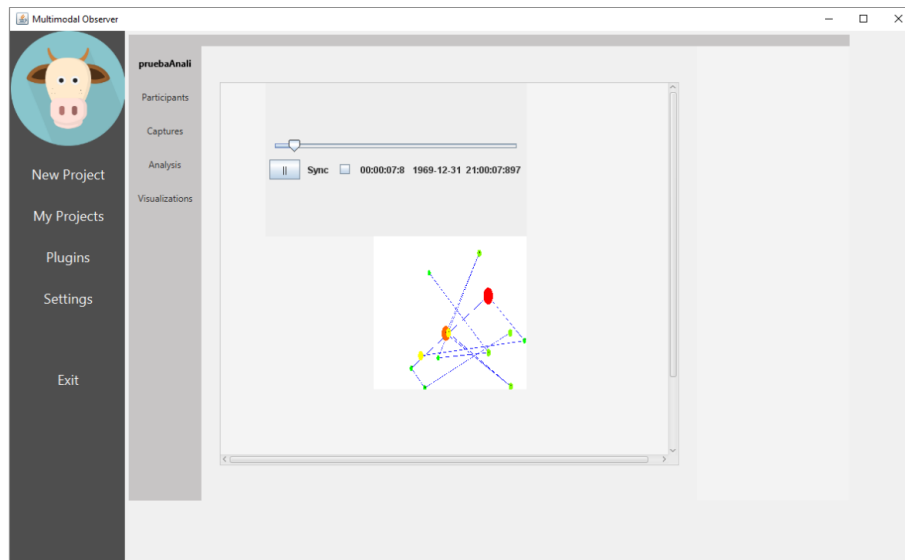


Figura 2.2: Interfaz de MO luego del rediseño del modelo de interacción.
Fuente: Jaramillo Vallejos (2024)

La arquitectura general de MO, tal como se describe en (Gutiérrez Gálvez, 2016), se organiza conceptualmente en tres capas principales: la capa del Núcleo (Core), que provee las funcionalidades base y el sistema de gestión de plugins; la Capa de Módulos, que agrupa funcionalidades específicas, contando actualmente con seis módulos principales activos (Captura, Visualización, Análisis, Comunicación, Actualización y Wizard); y la Capa de Extensiones (Plugins), donde residen los componentes desarrollados para añadir capacidades concretas a los módulos, sumando un total de 34 plugins desarrollados bajo el laboratorio InTeractiOn. En la Figura 2.3 se presenta un esquema que ilustra esta arquitectura en capas.



Figura 2.3: Arquitectura en capas de MO: Núcleo, Módulos y Extensiones.
Fuente: Huincahue Azócar (2024)

No obstante, la continua evolución del núcleo para dar soporte a estos nuevos módulos y funcionalidades ha llevado a que existan actualmente plugins que presentan incompatibilidades o no funcionan correctamente en todas las versiones de la aplicación. La Figura 2.4 resume en una

línea de tiempo los principales hitos del desarrollo de MO y sus módulos a lo largo del tiempo.

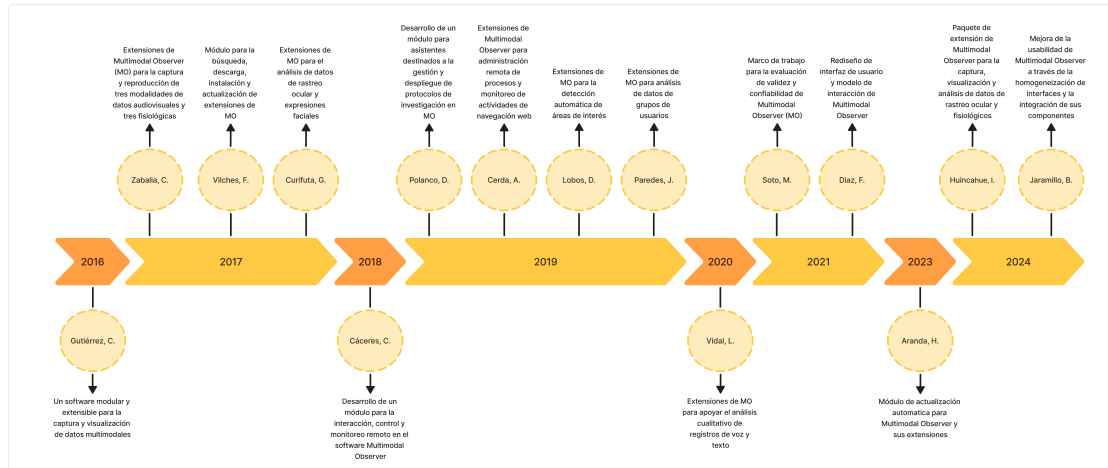


Figura 2.4: Línea de tiempo con los principales hitos del desarrollo de MO.

Fuente: Adaptado de Huincachue Azócar (2024)

2.1.2 Actualización de software

La actualización de software es el proceso de reemplazar una versión existente de un programa o sistema por una más reciente, con el fin de mejorarlo o adaptarlo. Este proceso puede abordarse mediante diversos enfoques. Los más comunes son las correcciones incrementales y parches, que aplican cambios puntuales para resolver errores o añadir pequeñas mejoras sobre la base existente, usualmente reflejados en cambios de versión de tipo Parche (*Patch*) o Menor (*Minor*) según el Versionado Semántico (*SemVer*) (Preston-Werner, s.f.; Syal, 2023). En situaciones donde se requieren cambios más sustanciales, se puede optar por una evolución mayor del sistema, que podría implicar la reescritura de partes importantes del sistema o la introducción de cambios arquitectónicos, frecuentemente resultando en cambios incompatibles con versiones anteriores y correspondiendo a un cambio de versión Mayor (*Major*) (Preston-Werner, s.f.; Syal, 2023). Finalmente, en casos más extremos o ante obsolescencia tecnológica, se puede recurrir a una reimplementación completa, que consiste en construir una nueva versión del software desde cero, a menudo utilizando nuevas tecnologías o herramientas base.

2.1.3 Extensibilidad

La extensibilidad en el desarrollo de software se refiere a la capacidad de un sistema para incorporar nuevas funcionalidades o características, sin necesidad de modificar el código fuente del núcleo existente. Es una cualidad de diseño que permite al software adaptarse a requisitos cambiantes y evolucionar a lo largo del tiempo sin necesidad de hacer cambios significativos en el sistema base (Prismic, 2024).

Las ventajas de un software diseñado con extensibilidad son considerables. En primer lugar, facilita la adaptabilidad y preparación para el futuro, permitiendo que el sistema crezca y se integre de manera eficiente (Prismic, 2024; VPN Unlimited, s.f.). Esto también conlleva a una reducción en los tiempos y costos de desarrollo a largo plazo, ya que las nuevas características pueden construirse sobre la base extensible existente en lugar de requerir modificaciones profundas (Prismic, 2024; VPN Unlimited, s.f.). Asimismo, la extensibilidad promueve la innovación, al permitir que tanto los desarrolladores originales como terceros puedan ampliar las capacidades del sistema para satisfacer necesidades específicas o explorar nuevas funcionalidades (Prismic, 2024).

2.1.4 Modularidad

La modularidad es un principio de diseño de software que consiste en dividir un sistema de software en componentes más pequeños, denominados módulos, que son relativamente independientes entre sí. Cada módulo encapsula un conjunto específico de funcionalidades y responsabilidades, interactuando con otros módulos a través de interfaces bien definidas (Brategren, s.f.).

Adoptar un diseño modular en el software ofrece múltiples beneficios. Una de las principales ventajas es la mejora en la mantenibilidad, ya que los cambios o correcciones de errores dentro de un módulo tienen un impacto limitado o nulo en el resto del sistema, facilitando su evolución. La modularidad también promueve la reusabilidad de código, dado que los módulos con funcionalidades genéricas pueden ser utilizados en diferentes partes del sistema o incluso en otros proyectos. Adicionalmente, simplifica la comprensión del sistema al permitir que los desarrolladores se enfoquen en partes más pequeñas y manejables. Finalmente, un sistema modular tiende a ser más flexible y escalable, ya que los módulos pueden ser modificados, reemplazados o añadidos con mayor facilidad para adaptar el software a nuevas necesidades (Brategren, s.f.).

2.1.5 Plugins

Un plugin, también conocido como complemento o extensión, es un componente de software que añade una funcionalidad específica a una aplicación existente. Estos componentes operan como módulos relativamente independientes que se integran en el programa principal, permitiendo ampliar o modificar su comportamiento usualmente sin necesidad de alterar el código fuente de la aplicación base (BasuMallick, 2024; IONOS, 2020).

La utilización de una arquitectura basada en plugins ofrece diversas ventajas en el desarrollo y mantenimiento de software. Una de las principales es la modularidad, ya que permite encapsular funcionalidades específicas en componentes separados, facilitando su desarrollo, prueba y actualización de manera aislada sin afectar al núcleo de la aplicación (IONOS, 2020). Esto también contribuye a la personalización y flexibilidad, dado que los usuarios o desarrolladores pueden seleccionar e instalar únicamente los plugins que necesitan, adaptando el software a sus requerimientos particulares.

2.2 ESTADO DEL ARTE

2.2.1 Software para el trabajo multimodal

2.2.1.1 NoldusHub

NoldusHub, desarrollado por Noldus Information Technology, se presenta como una plataforma robusta para la investigación del comportamiento humano, enfocada en la integración y sincronización de datos multimodales (Noldus, s.f.d). Esta herramienta opera como una solución comercial, lo que implica costos de licenciamiento para su utilización. La plataforma está concebida para simplificar la complejidad inherente a los estudios multimodales, centralizando la gestión de diversos dispositivos y flujos de datos.

Una de las características centrales de NoldusHub es su capacidad para integrar y sincronizar automáticamente diversas fuentes de datos, tales como rastreadores oculares, múltiples sensores fisiológicos validados por Noldus (EEG, GSR, ECG, EMG), y grabaciones de audio y video (Noldus, s.f.d,s). La plataforma ofrece un punto de control único para la calibración de dispositivos,

la monitorización en tiempo real de las señales a través de una función denominada "*system heartbeat*" que verifica la calidad de los datos y los sincroniza (Noldus, s.f.h).

2.2.1.2 iMotions

iMotions es una plataforma de software desarrollada con un fuerte enfoque en la investigación del comportamiento humano y la neurociencia aplicada, permitiendo la integración y sincronización de una amplia gama de sensores biométricos (iMotions, s.f.a). Esta solución se ofrece bajo un modelo comercial, con diferentes módulos y opciones de licenciamiento según las necesidades académicas o empresariales (iMotions, s.f.c).

La plataforma destaca por su capacidad para integrar y sincronizar automáticamente una gran variedad de sensores, incluyendo rastreadores oculares, electroencefalografía (EEG), análisis de expresiones faciales, entre otros (iMotions, s.f.a). Estos sensores son construidos en base a una arquitectura modular con un sistema core que permite integrar distintos módulos asociados a los sensores o a funcionalidades de análisis (iMotions, s.f.b). Cabe mencionar que cada módulo cuenta con un precio asociado para poder ser utilizado (iMotions, s.f.c).

2.2.1.3 The Observer XT

The Observer XT, desarrollado por Noldus Information Technology, es un software de escritorio para Windows utilizado en la investigación del comportamiento tanto humano como animal (Noldus, s.f.b,s). Se trata de una solución comercial, cuyos costos varían según los módulos y el tipo de licencia adquirida (Noldus, s.f.a). La herramienta está diseñada para la codificación detallada y el análisis cuantitativo de secuencias de comportamiento, permitiendo la integración sincronizada de múltiples flujos de datos, como video, audio, y datos fisiológicos provenientes de sistemas externos como rastreadores oculares o dispositivos de EEG (Noldus, s.f.b,s). The Observer XT cuenta con un sistema modular que permite extender sus funcionalidades base con módulos adicionales para tareas específicas, como la integración de datos externos o la colaboración entre codificadores (Noldus, s.f.e,s).

2.2.1.4 OpenSignals

OpenSignals es un software desarrollado por PLUX Biosignals, diseñada para la captura, visualización y procesamiento de bioseñales (PLUX Biosignals, s.f.a). Esta herramienta es compatible con todos los dispositivos de adquisición de bioseñales de PLUX, como las líneas de productos biosignalsplux y BITalino. El software base de OpenSignals se puede descargar gratuitamente desde el sitio web del proveedor (PLUX Biosignals, s.f.a), aunque algunas funcionalidades avanzadas de análisis se ofrecen a través de *Add-ons* o complementos que son de pago (PLUX Biosignals, s.f.b).

OpenSignals es una solución multiplataforma, con versiones disponibles para Windows, macOS y Android, y una versión discontinuada para Linux, lo que facilita su uso en diversos entornos de investigación y desarrollo (PLUX Biosignals, 2025). La herramienta está destinada a usarse en investigación y educación en ciencias de la vida (PLUX Biosignals, 2021). Su enfoque principal es la adquisición y visualización de bioseñales, con capacidades de análisis que pueden ser expandidas mediante los mencionados *Add-ons*.

2.2.2 Software con implementación de plugins

2.2.2.1 OBS Studio

OBS Studio (Open Broadcaster Software) es una aplicación de código abierto y multiplataforma, diseñada para grabar y transmitir contenido en vivo. Está desarrollada principalmente en C/C++ y ofrece una interfaz de usuario construida con Qt (Obsproject, s.f.).

La arquitectura de plugins de OBS Studio está diseñada para ser modular, permitiendo que gran parte de sus funcionalidades centrales y complementarias sean implementadas como plugins. Estos plugins son, en esencia, bibliotecas dinámicas (archivos .dll en Windows, .so en Linux, o .dylib en macOS) que la aplicación carga durante el inicio (Bailey, 2025). OBS Studio expone una API en C (libobs) que los plugins utilizan para registrarse e interactuar con el núcleo del software. Esta API define las estructuras y funciones necesarias para que los plugins puedan, por ejemplo, proveer nuevas fuentes de captura, filtros de audio/video, encoders para diferentes formatos o servicios de salida para interactuar con plataformas de streaming (Bailey, 2025).

OBS Studio facilita la gestión de plugins permitiendo a los usuarios instalarlos simplemente copiando los archivos del plugin en directorios designados (OBS, 2022). Además, mantiene una

lista de plugins populares y oficialmente soportados que los usuarios pueden descargar, muchos de los cuales vienen con sus propios instaladores para facilitar su integración.

2.2.2.2 Visual Studio Code

Visual Studio Code (VS Code) es un editor de código multiplataforma, desarrollado por Microsoft y disponible de forma gratuita (VS Code, s.f.). VS Code está construido utilizando tecnologías web mediante el framework Electron, que permite el desarrollo de aplicaciones de escritorio con HTML, CSS y JavaScript (Electron, s.f.).

VS Code está diseñado con la extensibilidad como principio fundamental, y gran parte de sus características centrales están implementadas como extensiones. Estas extensiones interactúan con el editor a través de una API que permite un alto grado de personalización, desde modificar la interfaz de usuario con temas y vistas personalizadas o agregar componentes, hasta añadir soporte para nuevos lenguajes de programación (incluyendo resaltado de sintaxis y verificación) e integrar depuradores para diversos entornos de ejecución (VS Code, 2025).

El descubrimiento, la instalación y la gestión de extensiones se centralizan en el *"Marketplace"* de extensiones de VS Code, integrado directamente en el editor para facilitar el acceso, la instalación y la actualización directa de las extensiones publicadas.

2.2.2.3 WordPress

WordPress es un sistema de gestión de contenidos (CMS) cuya funcionalidad se extiende de manera primordial a través de plugins. Estos constituyen paquetes de código, desarrollados principalmente en PHP, pudiendo incorporar también recursos como CSS, JavaScript e imágenes (WordPress, 2014b).

La arquitectura extensible de WordPress se basa en un sistema de *"hooks"* (ganchos), que se dividen en *"actions"* (acciones) y *"filters"* (filtros). Las *"actions"* permiten a los plugins ejecutar código en momentos específicos del ciclo de vida de WordPress o añadir nuevas capacidades, mientras que los *"filters"* ofrecen la posibilidad de modificar datos antes de su almacenamiento o presentación al usuario. Cada plugin se define, como mínimo, por un archivo PHP principal que incluye una cabecera con metadatos específicos, y puede hacer uso de diversas APIs proporcionadas por WordPress, como la Options API para la gestión de datos o la Plugin API para funciones auxiliares (WordPress, 2014a).

Los plugins se instalan en un directorio específico y se gestionan (activación, desactivación y eliminación) desde el panel de administración de WordPress. El repositorio oficial de WordPress es el principal canal de distribución para estos plugins (WordPress, 2014a).

2.3 RESUMEN

En el presente capítulo se establecen los cimientos conceptuales y técnicos del trabajo. En primer lugar, se realiza una revisión del estado actual de la herramienta Multimodal Observer (MO), y posteriormente se definen conceptos cruciales como actualización de software, extensibilidad, modularidad y plugins. Posteriormente, se presenta un estado del arte que examina tanto software comparable a MO en el ámbito multimodal como ejemplos de aplicaciones con arquitecturas de plugins consolidadas.

El Marco Conceptual profundiza inicialmente en Multimodal Observer (MO), describiendo su origen como un software modular en Java, orientado a la investigación multimodal, su evolución mediante módulos clave y su arquitectura de tres capas, además de señalar los desafíos de la compatibilidad actual de sus plugins. Posteriormente, se definen conceptos esenciales para el trabajo: la actualización de software, detallando sus diversos enfoques; la extensibilidad, como la capacidad de un sistema para incorporar nuevas funciones sin modificar su núcleo; la modularidad, referente a la división de un sistema en componentes independientes para facilitar su mantenimiento y evolución; y los plugins, como componentes que añaden funcionalidades específicas a una aplicación base.

En la sección de Estado del Arte, se realiza una revisión de dos categorías de software. En primer lugar, se examinan herramientas existentes para el trabajo con datos multimodales, con el fin de contextualizar las soluciones disponibles para problemáticas similares a las que MO aborda. Esta revisión se resume en la Tabla 2.1. En segundo lugar, se analizan diversas aplicaciones de software que implementan arquitecturas de plugins, con el objetivo de identificar distintos enfoques y buenas prácticas en el diseño de sistemas extensibles. La síntesis de estas herramientas se presenta en la Tabla 2.2.

Tabla 2.1: Software para el trabajo multimodal.

Herramienta	Extensibilidad de Software	Licencia	Plataforma Disponible
NoldusHub	No especificado	Comercial	Windows
iMotions	Sí, de manera interna	Comercial (más pago por módulos)	Windows
The Observer XT	Sí, de manera interna	Comercial (más pago por módulos)	Windows
OpenSignals	Sí, de manera interna	Núcleo gratuito, Add-ons de pago	Windows, macOS, Android (Linux discontinuado)

Fuente: Elaboración propia, 2025.

Tabla 2.2: Software con implementación de plugins.

Herramienta	Tecnología Base / Tipo de Extensión	Mecanismo de Extensibilidad Principal	Gestión/Distribución de Extensiones
OBS Studio	C/C++, Qt / Bibliotecas dinámicas	API en C (libobs) para registrar e interactuar.	Copia de archivos, lista oficial, instaladores.
Visual Studio Code	Electron (HTML,CSS,JS) / Extensiones (JS/TS)	API de Extensión para personalizar UI, lenguajes, etc.	Marketplace web e integrado en el editor.
WordPress	PHP / PHP, CSS, JS, imágenes	Sistema de "Hooks" (actions y filters), APIs internas.	Directorio en servidor WordPress y Repositorio oficial.

Fuente: Elaboración propia, 2025.

CAPÍTULO 3. ANÁLISIS

3.1 DIAGNÓSTICO

La presente sección detalla el diagnóstico realizado sobre el estado actual del software Multimodal Observer (MO) en sus diversas versiones. Este análisis es crucial y se alinea directamente con el primer objetivo específico del proyecto: "Diagnosticar el estado actual de MO en sus distintas versiones identificando componentes críticos y la compatibilidad con los plugins vigentes de MO". El propósito es identificar con precisión y a nivel de código las problemáticas previamente reconocidas, con el fin de comprender en detalle su origen, impacto y comportamiento dentro de la arquitectura del sistema. Esta comprensión profunda es necesaria para fundamentar adecuadamente los cambios requeridos en el núcleo del proyecto, garantizando soluciones efectivas y evitando que estas dificultades vuelvan a surgir en futuras versiones.

3.1.1 Revisión evolución de MO

Para llevar a cabo el diagnóstico asociado a la evolución del software Multimodal Observer (MO), se realizó una revisión minuciosa del código fuente correspondiente a distintas versiones del sistema. Se empleó la herramienta Meld para comparar detalladamente archivos y directorios, lo que permitió identificar las diferencias entre las siguientes versiones:

- **Versión inicial:** Corresponde al desarrollo original de MO Gutiérrez Gálvez (2016).
- **Versión con módulo análisis:** Integra capacidades para el procesamiento de datos Curifuta Poblete (2017)
- **Versión con módulo de comunicación:** Introduce funcionalidades para la interacción, control y monitoreo remoto Cáceres Rodríguez (2018).
- **Versión con la nueva interfaz:** Es la versión más reciente, resultado de esfuerzos por modernizar la experiencia de usuario mediante un rediseño de la interfaz gráfica Díaz Sandoval (2021).

El análisis reveló que, especialmente entre la versión que incorporó el módulo de análisis y la que introdujo el módulo de comunicación, se produjeron alteraciones sustanciales en el núcleo del sistema. Aunque estos cambios buscaban ampliar las capacidades de MO, también generaron consecuencias directas sobre la arquitectura de plugins y su compatibilidad:

- **Modificaciones en los Puntos de Extensión (*Extension Points*):** Se identificó una alteración en la forma en que los plugins se integran con los puntos de extensión definidos por el núcleo. Estas modificaciones en la estructura generaron incompatibilidades directas con plugins que habían sido desarrollados y probados sobre versiones anteriores del núcleo.
- **Incremento del acoplamiento entre Módulos:** Un hallazgo crítico fue la introducción de dependencias del módulo de comunicación dentro del módulo de captura. Este entrelazamiento obliga a que las extensiones de captura más recientes deban contemplar e implementar interfaces y lógica provenientes del módulo de comunicación, lo que aumenta la complejidad en el desarrollo y mantenimiento de dichos plugins, y se desvía del principio de bajo acoplamiento y división de responsabilidades entre módulos y plugins.
- **Reestructuración de la Gestión de Plugins:** Se introdujeron *observers* para el monitoreo dinámico del estado de los plugins, una nueva clase *PluginData* para centralizar la administración de su información, y nuevos métodos para asignación de configuraciones en formato XML y desinstalación de plugins. Además, se añadieron metadatos con atributos como autor, contacto y sitio web.

Durante la transición desde la versión con módulo de comunicación hasta la más reciente con la nueva interfaz gráfica, los cambios se enfocaron en aspectos visuales y en la experiencia de usuario, mediante la migración de componentes de la biblioteca Java Swing a JavaFX. Sin embargo, esta transición no estuvo exenta de problemas funcionales. Jaramillo Vallejos (2024) identificó que la coexistencia de componentes Swing (en plugins antiguos) y JavaFX (en la nueva interfaz) generó problemas de rendimiento y usabilidad. Adicionalmente, el diagnóstico actual evidenció un fallo específico en la gestión de proyectos, que impedía la correcta visualización de proyectos creados con versiones anteriores.

3.1.2 Revisión de compatibilidad con Plugins

La evolución del núcleo y la incorporación de nuevos módulos en MO, descrita previamente, ha tenido un impacto directo y negativo sobre el ecosistema existente de plugins. Los cambios estructurales, la introducción de nuevas dependencias entre módulos y las modificaciones en la gestión de extensiones han generado un entorno inconsistente, en el que la funcionalidad de un plugin depende estrictamente de la versión del núcleo para la cual fue desarrollado.

Con el fin de caracterizar y cuantificar este impacto, se llevó a cabo una revisión de la compatibilidad de los plugins de Captura, Visualización y Análisis a través de las distintas

versiones de MO evaluadas. A continuación, se presentan los hallazgos organizados por categoría:

3.1.2.1 Plugins de Captura

Los plugins de captura mostraron una mejor resistencia frente los cambios en comparación con otras categorías. Sin embargo, no estuvieron exentos de problemas:

- Se identificaron errores de compilación recurrentes en varios plugins, principalmente por la ausencia de paquetes requeridos, archivos de construcción (*build*) incorrectos o la falta de archivos JAR precompilados, lo que dificulta su integración y uso inmediato.
- Un fallo notable, especialmente pronunciado en la versión con la nueva interfaz gráfica, fue la imposibilidad de guardar correctamente las configuraciones de los plugins, impidiendo la reutilización de ajustes y afectando la eficiencia del flujo de trabajo.
- Se detectaron problemas puntuales como el caso del plugin de captura de cámara, que en ciertas configuraciones generaba archivos de video corruptos, inutilizando completamente los datos recolectados.

3.1.2.2 Plugins de Visualización

Esta categoría de plugins fue la más afectada por la evolución de la interfaz de MO. Los hallazgos revelan una incompatibilidad casi total con las versiones más recientes:

- Ningún plugin de visualización original funcionó correctamente en la versión con la nueva interfaz. Como fue analizado en profundidad por Jaramillo Vallejos (2024), este fallo masivo se debe al conflicto técnico entre los componentes Java Swing, utilizados en la implementación original de los plugins, y la nueva interfaz desarrollada en JavaFX. Esta incompatibilidad genera errores de renderizado y rendimiento que hacen que los plugins sean inutilizables.
- De manera similar a los plugins de captura, se encontraron problemas de compilación por falta de recursos o definiciones inadecuadas en los archivos de construcción. Adicionalmente, se observaron errores en tiempo de ejecución en plugins específicos como

"Visualization Plugin Camera" y "Web Browse Activity Visualization", relacionados con la gestión de librerías externas.

3.1.2.3 Plugins de Análisis

Los plugins de análisis, presentaron los siguientes problemas:

- La principal barrera fue la ausencia generalizada de plugins ya compilados o la falta de las librerías necesarias para su compilación, lo que en la práctica impide su uso por parte de un usuario final.
- En aquellos casos donde se logró compilar un plugin manualmente, se constató su incompatibilidad con las versiones más recientes. Específicamente, en la versión con el módulo Comunicación, la aplicación experimentaba un fallo crítico durante el inicio al intentar cargar estos plugins, con un error originado en la clase *PluginRegistry*.
- El plugin "Facial Analysis" no pudo ser probado funcionalmente debido a que su operación dependía de una clave de API para el servicio externo Kairos, el cual ya no se encuentra disponible bajo las mismas condiciones que al momento de su desarrollo.
- Al igual que en el caso de los plugins de visualización, se produjeron fallos con la nueva interfaz desarrollada, presentando errores relacionados con JavaFX.

3.1.3 Conclusión

En síntesis, el diagnóstico realizado confirma que la evolución no controlada del núcleo de MO ha provocado una inconsistencia crítica en su ecosistema de extensiones. Las incompatibilidades identificadas no son casos aislados, sino síntomas de un problema sistémico que compromete los principios de modularidad y escalabilidad de la plataforma. El estado actual, caracterizado por la inoperatividad casi total de los plugins de visualización y análisis en las versiones más recientes, justifica y orienta técnicamente la necesidad de una actualización integral del núcleo de MO, que permita restaurar la coherencia arquitectónica y asegurar la sostenibilidad del sistema a largo plazo.

3.2 CAPTURA DE REQUERIMIENTOS

El proceso de captura de requerimientos para la actualización del núcleo de Multimodal Observer (MO) se llevó a cabo mediante un enfoque multifacético, diseñado para asegurar una cobertura completa y actualizada de las necesidades del proyecto. Este proceso no solo se centró en la consolidación de funcionalidades previas, sino también en la incorporación de nuevas características y en la alineación con el estado del arte de herramientas similares. A continuación, se detallan las etapas y fuentes de información utilizadas.

En primera instancia, se realizó una revisión de los requerimientos funcionales y no funcionales que fueron recopilados durante el desarrollo de la versión original de la aplicación (Gutiérrez Gálvez, 2016). De este documento se extrajeron los requerimientos iniciales que definieron el propósito y alcance del sistema original. Esto permitió establecer una línea base de las funcionalidad que debían ser mantenidas, mejoradas o reevaluadas en la nueva versión.

Adicionalmente, se integraron directrices y aspectos clave mencionados por el profesor guía a lo largo del desarrollo del proyecto. Entre ellas, se destacó como un requerimiento prioritario e indispensable la integración de los protocolos de investigación del *Wizard* desarrollado por Polanco Berrios (2019), ubicándolo dentro del módulo de organización de la herramienta.

Con el objetivo de asegurar la competitividad y relevancia de la herramienta, se llevó a cabo un análisis de aplicaciones destacadas en el estado del arte, tales como OBS (Open Broadcaster Software), NoldusHub y OpenSignals. Este estudio permitió identificar características, flujos de trabajo y paradigmas de interacción en el dominio de la captura y visualización de datos multimodales. Los hallazgos de este análisis no solo inspiraron la implementación de ciertos requerimientos, sino que también aportaron una perspectiva valiosa sobre aspectos en el diseño de la interfaz y la arquitectura del sistema.

Finalmente, todos los requisitos recopilados de las fuentes mencionadas fueron reescritos con el propósito de unificar el lenguaje, eliminar ambigüedades y asegurar que cada requisito estuviera definido de manera clara, concisa y verificable, facilitando así las posteriores fases de diseño, desarrollo y pruebas del sistema.

3.2.1 Requerimientos funcionales

A continuación, en la Tabla 3.1, se presenta un resumen de los requerimientos funcionales identificados para el sistema. Estos definen las capacidades y operaciones específicas que MO debe ser capaz de realizar. La tabla sintetiza la cantidad de requerimientos por módulo junto con

algunos ejemplos representativos, con el fin de proporcionar una visión clara y organizada del comportamiento esperado de la aplicación. Para una descripción completa de cada requerimiento y su historia de usuario asociada, revisar el Apéndice A.

Tabla 3.1: Resumen de requerimientos funcionales por módulo.

Módulo	Cantidad de RF	Ejemplos de Requerimientos
Organización	21	Crear, editar, eliminar y proteger proyectos, participantes y protocolos de investigación.
Núcleo	11	Agregar, eliminar y configurar plugins; mantener configuración de ventanas; cambiar el idioma de la aplicación.
Captura	9	Iniciar, pausar, reanudar y detener capturas; visualizar información de sesiones capturadas; configurar teclas rápidas.
Visualización	8	Reproducir sesiones; configurar la visualización por plugin; mover, ocultar o abrir plugins de reproducción en ventanas independientes.
Plugin de captura	3	Capturar eventos del teclado, mouse y pantalla.
Plugin de reproducción	3	Visualizar datos de teclado, mouse y video capturado.
Total	55	

Fuente: Elaboración propia, 2025.

3.2.2 Requerimientos no funcionales

En la Tabla 3.2 se especifican los requerimientos no funcionales que rigen al sistema. En su mayoría, estos derivan de los establecidos en la implementación original de la herramienta.

Tabla 3.2: Requerimientos NO Funcionales.

ID	Requerimiento No Funcional
RNF-01	El sistema debe ser multiplataforma
RNF-02	El sistema debe ser modular y extensible, permitiendo la integración de nuevas funcionalidades y módulos
RNF-03	Se debe soportar la incorporación de nuevos idiomas a la aplicación
RNF-04	Se debe incluir documentación para los desarrolladores, incluyendo guías sobre la creación e integración de plugins o módulos
RNF-05	El sistema debe permitir agregar plugins en tiempo de ejecución
RNF-06	Los datos capturados se deben mostrar de manera consistente y sincronizada

Fuente: Adaptado de Gutiérrez Gálvez (2016).

3.3 FACTIBILIDAD TÉCNICA

Antes de proceder a realizar la actualización mediante una migración de las tecnologías usadas en la versión original de MO, se realizó un análisis de factibilidad técnica para seleccionar el conjunto de herramientas y tecnologías más adecuadas para la migración y actualización del núcleo de MO. Esta sección describe las tecnologías seleccionadas, justifica la elección frente a otras alternativas y valida la arquitectura propuesta a través de casos de éxito consolidados en la industria.

3.3.1 Criterios para la Selección Tecnológica

La elección de tecnologías para actualizar MO buscó principalmente mantener las características clave de MO: ser multiplataforma, modular y extensible. Una característica esencial de MO es su capacidad de ejecutarse en múltiples sistemas operativos, como Windows, macOS y Linux, permitiendo a distintos usuarios acceder fácilmente a la aplicación sin restricciones.

Otro criterio importante fue contar con comunidades activas y grandes en torno a estas tecnologías. Esto garantiza acceso a documentación, ejemplos, librerías disponibles y soporte técnico constante, facilitando así el desarrollo y mantenimiento del sistema en el largo plazo.

Además, era fundamental que las tecnologías elegidas permitieran desarrollar una aplicación modular y fácilmente extensible. Esto implica separar claramente la interfaz gráfica de la lógica interna, facilitando la incorporación de nuevas funcionalidades mediante plugins y garantizando que MO pueda adaptarse fácilmente a futuros requerimientos.

Por último, se buscó seleccionar tecnologías ya validadas en aplicaciones exitosas, lo cual garantiza que estas herramientas sean confiables y efectivas en contextos reales similares al que enfrenta MO.

3.3.2 Tecnologías Evaluadas y Comparación

Durante la evaluación de tecnologías para migrar MO, se consideraron varias opciones:

- **Desarrollo basado en OBS Studio:** OBS Studio es una aplicación robusta para captura audiovisual y streaming, construida principalmente en C. Cumple bien con ser multiplataforma y tiene una arquitectura modular y extensible a través de plugins. También

cuenta con una buena documentación. Sin embargo, su enfoque es muy específico hacia transmisiones en vivo y no hacia la captura y manejo de distintos tipos de datos multimodales, limitando su uso en contextos más amplios como los que enfrenta MO. Además, la curva de aprendizaje sería alta, al requerir entender profundamente la lógica interna de OBS.

- **Python con bibliotecas gráficas:** Python cumple completamente con ser multiplataforma, modular y extensible, con una comunidad activa y gran cantidad de documentación disponible (Python, s.f.). Además cuenta con una gran cantidad de paquetes enfocados en análisis de datos y manejo de dispositivos externos, lo cual es ideal para las funcionalidades clave de MO. Sin embargo, las bibliotecas gráficas más comunes como Tkinter y PyQt producen interfaces menos modernas y atractivas. Además, trabajar con estas herramientas gráficas es menos habitual, complicando el desarrollo visual de plugins.
- **Tauri:** Tauri, basado en Rust, cumple con ser multiplataforma y ofrece un rendimiento superior en comparación a tecnologías como Electron, con menor consumo de recursos (Tauri, 2024; Yegualp, 2024). No obstante, su principal desventaja es la menor madurez y una comunidad menos desarrollada, dificultando el acceso a documentación, ejemplos y soporte. A pesar de esto, Tauri podría ser una alternativa viable en el futuro si se busca mejorar el rendimiento y reducir el consumo de recursos.

3.3.3 Tecnologías Seleccionadas y Justificación

Las tecnologías finalmente elegidas fueron Electron, React y Python, porque combinadas cumplen todos los criterios clave definidos.

- **Electron:** Es un *framework* basado en Chromium que permite crear aplicaciones de escritorio multiplataforma usando tecnologías web comunes como HTML, CSS y JavaScript. Actúa como un contenedor que ejecuta la interfaz de usuario como una ventana nativa en Windows, macOS y Linux, cumpliendo así con el requisito de multiplataforma (OpenJS Foundation, s.f.). Electron tiene una comunidad amplia, documentación abundante y facilita una arquitectura modular y extensible. Su ventaja clave para MO radica en la facilidad de desarrollar interfaces visuales modernas e interactivas, aprovechando tecnologías web ampliamente utilizadas. Sin embargo, una desventaja relevante es que puede consumir más recursos debido a su dependencia en Chromium (Yegualp, 2024). Por otro lado, algunas aplicaciones conocidas que respaldan la eficacia de Electron son Visual Studio Code, con

su robusta arquitectura basada en plugins, y Streamlabs OBS, que combina la robustez técnica de OBS con una interfaz moderna en Electron.

- **React:** Es una biblioteca de JavaScript diseñada para facilitar la creación de interfaces de usuario mediante componentes modulares reutilizables. Cada componente administra su propio estado, permitiendo interfaces complejas e interactivas de forma ordenada y eficiente (Meta Platforms, s.f.). React satisface los criterios de modularidad y extensibilidad, y su extensa comunidad proporciona soporte, documentación, y librerías diversas. Para MO, React ofrece la ventaja de poder construir fácilmente interfaces modernas y adaptables que simplifican la visualización y manejo de datos multimodales.
- **Python:** Como se mencionó anteriormente, Python cumple con todos los criterios fundamentales (multiplataforma, modularidad, extensibilidad y comunidad activa) (Python, s.f.). Su uso en MO se orienta principalmente al análisis y gestión de datos, la comunicación con dispositivos externos y el procesamiento estadístico. Si bien una de sus principales desventajas es la dificultad para construir interfaces visuales modernas mediante sus bibliotecas gráficas tradicionales, esta limitación se resuelve integrando Electron y React para la creación de interfaces visuales modernas. La efectividad de Python en sistemas similares se ve respaldada por herramientas como OpenSignals, orientada a la captura y análisis de bioseñales, y Sublime Text, un editor de código que utiliza una arquitectura extensible basada en plugins desarrollados en Python. Estas aplicaciones validan su capacidad para operar en contextos exigentes y modulares como el que plantea MO.

3.4 PROTOTIPADO

En esta sección se presentan los prototipos elaborados, cada prototipo fue concebido como una iteración que buscó cumplir con objetivos específicos, siempre orientados a satisfacer los requisitos funcionales y no funcionales definidos en la sección anterior. Mientras que el primer prototipo tuvo como propósito principal validar la viabilidad técnica de la solución planteada, los siguientes prototipos se enfocaron en incrementar gradualmente las capacidades de la aplicación, permitiendo su evolución controlada hasta alcanzar el producto final.

Cabe destacar que, al finalizar cada iteración correspondiente a un prototipo funcional (excluyendo el primero), se llevó a cabo un proceso de documentación y validación conforme a la metodología establecida. Este proceso incluyó el registro de los cambios implementados, así como la documentación tanto en el código como en el repositorio fuente de la aplicación y los plugins desarrollados.

A continuación, se describen los prototipos construidos, detallando para cada uno sus objetivos, funcionalidades implementadas y principales resultados.

3.4.1 Primer prototipo: Replicación de funcionalidades críticas

El primer prototipo tuvo como objetivo principal determinar la viabilidad técnica de replicar las funcionalidades esenciales de MO utilizando las nuevas tecnologías seleccionadas: Electron con React para la interfaz de usuario y Python para la capa de captura. Antes de continuar con el desarrollo del sistema completo, era fundamental verificar que los conceptos clave del sistema original, como la arquitectura modular basada en plugins, la captura de datos multimodales, la comunicación y la visualización podían ser implementadas con éxito en el nuevo entorno tecnológico.

La Tabla 3.3 presenta un resumen de este prototipo, sus objetivos, una descripción general y los requisitos funcionales y no funcionales abordados.

Tabla 3.3: Primer prototipo - Replicación de funcionalidades críticas.

ID Prototipo	P-01
Nombre	Replicación de funcionalidades críticas
Objetivos	Validar la viabilidad técnica de la nueva arquitectura. Implementar un sistema de plugins en captura y visualización. Replicar funcionalidades esenciales de MO utilizando Electron, React y Python.
Descripción	Prototipo orientado a la validación temprana de la arquitectura mediante la construcción de plugins básicos para la captura de eventos de mouse, teclado y video, así como la visualización de estos datos en tiempo real. Se implementó comunicación HTTP, sockets, gestión de procesos y un menú System Tray.
Requisitos Funcionales	RF-22, RF-33, RF-36, RF-50, RF-51, RF-52, RF-53, RF-54, RF-55
Requisitos No Funcionales	RNF-01, RNF-02, RNF-05, RNF-06

Fuente: Elaboración propia, 2025.

Este prototipo se desarrolló mediante un conjunto de incrementos rápidos, priorizando la construcción funcional por sobre la calidad del código, las buenas prácticas de diseño o la apariencia visual. El enfoque estuvo centrado en obtener resultados tempranos que permitieran tomar decisiones informadas.

En primer lugar, se validó la comunicación entre Electron y Python, logrando establecer sin inconvenientes un canal de interacción utilizando el protocolo HTTP. Esta comunicación sentó

las bases para la separación de responsabilidades entre la visualización y la captura de datos. Posteriormente, se implementó la arquitectura de plugins en Python para la captura de datos. Para demostrar esta capacidad, se desarrollaron plugins que permitieron capturar eventos básicos como la posición del mouse, las pulsaciones del teclado y la captura de video. Estos plugins fueron diseñados para ser integrados en tiempo de ejecución, replicando así la característica de extensibilidad de MO.

En paralelo, se llevó a cabo la implementación de la arquitectura de plugins en Electron con React para la visualización de datos. Se crearon varios plugins de visualización que permitieron desplegar la información capturada en tiempo real. Específicamente se mostró el texto escrito mediante eventos de teclado, la posición actual del cursor del mouse, y una vista en miniatura de la ventana de captura de video (Figuras 3.1 y 3.2). Esto permitió validar que el sistema demostró ser capaz de presentar múltiples flujos de información simultáneamente, tal como lo hacía la versión original.

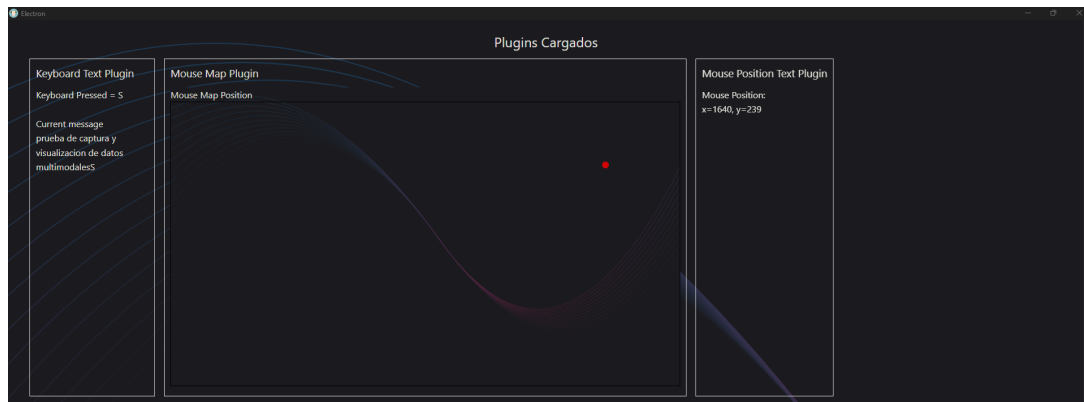


Figura 3.1: Primer prototipo - Plugins de teclado y mouse.

Fuente: Elaboración propia, 2025.

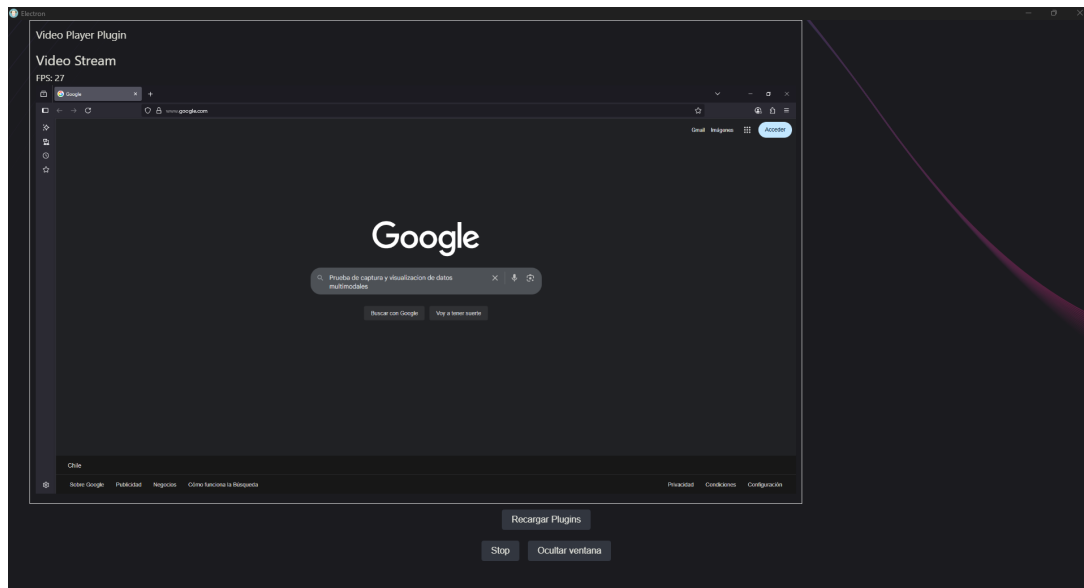


Figura 3.2: Primer prototipo - Plugin de video.

Fuente: Elaboración propia, 2025.

Otro aspecto relevante abordado en este prototipo fue el trabajo con procesos. Demostrando que el sistema era capaz de abrir y cerrar procesos de forma controlada, específicamente con un plugin que levantaba un navegador para realizar una captura de video al iniciar la captura de datos y cerrándolo automáticamente al detener la captura (Figura 3.3).

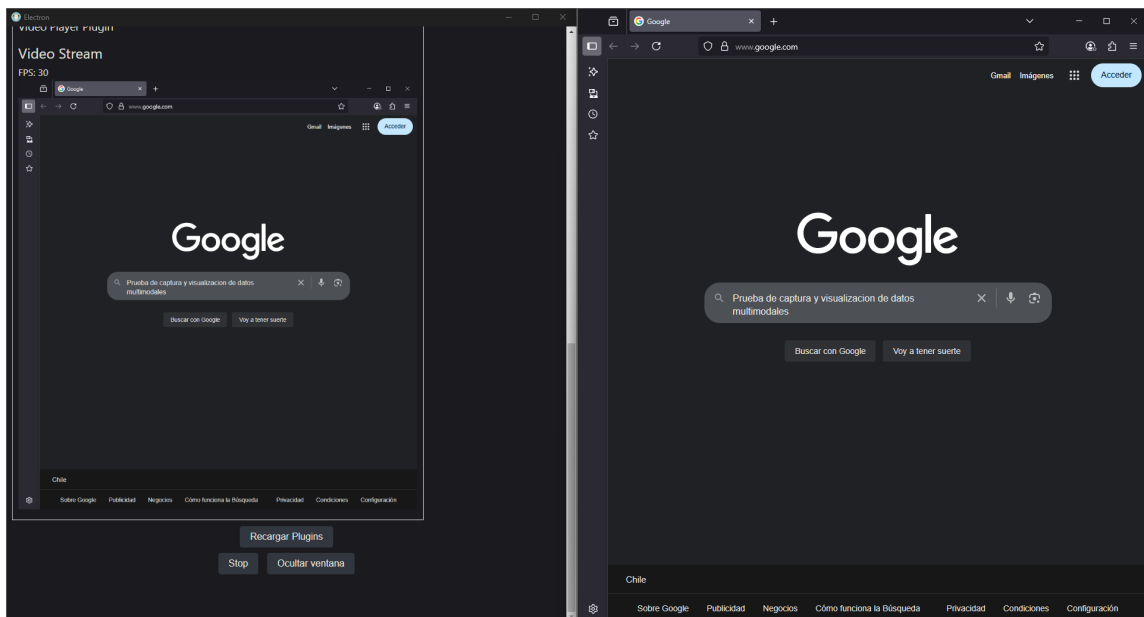


Figura 3.3: Primer prototipo - Procesos.

Fuente: Elaboración propia, 2025.

Asimismo, se implementó un mecanismo para la captura y visualización de mensajes mediante

sockets, lo que permitió validar la capacidad de interactuar con dispositivos remotos a través de la red. Para ello, se construyó un plugin de captura en Python encargado de abrir un socket, recibir mensajes y cerrarlo al finalizar la captura. Paralelamente, un plugin de visualización en Electron permitía mostrar estos mensajes en pantalla. En la figura 3.4 se muestra una prueba de esta característica, probada con la aplicación para celular "UDP Sender Receiver".

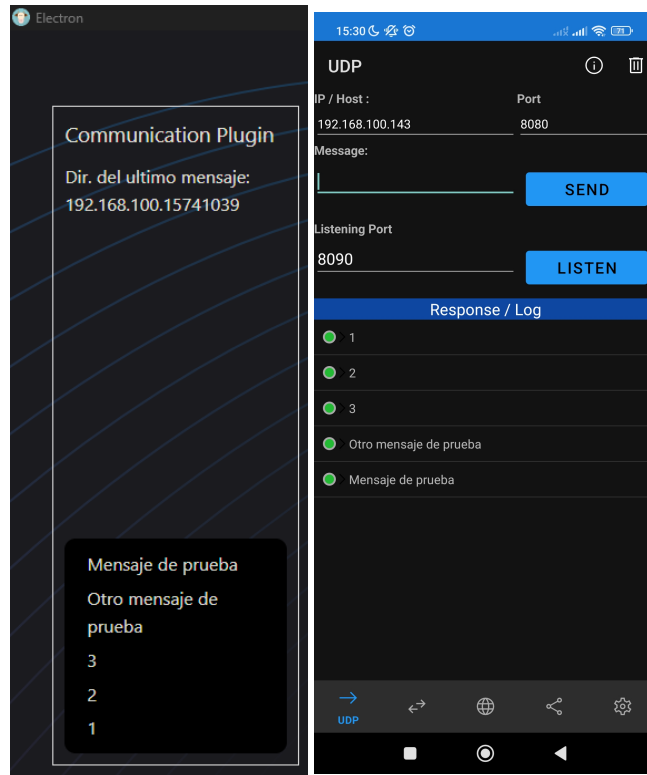


Figura 3.4: Primer prototipo - Comunicación.

Fuente: Elaboración propia, 2025.

Finalmente, se incorporó la bandeja del sistema (*System Tray*) en la aplicación, permitiendo acceder a acciones rápidas como iniciar o detener una captura sin necesidad de abrir la interfaz principal (Figura 3.5).

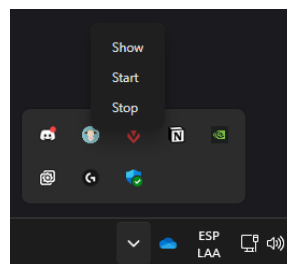


Figura 3.5: Primer prototipo - System Tray.

Fuente: Elaboración propia, 2025.

Este primer prototipo permitió confirmar que las funcionalidades críticas del sistema original podían ser replicadas y que el proyecto contaba con una base tecnológica sólida para continuar su desarrollo.

3.4.2 Segundo prototipo: Implementación del módulo de organización

El segundo prototipo corresponde a una iteración concreta dentro del desarrollo de MO, cuyo objetivo fue la construcción e integración del módulo de organización. A diferencia del primer prototipo, cuyo foco estuvo en la validación técnica general, en esta iteración se desarrolló un conjunto de funcionalidades que forman parte de la aplicación final, siguiendo un enfoque iterativo conforme a la metodología RAD explicada anteriormente.

Este prototipo estuvo precedido por la elaboración de mockups en Figma, los cuales guiaron el diseño de la interfaz de usuario durante el desarrollo, permitiendo mantener la consistencia visual y funcional a lo largo de las diferentes vistas de la aplicación. La Tabla 3.4 presenta un resumen estructurado de este segundo prototipo.

Tabla 3.4: Segundo prototipo - Implementación del módulo de organización.

ID Prototipo	P-02
Nombre	Implementación del módulo de organización
Objetivos	Permitir la gestión de proyectos, participantes y protocolos de investigación.
Descripción	Iteración enfocada en la creación e integración del módulo de organización dentro de MO, permitiendo a los usuarios gestionar proyectos de investigación, participantes y protocolos asociados. Se implementaron tanto las interfaces gráficas como la lógica de negocio y almacenamiento.
Requisitos Funcionales	RF-01 al RF-20
Requisitos No Funcionales	RNF-01, RNF-02, RNF-04

Fuente: Elaboración propia, 2025.

Esta versión integró un conjunto de funcionalidades que permiten a los usuarios gestionar proyectos de investigación, participantes y protocolos asociados, consolidando así las bases del módulo de organización de MO.

En primer lugar, se implementaron las acciones relacionadas con la gestión de proyectos. Los usuarios pueden crear, editar, eliminar y visualizar proyectos de investigación (Figuras 3.6, 3.7, 3.8, 3.9, 3.11 y 3.10). Para cada proyecto se habilitó además la posibilidad de bloquear

o desbloquear su edición, previniendo modificaciones o eliminaciones accidentales, como se muestra en la Figura 3.12.

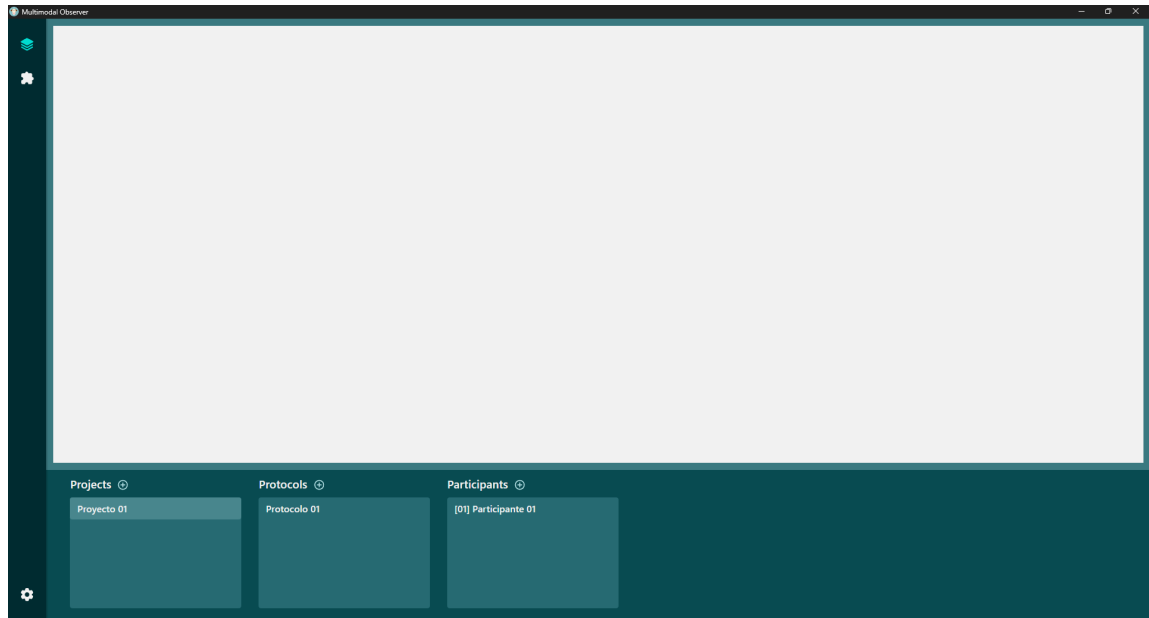


Figura 3.6: Segundo prototipo - Vista general.

Fuente: Elaboración propia, 2025.

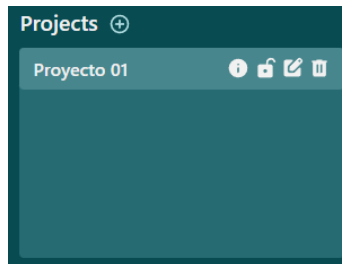
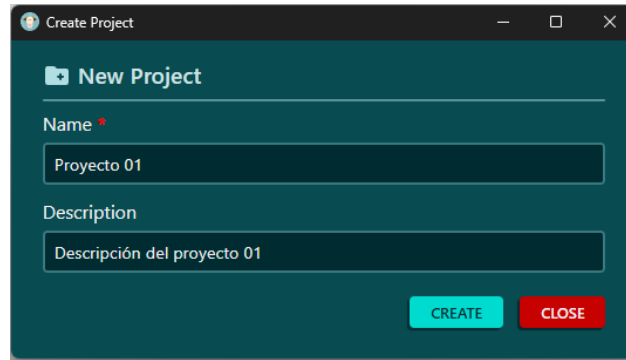


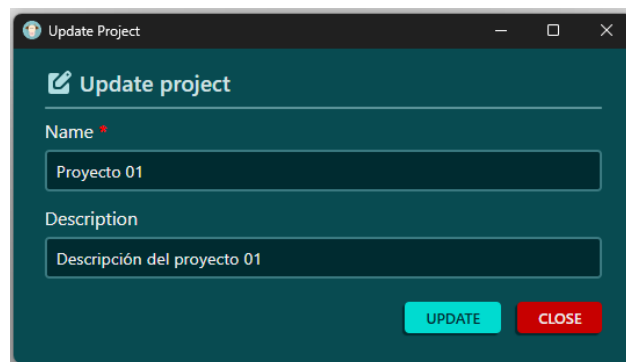
Figura 3.7: Segundo prototipo - Acciones para un proyecto.

Fuente: Elaboración propia, 2025.



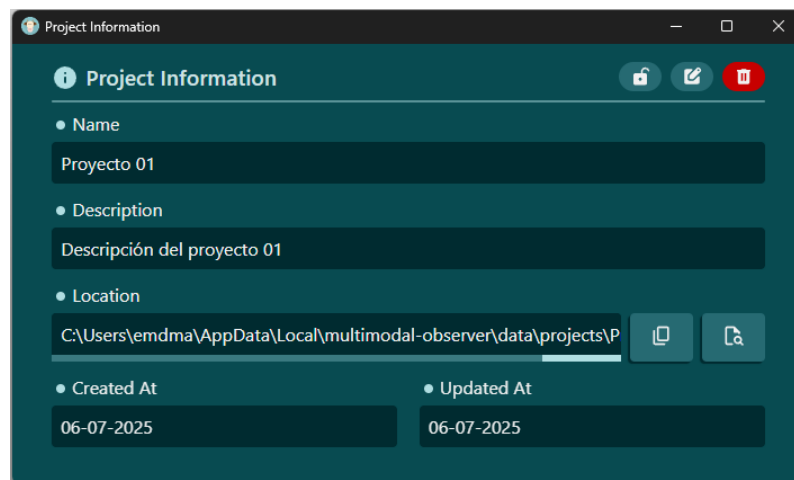
A screenshot of a 'Create Project' dialog box. The title bar says 'Create Project'. The main title is 'New Project'. There are two text input fields: 'Name' with the value 'Proyecto 01' and 'Description' with the value 'Descripción del proyecto 01'. At the bottom right, there are two buttons: 'CREATE' (cyan) and 'CLOSE' (red).

Figura 3.8: Segundo prototipo - Crear proyecto.
Fuente: Elaboración propia, 2025.



A screenshot of an 'Update Project' dialog box. The title bar says 'Update Project'. The main title is 'Update project'. There are two text input fields: 'Name' with the value 'Proyecto 01' and 'Description' with the value 'Descripción del proyecto 01'. At the bottom right, there are two buttons: 'UPDATE' (cyan) and 'CLOSE' (red).

Figura 3.9: Segundo prototipo - Actualizar proyecto.
Fuente: Elaboración propia, 2025.



A screenshot of a 'Project Information' window. The title bar says 'Project Information'. The main title is 'Project Information'. There are three sections: 'Name' with the value 'Proyecto 01', 'Description' with the value 'Descripción del proyecto 01', and 'Location' with the value 'C:\Users\emdma\AppData\Local\multimodal-observer\data\projects\P'. There are also two buttons: 'COPY' and 'PASTE'. At the bottom, there are two sections: 'Created At' with the value '06-07-2025' and 'Updated At' with the value '06-07-2025'.

Figura 3.10: Segundo prototipo - Ver información de un proyecto.
Fuente: Elaboración propia, 2025.

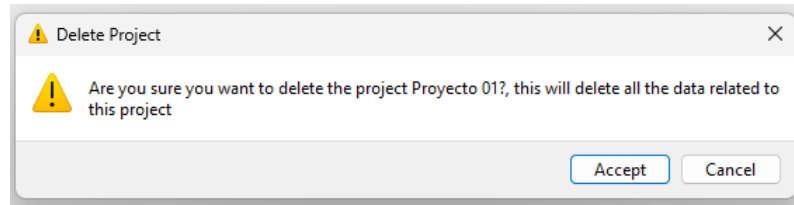


Figura 3.11: Segundo prototipo - Eliminar proyecto.

Fuente: Elaboración propia, 2025.

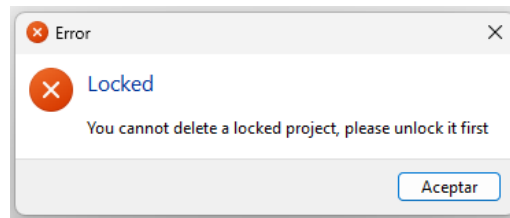


Figura 3.12: Segundo prototipo - Eliminar proyecto bloqueado.

Fuente: Elaboración propia, 2025.

En segundo lugar, se desarrolló la gestión de participantes, permitiendo crear, editar, eliminar y visualizar participantes asociados a los proyectos (Figuras 3.13, 3.14 y 3.15). Al igual que los proyectos, los participantes pueden ser bloqueados para resguardar la integridad de sus datos y prevenir cambios no deseados.

A form titled "Add Participant" with a dark teal background. It has a header "New Participant" with a plus icon. Below are input fields for "Code" (containing "01"), "Name" (containing "Participante 01"), and "Notes" (containing "Nota 1" and "Nota 2"). Each note field has a red "X" icon to its right. At the bottom right, there are two buttons: "CREATE" (cyan) and "CLOSE" (red).

Figura 3.13: Segundo prototipo - Agregar participante a un proyecto.

Fuente: Elaboración propia, 2025.

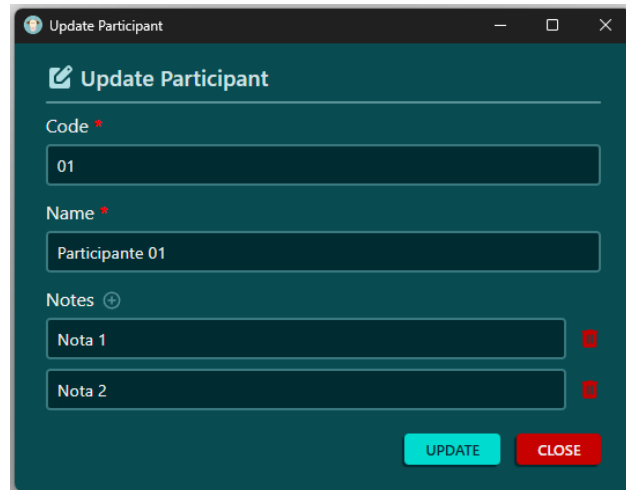


Figura 3.14: Segundo prototipo - Actualizar participante.
Fuente: Elaboración propia, 2025.

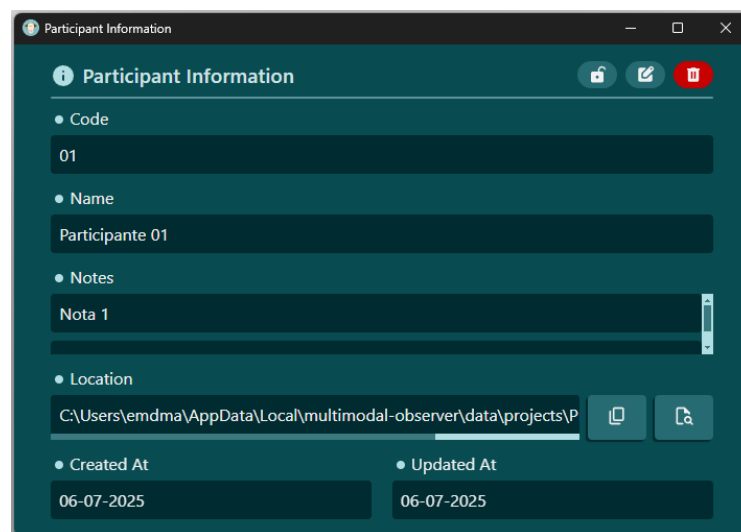


Figura 3.15: Segundo prototipo - Ver información de un participante.
Fuente: Elaboración propia, 2025.

Asimismo, se incorporó la gestión de protocolos de investigación, funcionalidad solicitada durante el desarrollo por el profesor guía proveniente del trabajo de Polanco Berrios (2019) para ser integrada en esta iteración. Esta característica permite a los usuarios crear protocolos, definir actividades dentro de ellos, eliminarlos cuando sea necesario, y ejecutar protocolos siguiendo la secuencia de actividades establecida (Figuras 3.16, 3.17, 3.18, 3.19, 3.20, 3.21 y 3.22). De igual forma, se añadió la capacidad de bloquear o desbloquear los protocolos para evitar su edición una vez validados.

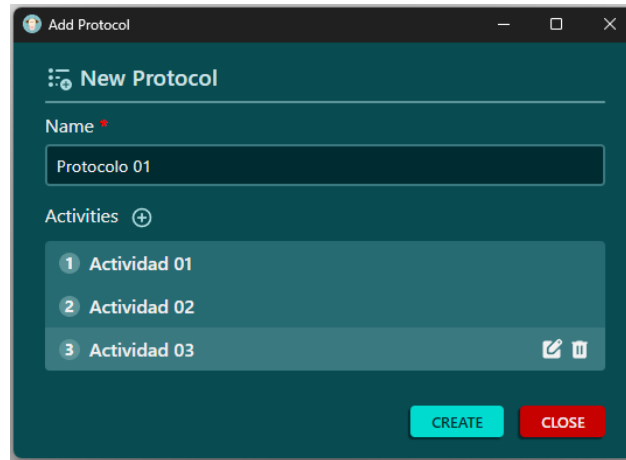


Figura 3.16: Segundo prototipo - Agregar protocolo a un proyecto.

Fuente: Elaboración propia, 2025.

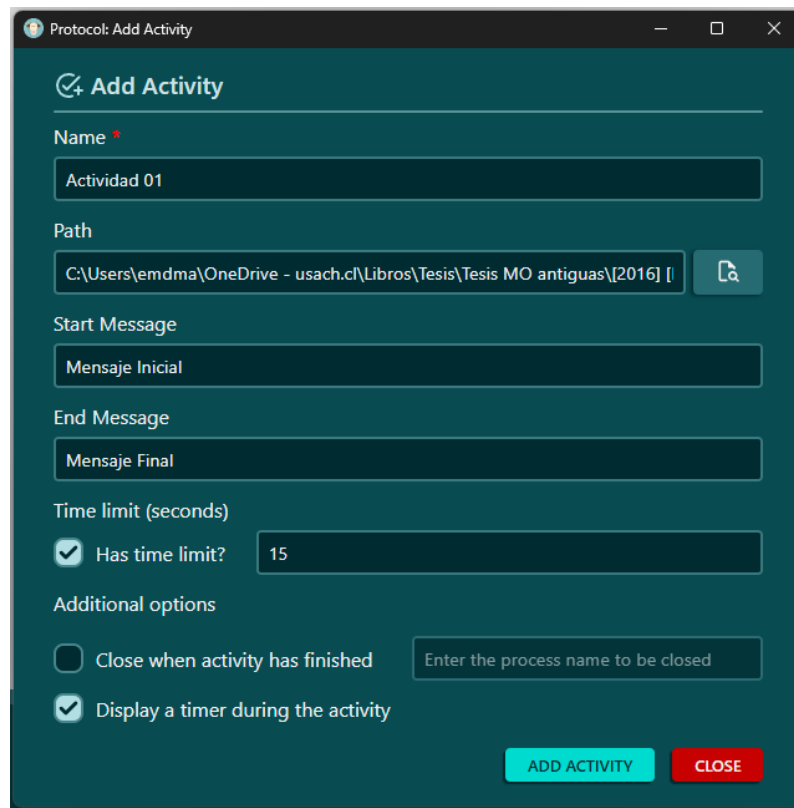


Figura 3.17: Segundo prototipo - Agregar actividad a un protocolo.

Fuente: Elaboración propia, 2025.

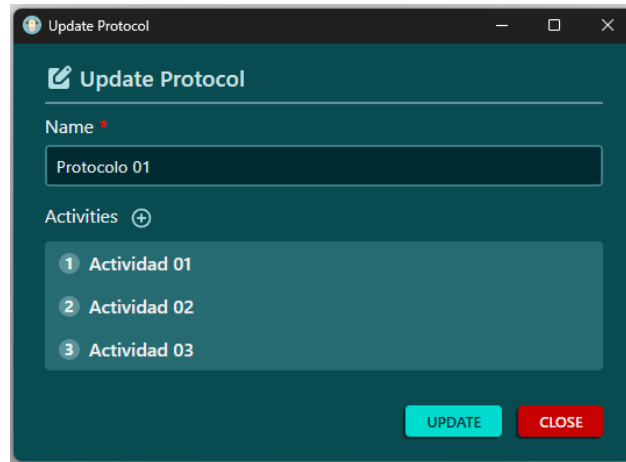


Figura 3.18: Segundo prototipo - Actualizar protocolo.

Fuente: Elaboración propia, 2025.

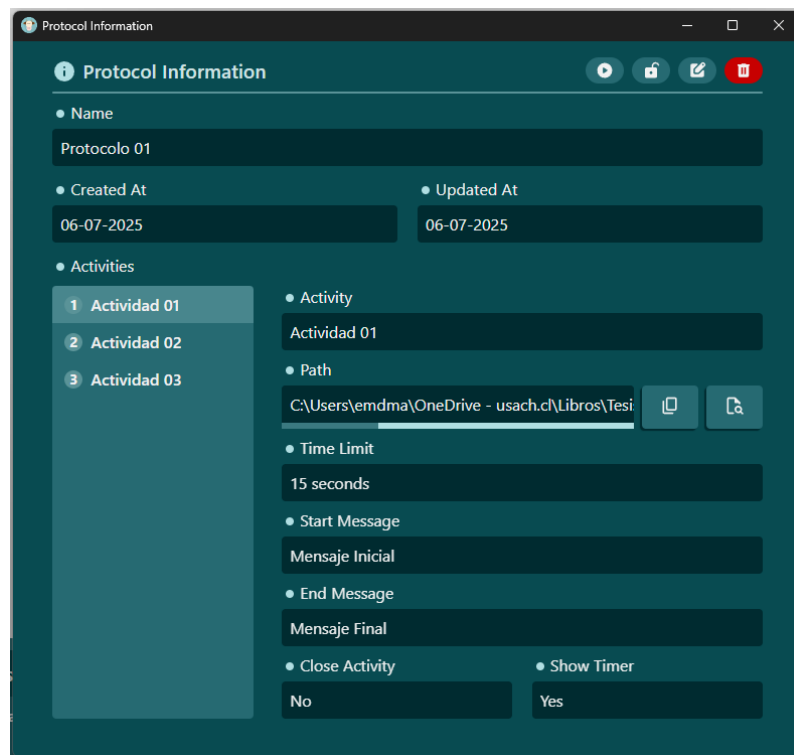


Figura 3.19: Segundo prototipo - Información de un protocolo.

Fuente: Elaboración propia, 2025.

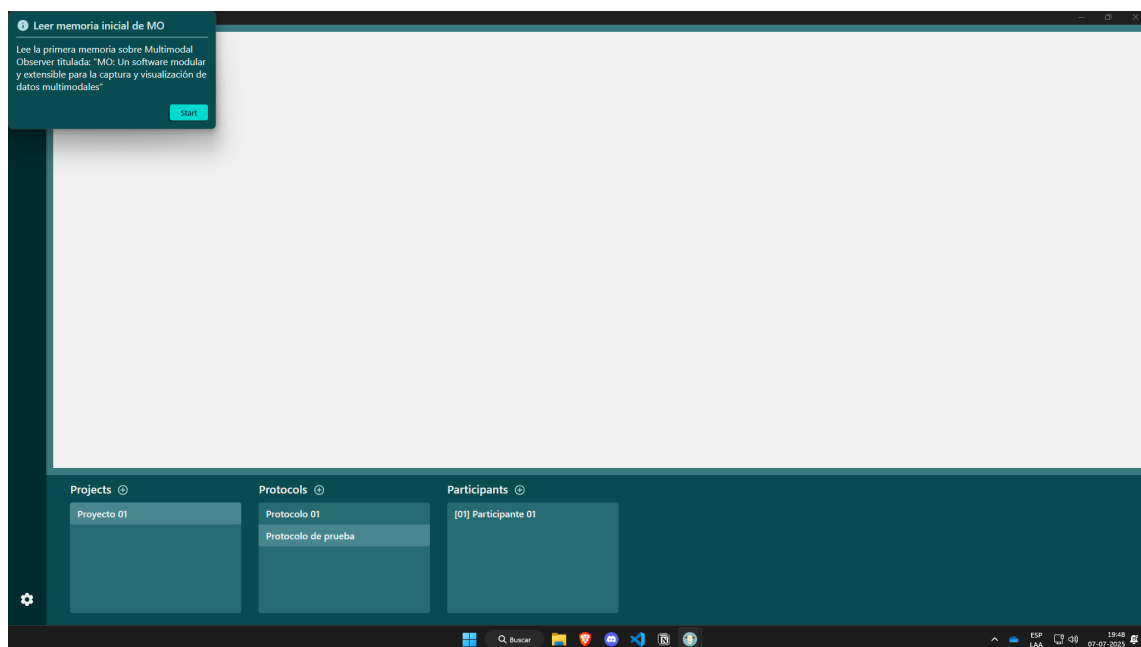


Figura 3.20: Segundo prototipo - Protocolo en ejecución 1.

Fuente: Elaboración propia, 2025.

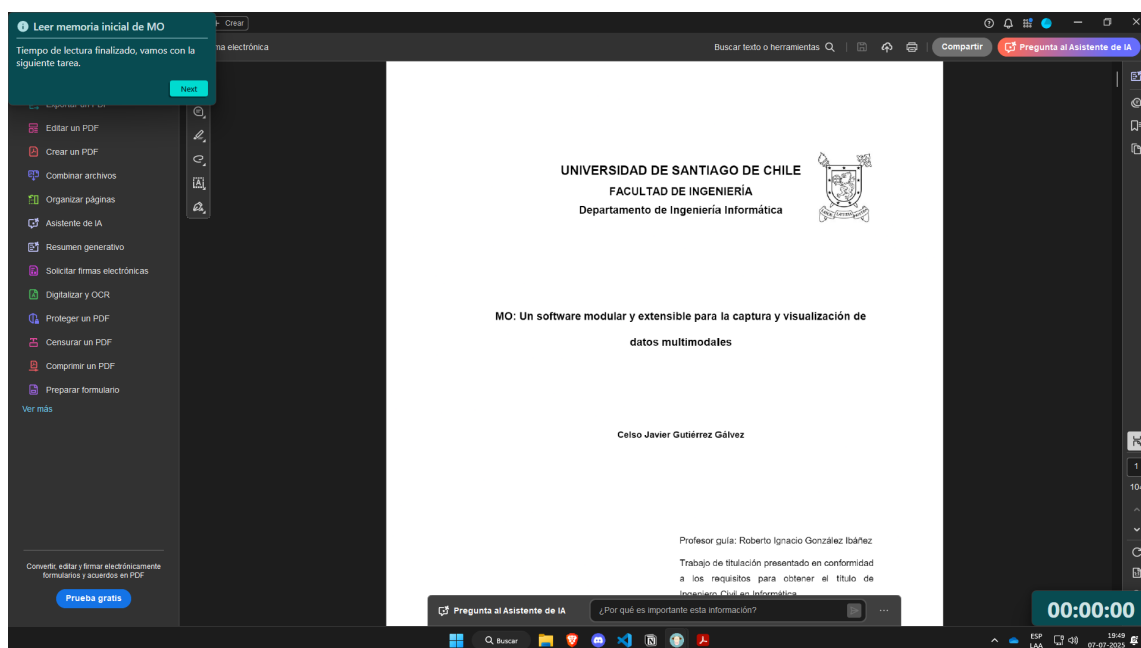


Figura 3.21: Segundo prototipo - Protocolo en ejecución 2.

Fuente: Elaboración propia, 2025.

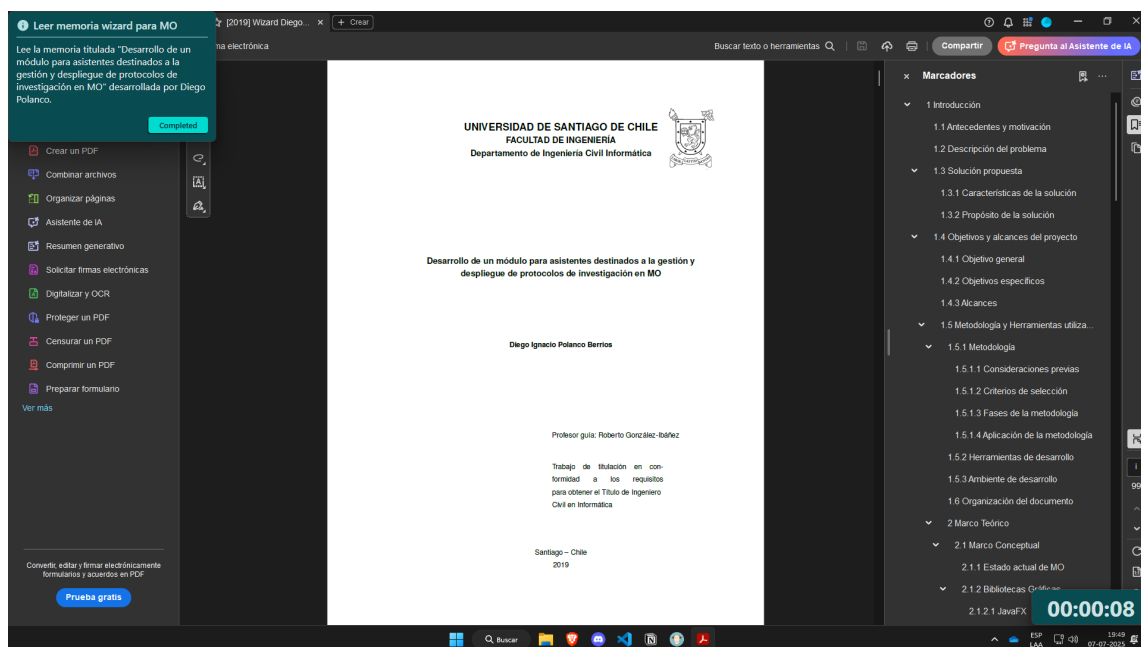


Figura 3.22: Segundo prototipo - Protocolo en ejecución 3.

Fuente: Elaboración propia, 2025.

Este prototipo permitió establecer las bases funcionales necesarias para la gestión de proyectos, participantes y protocolos dentro de MO, habilitando por primera vez la ejecución de estos flujos de trabajo de forma completa. La versión fue lanzada utilizando un instalador básico provisto por Electron y fue probado en Windows 11 y en Linux Ubuntu 24, este último mediante una máquina virtual ejecutada con VirtualBox.

3.4.3 Tercer prototipo: Implementación del módulo de captura y gestión de plugins

El tercer prototipo aborda dos aspectos fundamentales del sistema: por un lado, la implementación del módulo de captura de datos y, por otro, la creación del sistema de gestión de plugins en el backend, orientado a la extensión de funcionalidades que en este caso concreto apoyaran a la creación de plugins de captura.

En este prototipo se amplió el alcance del sistema para incluir la ejecución de captura de datos, gestión de sesiones de captura, y administración de plugins que funcionan en la capa de la API. En la Tabla 3.5 se presenta un resumen estructurado de este tercer prototipo.

Tabla 3.5: Tercer prototipo - Implementación del módulo de captura y gestión de plugins.

ID Prototipo	P-03
Nombre	Implementación del módulo de captura y gestión de plugins
Objetivos	Desarrollar el módulo de captura de datos. Implementar un sistema de gestión de plugins en la API. Incorporar los primeros plugins de captura (pantalla, teclado, mouse).
Descripción	Iteración enfocada en la integración de la funcionalidad de captura de datos mediante plugins configurables, la gestión de plugins dentro de la aplicación y la creación de plugins de pantalla, teclado y mouse.
Requisitos Funcionales	RF-22 al RF-30, RF-33 al RF-39 y RF-50 a RF-52
Requisitos No Funcionales	RNF-01, RNF-02, RNF-04, RNF-05

Fuente: Elaboración propia, 2025.

La primera línea de trabajo se centró en la construcción de un sistema de gestión de plugins, permitiendo a los usuarios administrar extensiones que amplían las capacidades del sistema. A través de una nueva interfaz, los usuarios pueden agregar, visualizar, eliminar y obtener información detallada de los plugins instalados (Figuras 3.23, 3.24 y 3.25). Estas características se enfocaron exclusivamente en plugins para la API.

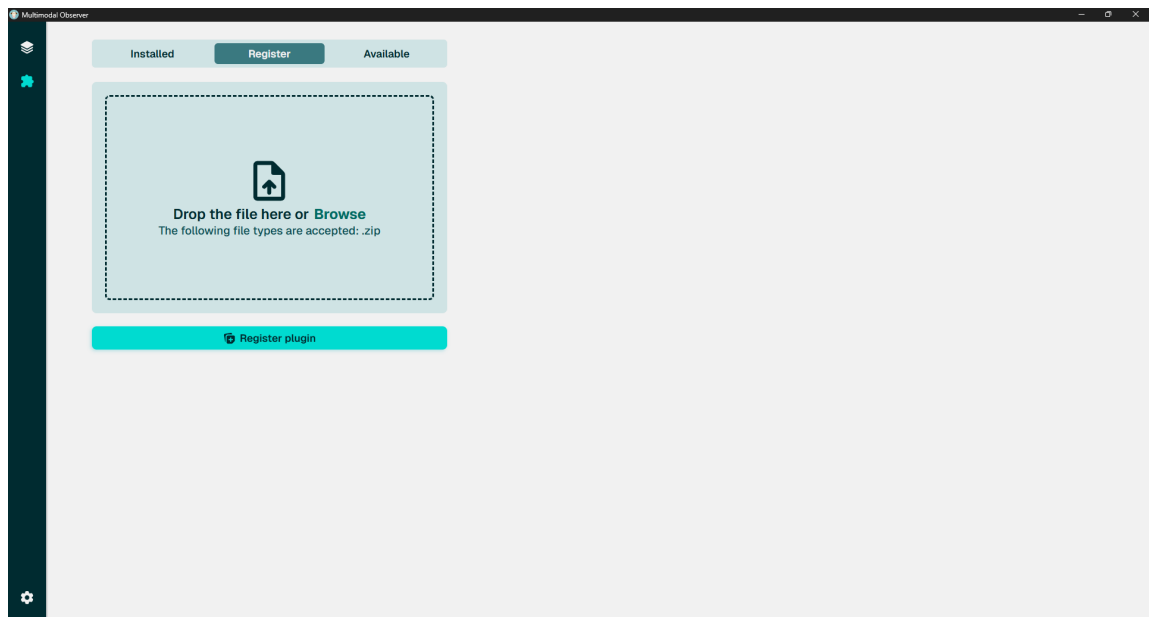


Figura 3.23: Tercer prototipo - Agregar plugin.

Fuente: Elaboración propia, 2025.

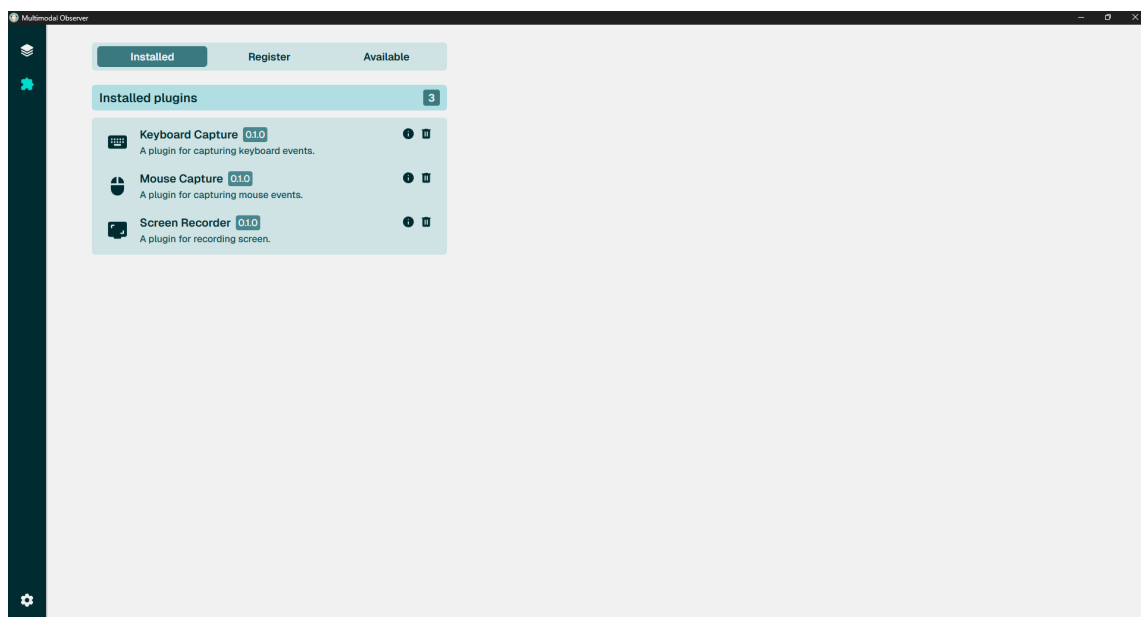


Figura 3.24: Tercer prototipo - Plugins instalados.

Fuente: Elaboración propia, 2025.

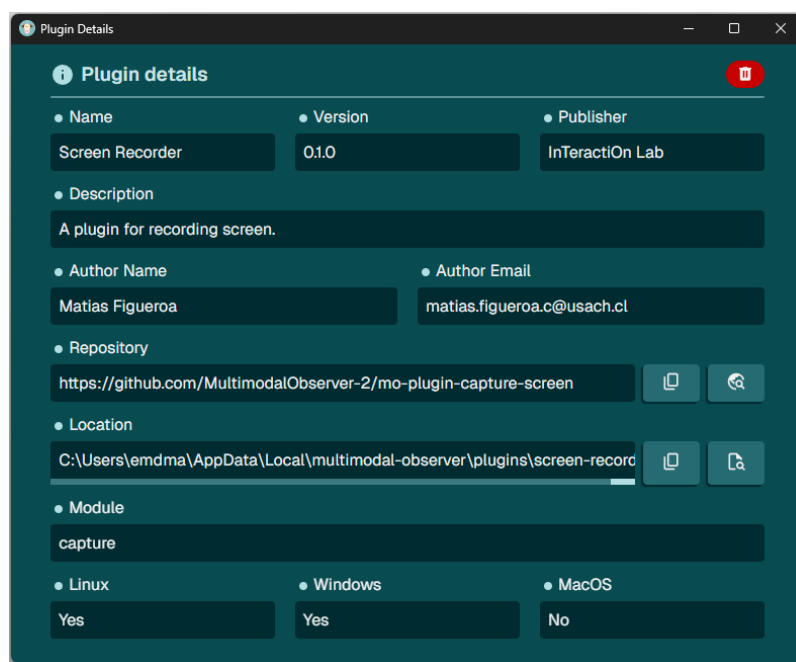


Figura 3.25: Tercer prototipo - Información de un Plugin.

Fuente: Elaboración propia, 2025.

En segundo lugar, se introdujo un nuevo módulo de captura, el cual permite a los usuarios iniciar, pausar, reanudar y detener sesiones de captura mediante el sistema de plugins (Figuras 3.26, 3.27 y 3.28). Para cada proyecto, se habilitó la posibilidad de definir configuraciones específicas

por plugin de captura, facilitando la personalización de parámetros como por ejemplo la resolución de video, el codec a utilizar o las opciones de registro de eventos de teclado o mouse.

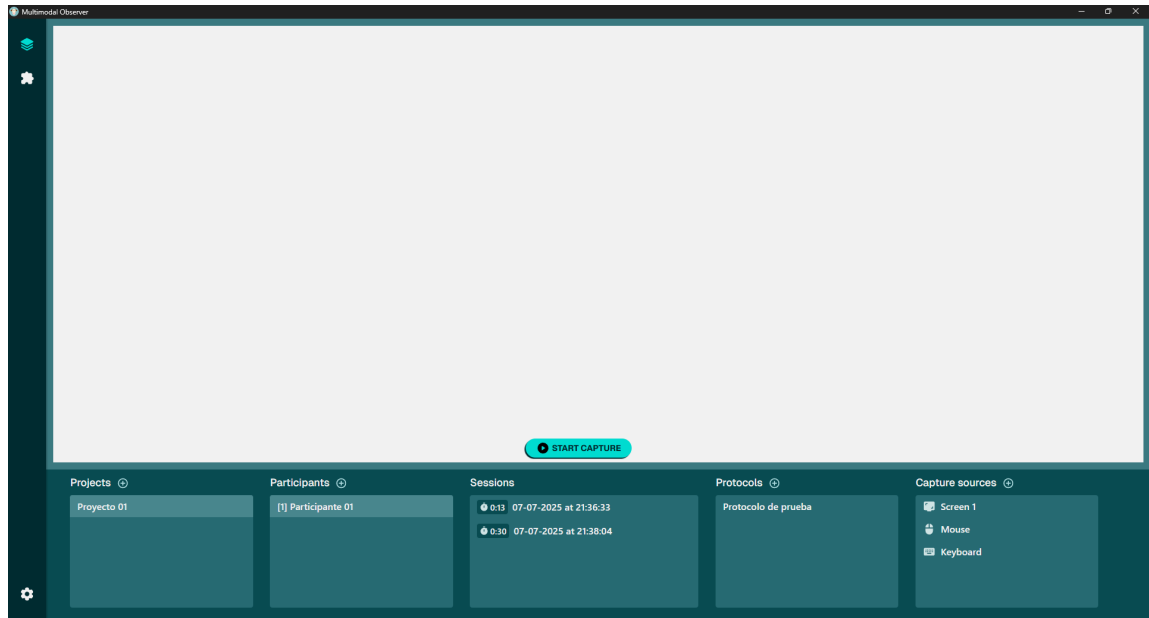


Figura 3.26: Tercer prototipo - Vista general.

Fuente: Elaboración propia, 2025.

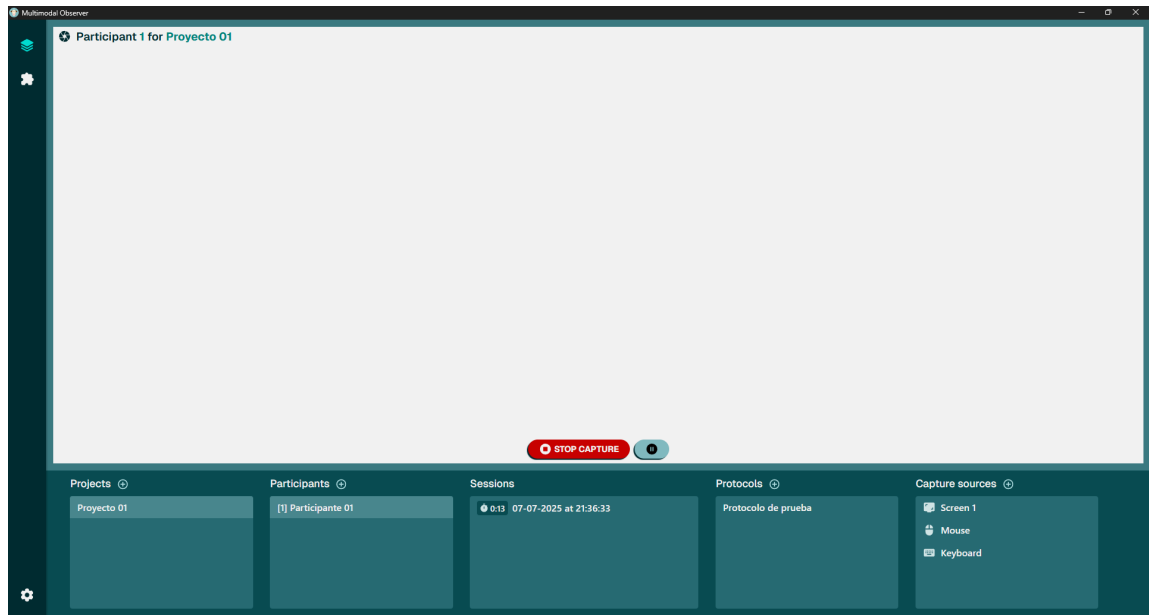


Figura 3.27: Tercer prototipo - Capturando.

Fuente: Elaboración propia, 2025.

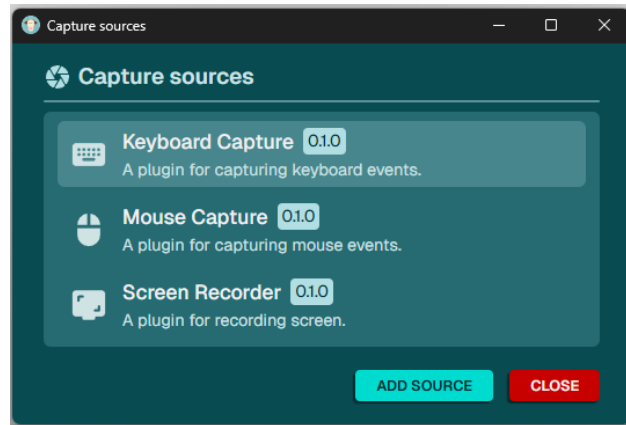


Figura 3.28: Tercer prototipo - Fuentes de captura.
Fuente: Elaboración propia, 2025.

Las sesiones de captura realizadas pueden ser gestionadas desde la aplicación, permitiendo ver un listado de sesiones pasadas, inspeccionar metadatos asociados a cada sesión o eliminar registros no deseados (Figuras 3.29, 3.30 y 3.31). Esta funcionalidad sentó las bases para la gestión integral de los datos observacionales que se recolectan mediante MO.

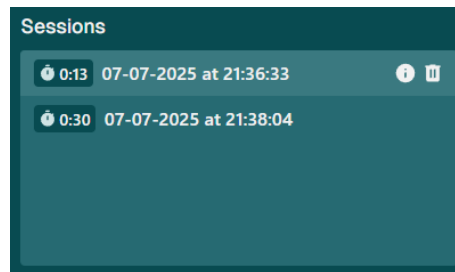


Figura 3.29: Tercer prototipo - Listado de sesiones.
Fuente: Elaboración propia, 2025.

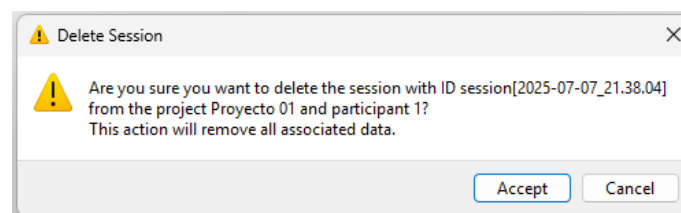


Figura 3.30: Tercer prototipo - Eliminar sesión.
Fuente: Elaboración propia, 2025.

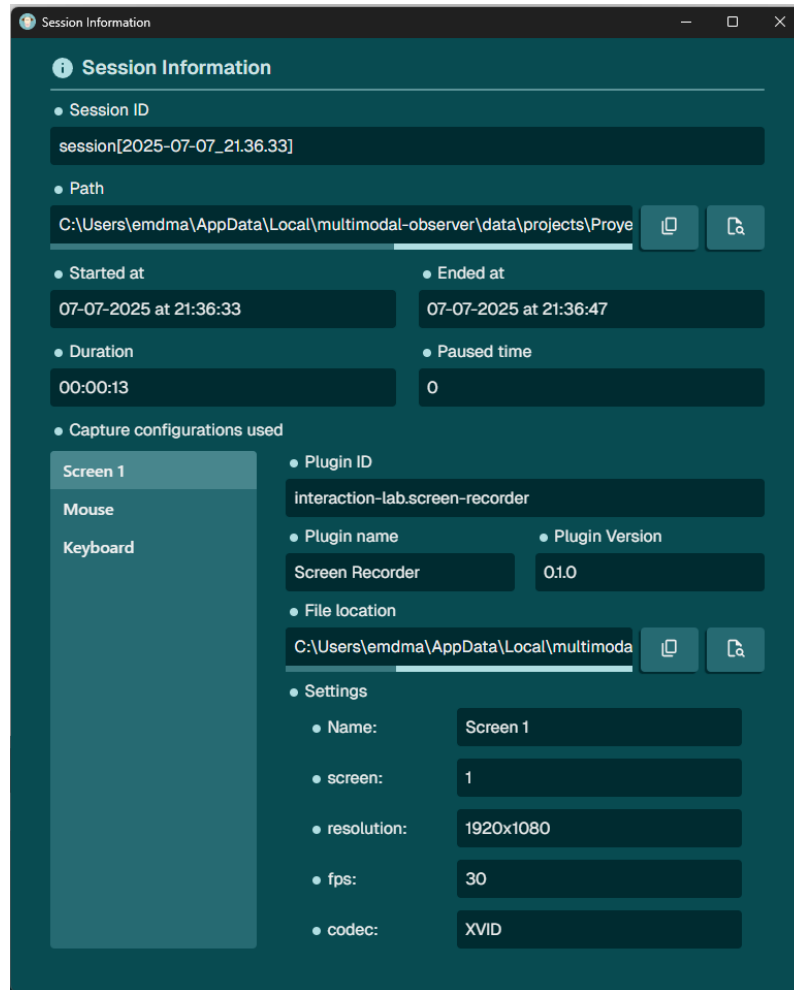


Figura 3.31: Tercer prototipo - Información de sesión.

Fuente: Elaboración propia, 2025.

Dentro de esta iteración, se crearon los primeros plugins reales de captura para ser utilizados en el sistema:

- **Plugin de captura de pantalla:** Permite grabar contenido de pantalla en video, soportando múltiples pantallas, diferentes resoluciones, tasas de cuadros por segundo configurables y selección de codecs de video. El resultado de cada sesión de captura se almacena en archivos de video en formato .avi o .mp4 (Figura 3.33).
- **Plugin de captura de teclado:** Permite registrar eventos de teclado, incluyendo presiones y liberaciones de teclas, con la posibilidad de reconstruir los mensajes escritos. Los eventos se almacenan en archivos estructurados en formato .json para su análisis posterior (Figuras 3.34 y 3.35).
- **Plugin de captura de mouse:** Permite capturar movimientos, clics y desplazamientos

de scroll del mouse. Al igual que el plugin de teclado, genera archivos .json con datos detallados de cada evento (Figuras 3.36 y 3.37).

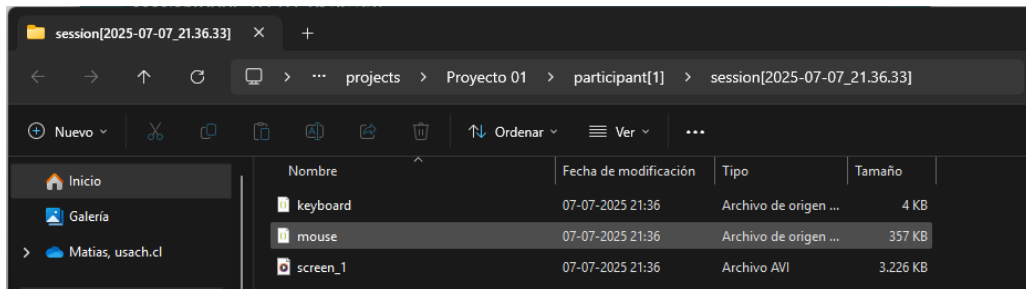


Figura 3.32: Tercer prototipo - Archivos capturados.

Fuente: Elaboración propia, 2025.

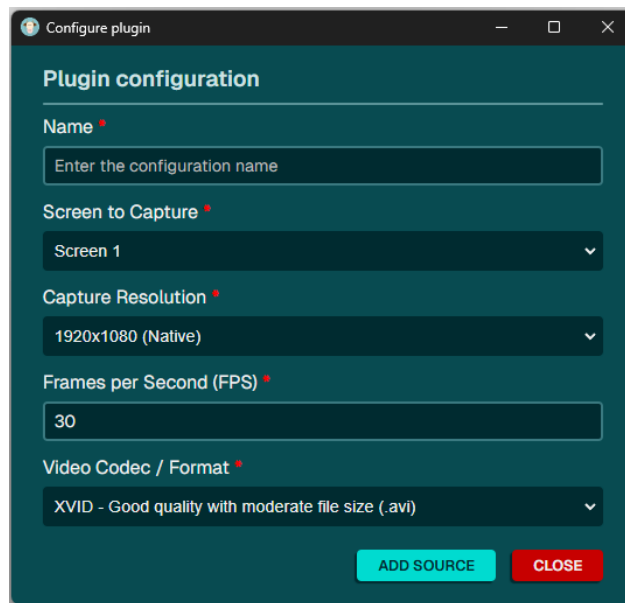


Figura 3.33: Tercer prototipo - Configuración de plugin de captura de pantalla.

Fuente: Elaboración propia, 2025.

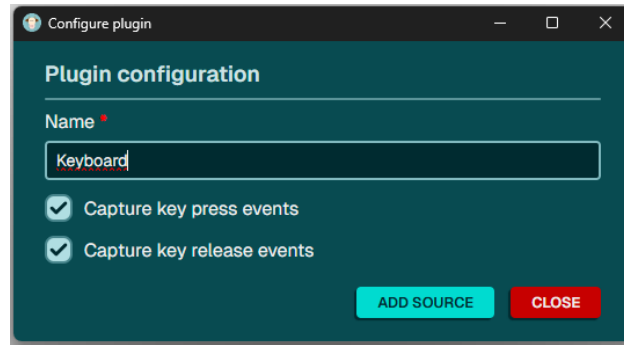


Figura 3.34: Tercer prototipo - Configuración de plugin de captura de teclado.

Fuente: Elaboración propia, 2025.

```

1  {
2
3      "event": "press",
4      "key": "Key.caps_lock",
5      "timestamp": 889091.625,
6      "message": ""
7  },
8  {
9      "event": "press",
10     "key": "'p'",
11     "timestamp": 889092.187,
12     "message": "p"
13 },
14 {
15     "event": "press",
16     "key": "Key.caps_lock",
17     "timestamp": 889092.296,
18     "message": "p"
19 },
20 {
21     "event": "press",
22     "key": "'r'",
23     "timestamp": 889092.437,
24     "message": "pr"
25 },
26 {
27     "event": "press",
28     "key": "'u'",
29     "timestamp": 889092.671,
30     "message": "pru"
31 },
32 {
33     "event": "press",
34     "key": "'e'",
35     "timestamp": 889092.781,
36     "message": "prue"
37 },

```

Figura 3.35: Tercer prototipo - Archivo resultante captura de teclado.

Fuente: Elaboración propia, 2025.

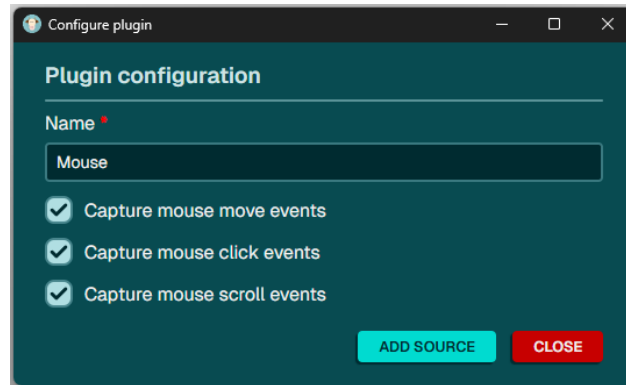


Figura 3.36: Tercer prototipo - Configuración de plugin de captura del mouse.

Fuente: Elaboración propia, 2025.

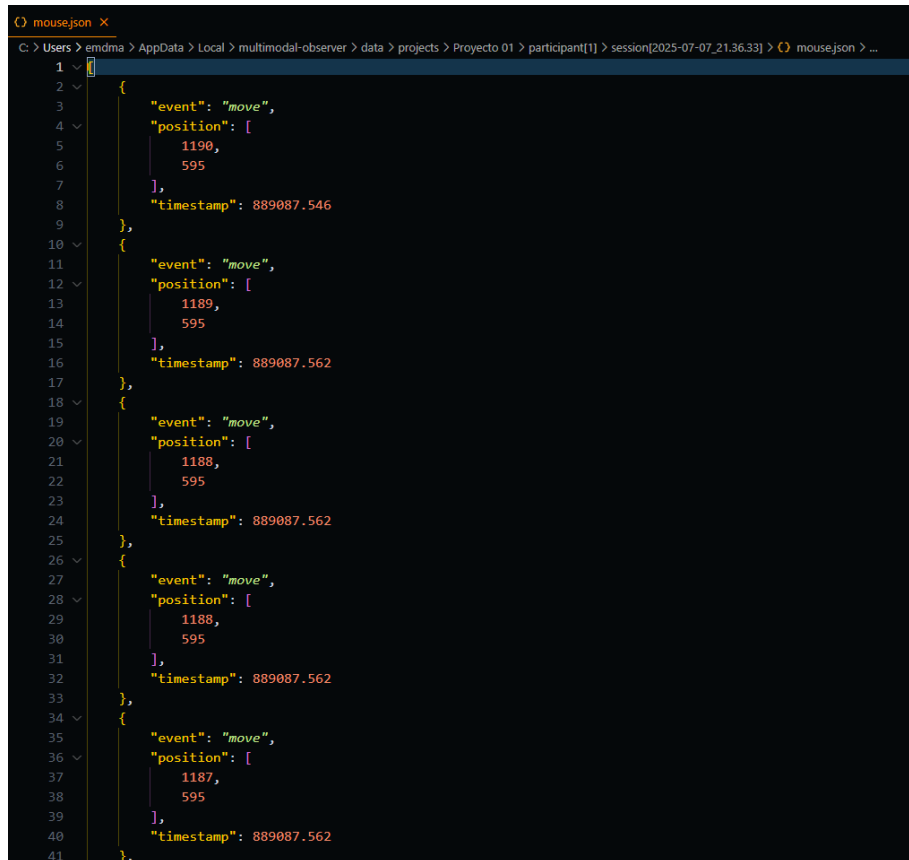


Figura 3.37: Tercer prototipo - Archivo resultante captura del mouse.

Fuente: Elaboración propia, 2025.

Cada uno de estos plugins fue empaquetado como un archivo .zip e integrado mediante el sistema de carga de plugins dentro de MO, lo que permitió validar la extensibilidad y modularidad del sistema.

Este prototipo amplió las capacidades funcionales de MO, permitiendo por primera vez realizar

la captura de datos mediante un enfoque flexible y basado en plugins, sentando las bases para futuras integraciones con dispositivos y fuentes de datos externas.

3.4.4 Cuarto prototipo: Implementación del módulo de visualización y gestión de plugins en la interfaz

El cuarto prototipo corresponde a la incorporación del modulo de visualización que permite la reproducción de sesiones capturadas, junto con el desarrollo de un sistema de gestión de plugins desde la interfaz grafica (UI). Esta iteración se enfocó en habilitar la revisión visual de datos multimodales, al mismo tiempo que fortaleció la extensibilidad de la aplicación desde el lado del frontend. La Tabla 3.6 presenta un resumen estructurado de este cuarto prototipo.

Tabla 3.6: Cuarto prototipo - Implementación del módulo de visualización y gestión de plugins en la interfaz.

ID Prototipo	P-04
Nombre	Implementación del módulo de visualización y gestión de plugins en la interfaz
Objetivos	Desarrollar el módulo de visualización para la reproducción de sesiones capturadas. Implementar el sistema de gestión de plugins desde la interfaz gráfica. Crear los primeros plugins de visualización (Playback Plugins).
Descripción	Iteración enfocada en la incorporación de un módulo de visualización flexible y extensible, junto con un sistema de gestión de plugins desde la UI. Se desarrollaron los primeros plugins de reproducción y se reorganizó la interfaz para mejorar la navegación y el acceso a las funciones del sistema.
Requisitos Funcionales	RF-42 a RF-49 y RF-53 a RF-55
Requisitos No Funcionales	RNF-01, RNF-02, RNF-04, RNF-05, RNF-06

Fuente: Elaboración propia, 2025.

En primer lugar, se implementó un sistema de gestión de plugins directamente en la interfaz de usuario, permitiendo a los usuarios realizar acciones como agregar, eliminar, visualizar y obtener información detallada de los plugins instalados. A partir de esta iteración, los plugins se agrupan por tipo (en este caso *Capture Plugins* y *Playback Plugins*) facilitando así la organización y administración dentro de la aplicación como se observa en la Figura 3.38.

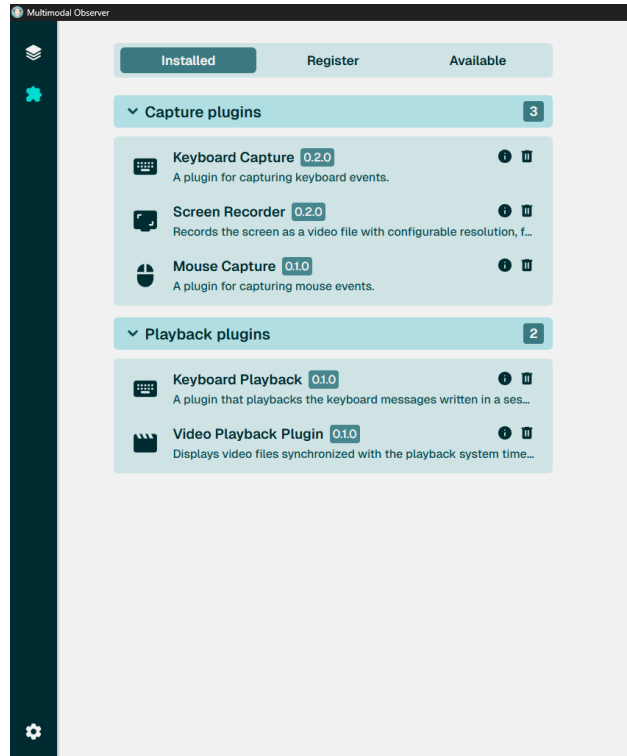


Figura 3.38: Cuarto prototipo - Plugins instalados.

Fuente: Elaboración propia, 2025.

La gestión de plugins se complementó con la introducción de un sistema de configuración por proyecto, permitiendo a los usuarios definir y personalizar tanto plugins de captura como plugins de reproducción (visualización). Los usuarios pueden agregar, ocultar/mostrar, editar y eliminar estas configuraciones de forma independiente por cada proyecto (Figuras 3.39 y 3.40).

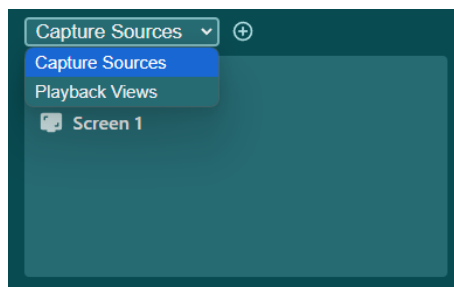


Figura 3.39: Cuarto prototipo - Cambio de tipo de configuración.

Fuente: Elaboración propia, 2025.

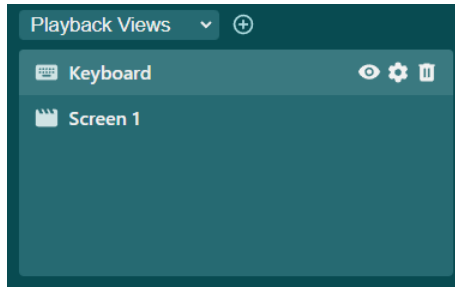


Figura 3.40: Cuarto prototipo - Acciones configuración de "Playback Views".

Fuente: Elaboración propia, 2025.

En segundo lugar, se desarrolló e integró el módulo de visualización, el cual permite la reproducción de sesiones previamente capturadas dentro de la aplicación. Este módulo se diseñó en base a la gestión de plugins de la UI mencionada anteriormente, permitiendo extender fácilmente los tipos de datos que pueden ser visualizados en el futuro, preservando así esta característica de MO al igual que el tercer prototipo.

Durante esta iteración se crearon los primeros *Playback Plugins* para la visualización de sesiones:

- **Plugin de visualización de video:** Permite mostrar los videos capturados durante una sesión de captura (Figura 3.41).
- **Plugin de visualización de texto (teclado):** Muestra los mensajes escritos a lo largo de la sesión, reconstruidos a partir de los datos capturados por el plugin de teclado (Figura 3.41).
- **Plugin de visualización de mouse:** Visualiza los movimientos y eventos del mouse durante la sesión, mostrando las coordenadas y la posición del cursor (Figura 3.41).

El sistema de reproducción fue diseñado para ser flexible y centrado en la experiencia del usuario. Permite controlar la reproducción de las sesiones capturadas mediante acciones globales, como pausar, reanudar o desplazarse a través de la línea de tiempo. Estos controles afectan de manera conjunta a todos los *Playback Plugins* activos, los cuales se sincronizan con el estado global de la reproducción para asegurar que la visualización de los distintos flujos de datos se mantenga alineada en todo momento (Figura 3.41).

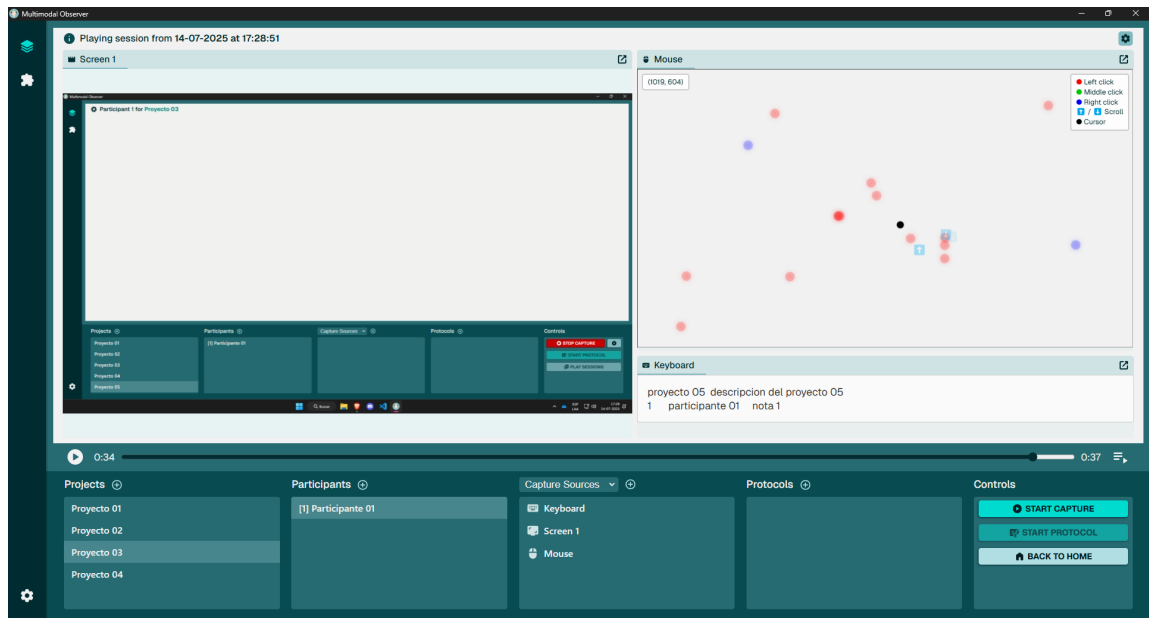


Figura 3.41: Cuarto prototipo - Vista de reproducción de sesiones capturadas.

Fuente: Elaboración propia, 2025.

Además, los usuarios pueden organizar los plugins de visualización dentro de un espacio de trabajo dinámico, en el que es posible arrastrar, redimensionar, ocultar o abrir los plugins en ventanas independientes, ajustando la interfaz según las preferencias o necesidades de visualización de los datos. La aplicación conserva automáticamente la configuración de este espacio para futuras sesiones. En la Figura 3.42 se puede observar un cambio en el espacio de trabajo con respecto a la Figura 3.41. Por otro lado la Figura 3.43 muestra la característica de abrir en una ventana independiente una vista.

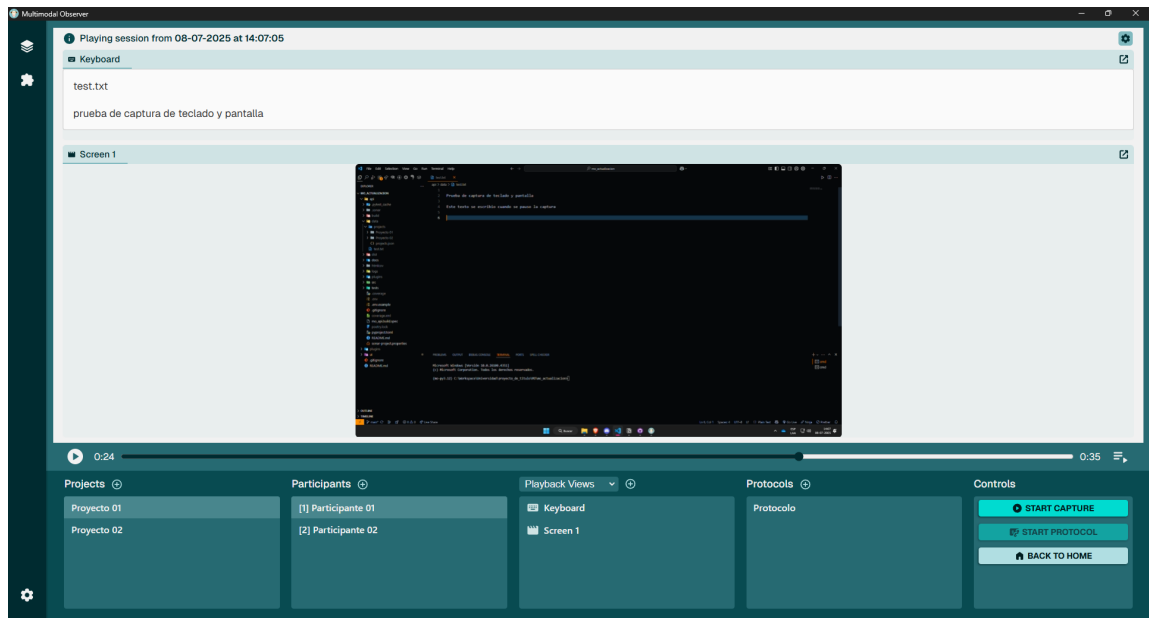


Figura 3.42: Cuarto prototipo - Redimensión y cambio de espacio de trabajo.
Fuente: Elaboración propia, 2025.

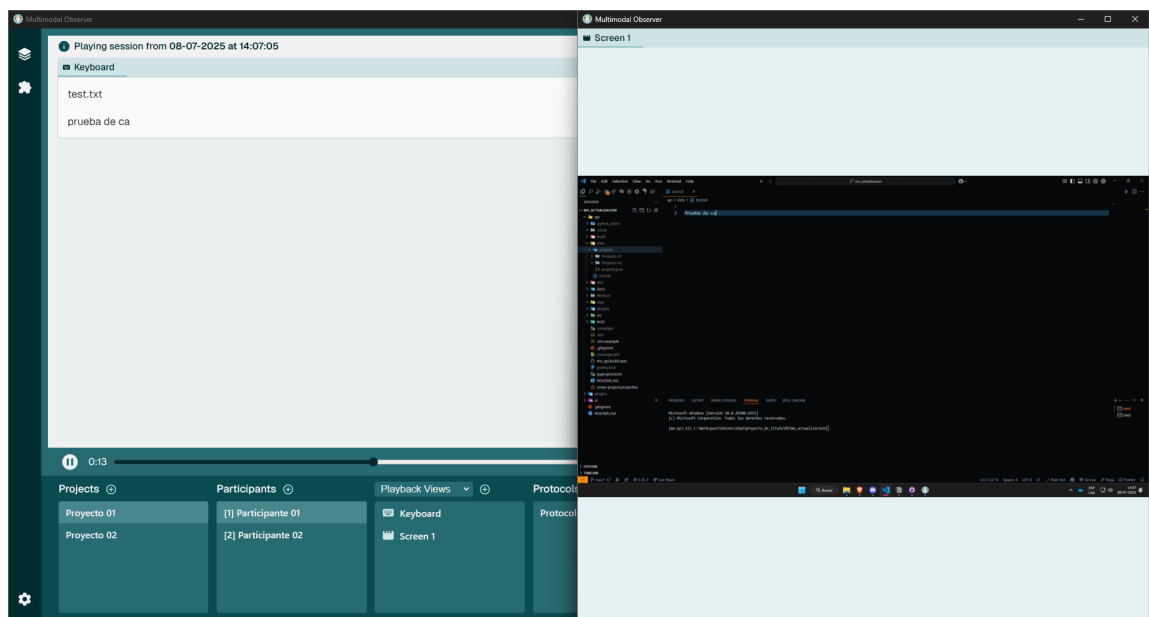


Figura 3.43: Cuarto prototipo - Ventana independiente en la reproducción de sesiones.
Fuente: Elaboración propia, 2025.

En paralelo, la interfaz de usuario fue reorganizada con el objetivo de optimizar la navegación y el acceso a las distintas secciones de la aplicación. Se reestructuró el flujo de trabajo entre proyectos, participantes, configuraciones de plugins y protocolos, mientras que la gestión de sesiones de captura y reproducción se trasladó a una página específica (Figura 3.44), y los

controles de captura y protocolos se consolidaron en una sección dedicada dentro del panel inferior (Figura 3.45).

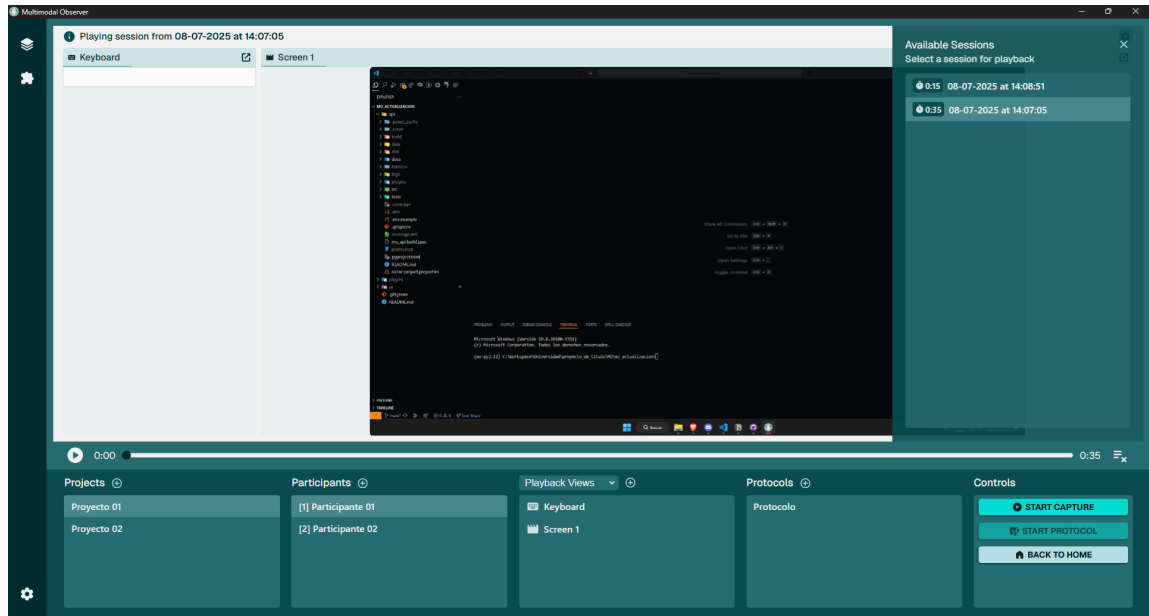


Figura 3.44: Cuarto prototipo - Sesiones disponibles.

Fuente: Elaboración propia, 2025.

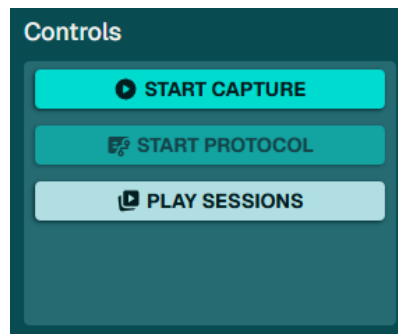


Figura 3.45: Cuarto prototipo - Sección de controles.

Fuente: Elaboración propia, 2025.

Este prototipo permitió consolidar las capacidades de MO al integrar un módulo de visualización completo con controles globales de reproducción, mejorar la gestión de plugins desde la interfaz y habilitar un entorno flexible para la revisión visual de sesiones capturadas.

3.4.5 Quinto prototipo: Preferencias de usuario y ajustes de la aplicación

El quinto prototipo se centró en incorporar una serie de opciones de configuración orientadas a personalizar el comportamiento de la aplicación, mejorar la experiencia de uso y mantener la continuidad del entorno de trabajo entre sesiones. Un resumen se presenta en la Tabla 3.7.

Tabla 3.7: Quinto prototipo - Preferencias de usuario y ajustes de la aplicación.

ID Prototipo	P-05
Nombre	Preferencias de usuario y ajustes de la aplicación
Objetivos	Incorporar una sección de ajustes para personalizar el comportamiento general de la aplicación. Permitir el cambio de idioma entre español e inglés. Implementar teclas de acceso rápido para acciones de captura. Guardar el tamaño y posición de la ventana principal entre sesiones. Agregar opciones para minimizar/restaurar la aplicación durante la captura. Integrar control de captura desde la bandeja del sistema.
Descripción	Iteración centrada en el desarrollo de una sección de ajustes desde la cual el usuario puede personalizar aspectos clave del comportamiento de la aplicación. Se implementaron funciones de persistencia visual, selección de idioma, atajos de teclado, y preferencias para el flujo de captura de datos.
Requisitos Funcionales	RF-21, RF-31, RF-32, RF-40, RF-41
Requisitos No Funcionales	RNF-01, RNF-02, RNF-03, RNF-04

Fuente: Elaboración propia, 2025.

En esta iteración se añadió un sistema de persistencia del estado de la ventana principal, que guarda su tamaño y posición al cerrar la aplicación, restaurándolos automáticamente al volver a abrirla. Esto permite retomar el trabajo en un entorno familiar, sin necesidad de reconfigurar la ventana en cada uso.

También se incorporó una integración con la bandeja del sistema (*system tray*), que permite ejecutar acciones como iniciar, pausar, reanudar o detener una captura sin necesidad de abrir la interfaz completa (Figuras 3.46 y 3.47).

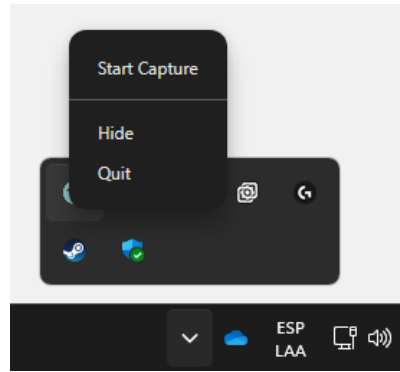


Figura 3.46: Quinto prototipo - System tray 1.
Fuente: Elaboración propia, 2025.

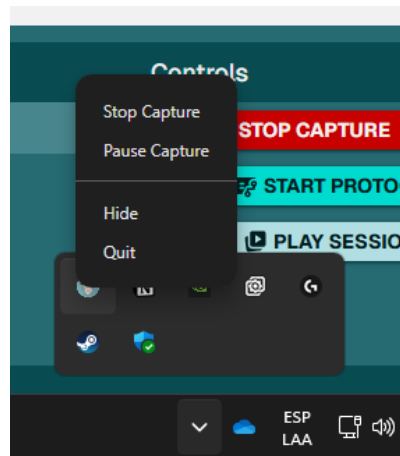


Figura 3.47: Quinto prototipo - System tray 2.
Fuente: Elaboración propia, 2025.

Por otro lado, se agregó una sección de ajustes accesible desde la interfaz principal (Figura 3.48). Esta vista permite a los usuarios cambiar el idioma de la aplicación entre español e inglés. Esta funcionalidad se aplica a toda la interfaz principal de MO y también está disponible para ser utilizada por los plugins desarrollados para la aplicación, tanto en la interfaz como en la API (Figura 3.49).

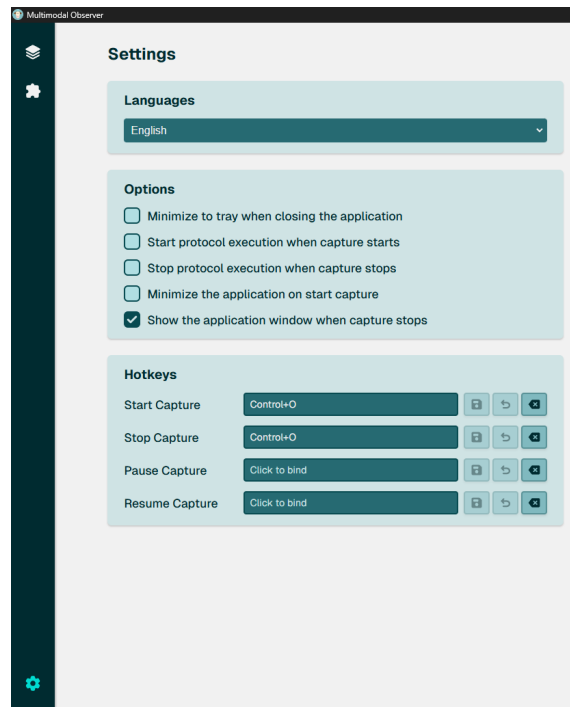


Figura 3.48: Quinto prototipo - Sección de ajustes.

Fuente: Elaboración propia, 2025.

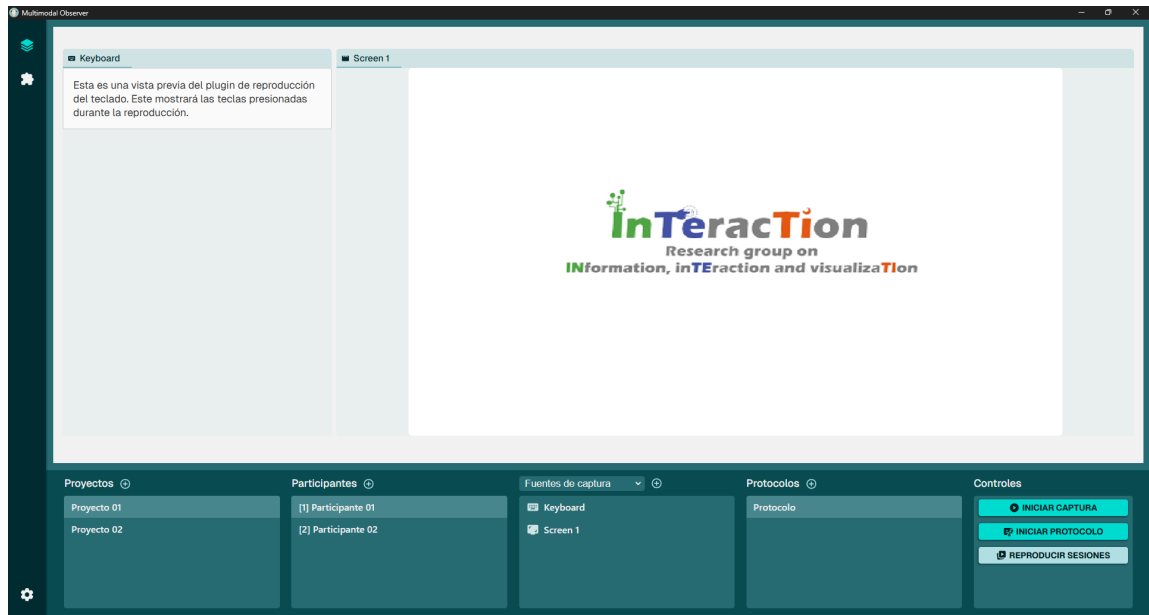


Figura 3.49: Quinto prototipo - Vista principal en español.

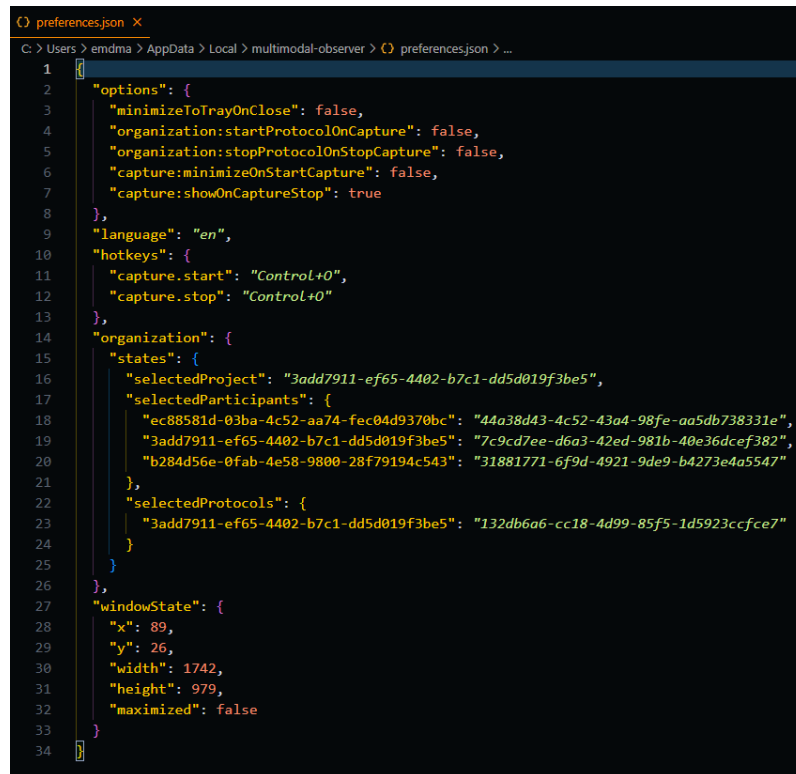
Fuente: Elaboración propia, 2025.

Dentro de esta misma sección es posible modificar diversas configuraciones generales. Una de ellas es la opción para minimizar automáticamente la aplicación al iniciar una captura y restaurarla

al finalizarla, lo que evita que la interfaz interfiera con la sesión de captura. Además, se incorporó una opción para que la ejecución del protocolo de investigación comience automáticamente al iniciar una captura y finalice al detenerla, lo que facilita acciones recurrentes durante el uso de la aplicación.

En esta versión también se agregó la posibilidad de configurar teclas de acceso rápido (hotkeys) para las acciones de captura. Los atajos definidos por el usuario se almacenan y conservan entre sesiones, ofreciendo una forma rápida y directa de operar la aplicación.

Todas estas configuraciones se almacenan automáticamente en un archivo *preferences.json* al cerrar la aplicación, garantizando que las preferencias del usuario sean persistentes entre sesiones (Figura 3.50). Dentro de este archivo, como se observa, también se persiste el contexto de trabajo, recordando el último proyecto, participante y protocolo seleccionados. Esto permite retomar el trabajo en el mismo estado en que fue dejado, sin necesidad de repetir configuraciones manuales.



```
1 2
2  "options": {
3    "minimizeToTrayOnClose": false,
4    "organization:startProtocolOnCapture": false,
5    "organization:stopProtocolOnStopCapture": false,
6    "capture:minimizeOnStartCapture": false,
7    "capture:showOnCaptureStop": true
8  },
9  "language": "en",
10 "hotkeys": {
11   "capture.start": "Control+O",
12   "capture.stop": "Control+O"
13 },
14 "organization": {
15   "states": {
16     "selectedProject": "3add7911-ef65-4402-b7c1-dd5d019f3be5",
17     "selectedParticipants": {
18       "ec88581d-03ba-4c52-aa74-fec04d9370bc": "44a38d43-4c52-43a4-98fe-aa5db738331e",
19       "3add7911-ef65-4402-b7c1-dd5d019f3be5": "7c9cd7ee-d6a3-42ed-981b-40e36dcef382",
20       "b284d56e-0fab-4e58-9800-28f79194c543": "31881771-6f9d-4921-9de9-b4273e4a5547"
21     },
22     "selectedProtocols": {
23       "3add7911-ef65-4402-b7c1-dd5d019f3be5": "132db6a6-cc18-4d99-85f5-1d5923ccfce7"
24     }
25   },
26 },
27 "windowState": {
28   "x": 89,
29   "y": 26,
30   "width": 1742,
31   "height": 979,
32   "maximized": false
33 }
34
```

Figura 3.50: Quinto prototipo - Archivo para almacenar preferencias.

Fuente: Elaboración propia, 2025.

3.4.6 Sexto prototipo: Instalador de MO y conjunto de plugins

Este prototipo se enfocó en la implementación de un instalador gráfico que automatiza el proceso de instalación del núcleo de MO y de un conjunto de plugins seleccionados. Esta iteración da cumplimiento directo al objetivo específico: “Implementar un instalador para el núcleo de MO y el conjunto de plugins existentes”. El instalador permite a los usuarios:

- Elegir los plugins que desean instalar (Figura 3.51).
- Seleccionar el directorio de instalación (Figura 3.52).
- Confirmar la finalización del proceso con la opción de ejecutar MO al finalizar (Figura 3.53).

Estas funcionalidades se resumen en la Tabla 3.8, que presenta los objetivos, descripción y requerimientos abordados durante esta iteración.

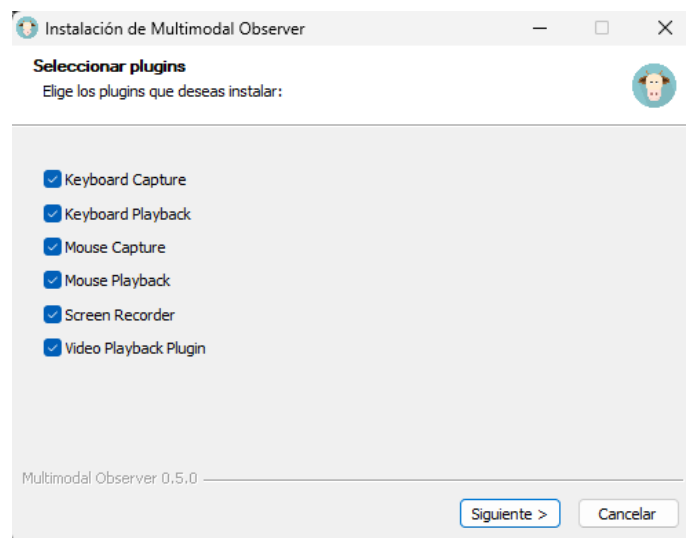


Figura 3.51: Sexto prototipo - Selección de plugins a instalar.

Fuente: Elaboración propia, 2025.

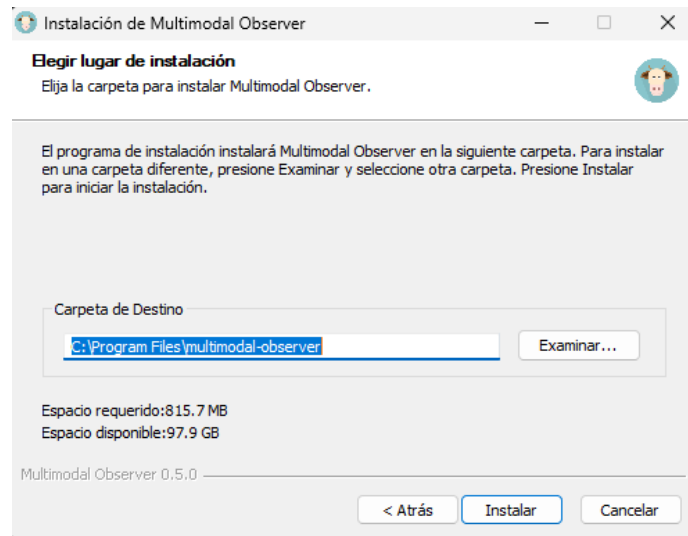


Figura 3.52: Sexto prototipo - Elección del directorio de instalación.
Fuente: Elaboración propia, 2025.

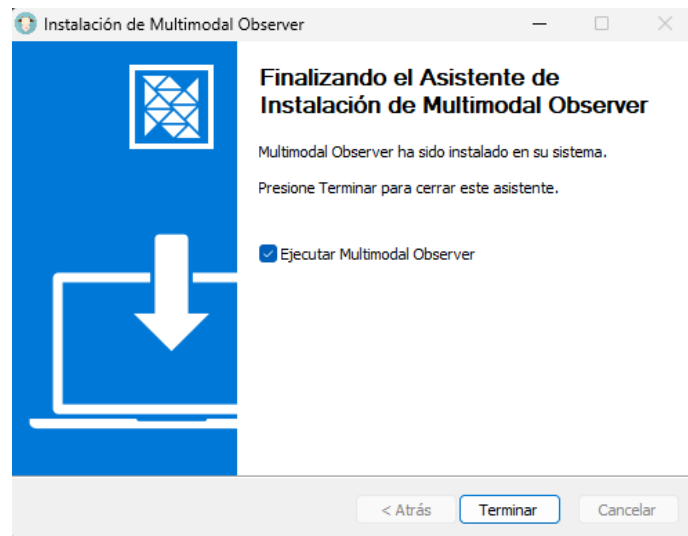


Figura 3.53: Sexto prototipo - Instalación finalizada.
Fuente: Elaboración propia, 2025.

Tabla 3.8: Sexto prototipo - Instalador de MO y el conjunto de plugins.

ID Prototipo	P-06
Nombre	Instalador del núcleo y conjunto de plugins
Objetivos	Desarrollar un instalador que incluya el núcleo de MO y un conjunto inicial de plugins. Permitir la selección de plugins a instalar. Permitir la elección del directorio de instalación. Ejecutar MO automáticamente al finalizar la instalación.
Descripción	Iteración centrada en crear un instalador gráfico para la distribución de MO. Incluye funcionalidades para elegir plugins, definir ruta de instalación y ejecutar la aplicación tras completar el proceso. Este instalador facilita la instalación del sistema en entornos reales.
Requisitos Funcionales	RF-22
Requisitos No Funcionales	RNF-01, RNF-02

Fuente: Elaboración propia, 2025.

3.5 RESUMEN

Para finalizar, la Tabla 3.9 presenta un resumen de los requerimientos funcionales abordados por cada prototipo, mientras que la Tabla 3.10 sintetiza los requerimientos no funcionales considerados durante su desarrollo. Estas tablas permiten observar de forma global cómo cada iteración contribuyó al cumplimiento progresivo de los requerimientos establecidos para el sistema.

Tabla 3.9: Resumen de requerimientos funcionales cubiertos por prototipo.

	P-01	P-02	P-03	P-04	P-05	P-06
RF-01 al RF-20		X				
RF-21					X	
RF-22	X		X			X
RF-23 al RF-30			X			
RF-31					X	
RF-32					X	
RF-33 al RF-39	X		X			
RF-40					X	
RF-41					X	
RF-42 al RF-49				X		
RF-50 al RF-52	X		X			
RF-53 al RF-55	X			X		

Fuente: Elaboración propia, 2025.

Tabla 3.10: Resumen de requerimientos no funcionales cubiertos por prototipo.

	P-01	P-02	P-03	P-04	P-05	P-06
RNF-01	X	X	X	X	X	X
RNF-02	X	X	X	X	X	X
RNF-03					X	
RNF-04		X	X	X	X	
RNF-05	X		X	X		
RNF-06	X			X		

Fuente: Elaboración propia, 2025.

CAPÍTULO 4. DISEÑO Y ASPECTOS DE IMPLEMENTACIÓN

En este capítulo se abordan los principales aspectos relacionados con el diseño y la implementación de la nueva versión de Multimodal Observer (MO). Para estructurar esta sección, se adopta el modelo de **vistas "4+1" de Kruchten (1995)**, el cual permite organizar las decisiones de diseño en torno a diferentes perspectivas del sistema, tales como su arquitectura lógica, de desarrollo, de procesos, física y de escenarios. Esta aproximación facilita comunicar tanto la estructura interna como el comportamiento del sistema desde distintos niveles de abstracción, permitiendo así una mejor comprensión.

4.1 VISTA FÍSICA

La vista física describe cómo se realiza el despliegue concreto de la aplicación, detallando la estructura de los nodos físicos y componentes que conforman su arquitectura. La Figura 4.1 muestra el diagrama de despliegue correspondiente.

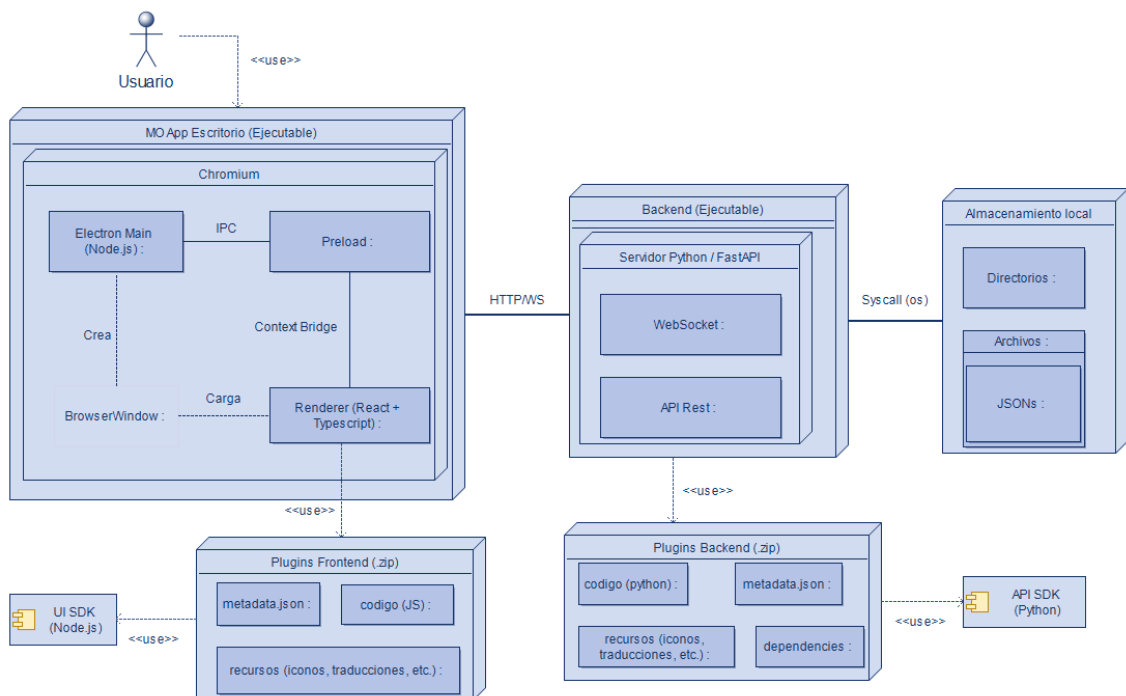


Figura 4.1: Diagrama de despliegue.

Fuente: Elaboración propia, 2025.

4.1.1 Aplicación Escritorio (Frontend)

La aplicación de escritorio MO está empaquetada como un ejecutable, generado mediante Vite y Electron Builder. Vite es responsable de transpilar todo el código fuente para hacerlo compatible con los diferentes contextos de ejecución requeridos por Electron, incluyendo el proceso principal (Main), el proceso de precarga (Preload) y el renderizador (Renderer).

El ejecutable de escritorio se basa en Electron, que a su vez utiliza internamente Chromium para presentar la interfaz gráfica mediante tecnologías web (HTML, CSS, JavaScript).

Dentro de este entorno se identifican claramente los siguientes componentes de la arquitectura de **Electron (OpenJS & Electron, s.f.)**:

- **Electron Main (Node.js)**: Es el proceso principal de la aplicación y actúa como un orquestador general, gestionando la creación y administración de múltiples ventanas (*BrowserWindows*), cada una ejecutada como un proceso independiente. **Este proceso principal se basa en Node.js, lo que le permite interactuar directamente con el sistema operativo, por ejemplo, para registrar teclas rápidas globales, integrarse con el *system tray* o acceder al sistema de archivos. Además, administra la comunicación inter-procesos (IPC) entre los distintos renderizadores y el propio proceso principal, coordinando el intercambio de datos y eventos entre ellos.**
- **Preload**: Constituye una capa intermedia entre el proceso principal de Electron y el proceso del renderizado. El script de precarga se inyecta antes de cargar las páginas del renderizado, permitiendo extender sus capacidades mediante un conjunto controlado de APIs de Node.js y Electron. Esta capa facilita la comunicación segura entre el renderizado y el proceso principal mediante el módulo IPC (*Inter Process Communication*), utilizando el módulo *Context Bridge* para exponer funcionalidades específicas del proceso principal, asegurando que el proceso del renderizador pueda acceder únicamente a las funcionalidades autorizadas, aumentando así la seguridad y el control de la aplicación.
- **Renderer (React + Typescript)**: Este componente representa todo el contenido visible para el usuario. El renderizador ejecuta código JavaScript transpilado por Vite, específicamente escrito con React y TypeScript, utilizando librerías como React-Router y Redux para gestión de navegación y estado respectivamente. El contenido generado por Vite se ejecuta en el entorno proporcionado por Chromium dentro de Electron.

Para soportar plugins del frontend se implementaron soluciones específicas que requirieron ciertos "*Workarounds*", ya que los plugins necesitan compartir la misma instancia de React que usa la aplicación principal. **Esta necesidad surge por cómo React gestiona la manipulación del**

DOM (Document Object Model), que representa la estructura jerárquica del contenido mostrado en pantalla. React mantiene un “DOM virtual” para optimizar las actualizaciones visuales, comparando cambios entre versiones y actualizando únicamente los elementos necesarios (Mateusz, 2024). Si cada plugin tuviera su propia instancia de React, se producirían conflictos al sincronizar el DOM y los *hooks* (funciones especiales de React que permiten gestionar el estado y el ciclo de vida de los componentes funcionales) generando errores en la renderización. Compartir la misma instancia de React, por tanto, garantiza una interacción coherente con el DOM y una gestión centralizada del estado global, además de reducir el tamaño de los plugins al no incluir la biblioteca en su código transpilado. Para lograr esto, se generaron archivos específicos de salida (*chunks*) que contienen dependencias comunes como React. Estos chunks luego son expuestos mediante *importmaps*, un mecanismo que permite definir rutas lógicas para los módulos JavaScript, indicando cómo y desde dónde deben cargarse (MDN, s.f.). En este contexto, los *importmaps* actúan como un mapa de claves (*mapkeys*) que asocia nombres simbólicos con las ubicaciones reales de los recursos compartidos, asegurando que los plugins puedan importar correctamente las dependencias comunes durante la ejecución dinámica sin duplicar bibliotecas ni generar conflictos de carga.

4.1.2 Plugins del Frontend

Los plugins de interfaz están encapsulados en archivos *.zip* y desarrollados mediante un SDK (*Software Development Kit*) específico para la interfaz gráfica (UI SDK). Este SDK proporciona las interfaces necesarias para integrar plugins, herramientas para transpilar el código, empaquetarlo y dejarlo listo para su uso en la aplicación. Además, incluye documentación, pautas de desarrollo y utilidades que facilitan la creación de nuevos plugins de manera estructurada y consistente. Cada plugin contiene:

- **metadata.json:** Información clave como identificadores, versión, descripción, iconos, traducciones y puntos de entrada para la carga dinámica.
- **Código:** Tiene todo lo funcional del plugin, y tiene que ser transpilado mediante Vite para generar los archivos JavaScript correspondientes
- **Recursos adicionales:** Iconos, traducciones, imágenes, entre otros que necesite el plugin.

4.1.3 Backend (Servidor FastAPI)

El backend se distribuye como un ejecutable generado mediante PyInstaller, lanzando un servidor HTTP construido con Python y FastAPI usando Uvicorn. Uvicorn es un servidor web rápido basado en asyncio, diseñado para ejecutar aplicaciones asincrónicas desarrolladas con Python (Trylesinski, s.f.). FastAPI es un framework moderno para desarrollar APIs RESTful de forma rápida y eficiente, utilizando funciones asíncronas (*async-await*) para manejar peticiones concurrentes sin bloquear la aplicación durante operaciones intensivas (Ramírez, s.f.). Además, FastAPI ofrece soporte nativo para WebSockets, lo que permite establecer conexiones persistentes entre cliente y servidor, facilitando una comunicación en tiempo real entre frontend y backend cuando sea necesario.

4.1.4 Plugins del Backend

Al igual que los plugins de interfaz, los plugins del backend están empaquetados como archivos *.zip* generados mediante un SDK especializado (API SDK) construido en Python. Cada plugin incluye:

- **metadata.json:** Información clave como identificadores, versión, descripción, iconos, traducciones y puntos de entrada para la carga dinámica.
- **Código:** En archivos Python que implementan las funcionalidades específicas.
- **Dependencias:** En una carpeta *dependencies* contiene todo el código que usa el plugin como dependencia y que deberá ser cargado por el backend para que este disponible para su uso.
- **Recursos adicionales:** Iconos, traducciones, imágenes, entre otros que necesite el plugin.

Cada plugin del backend es lanzado como un proceso independiente en tiempo de ejecución, lo que incrementa la modularidad y aislamiento en el sistema (más detalle en la sección 4.4)

4.1.5 Almacenamiento Local

El almacenamiento local es gestionado directamente por el backend, utilizando llamadas al sistema operativo (syscalls OS). Este nodo permite almacenar, actualizar y recuperar datos en

forma de directorios, archivos JSON y otros formatos que MO requiere para operar y persistir información entre sesiones.

La división clara entre frontend y backend propuesta por esta arquitectura permite aislar completamente la interfaz de usuario de la lógica de negocio, proporcionando una separación efectiva de responsabilidades. Esto simplifica el desarrollo, facilita el mantenimiento, mejora la escalabilidad y permite utilizar diferentes tecnologías de manera independiente en cada capa, mejorando así la flexibilidad y la eficiencia general del sistema.

4.2 VISTA DE DESARROLLO

Multimodal Observer (MO) presenta una organización interna que sigue los principios de modularidad, extensibilidad y escalabilidad heredados desde su primera versión. Esta estructura original consiste en una separación clara entre tres conceptos principales: núcleo, módulos y plugins, que en conjunto permiten desarrollar un sistema flexible y adaptable a distintas necesidades. En esta nueva versión de MO, además se introduce una división adicional entre frontend (interfaz de usuario) y backend (API), cuyo objetivo principal es desacoplar la lógica de presentación de la lógica del negocio y almacenamiento. Esta separación promueve una evolución independiente de ambas partes y mejora la mantenibilidad general del sistema. La Figura 4.2 muestra el diagrama de componentes que resume esta estructura.

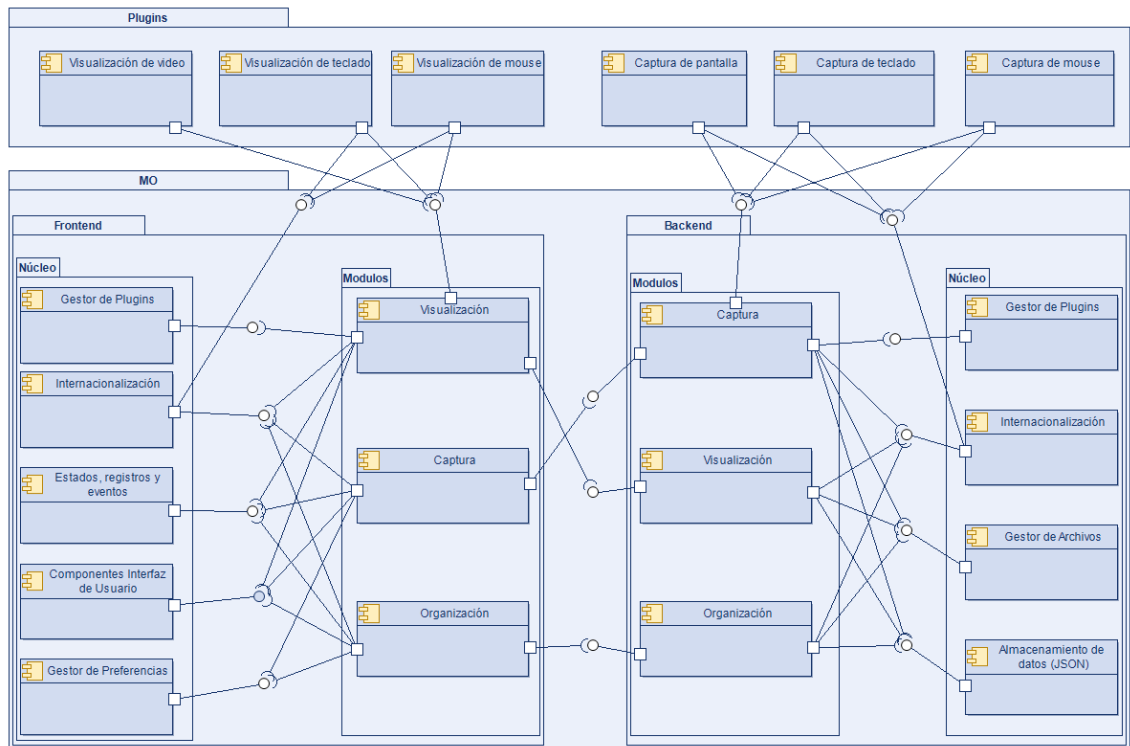


Figura 4.2: Diagrama de componentes.

Fuente: Elaboración propia, 2025.

4.2.1 Frontend

En el frontend se encuentran tanto el núcleo como los módulos visuales. El núcleo tiene un carácter transversal, es decir, incluye funcionalidades que pueden ser utilizadas de manera general por todos los módulos. Los principales componentes del núcleo en el frontend se resumen en la Tabla 4.1

Tabla 4.1: Componentes del núcleo en el frontend.

Componentes	Descripción
Gestor de Plugins	Administra dinámicamente plugins visuales: descubre, carga, inicializa y registra plugins. Además, proporciona interfaces base para el desarrollo de plugins y gestión de configuraciones.
Internacionalización	Proporciona herramientas para implementar soporte multilingüaje en la interfaz y plugins visuales, facilitando la adaptación a múltiples idiomas.
Estados, registros y eventos	Provee mecanismos para compartir estados globales mediante Redux entre módulos, permitiendo el intercambio eficiente de datos. Además, gestiona el registro dinámico de componentes visuales en áreas comunes de la interfaz, fomentando un diseño desacoplado. También permite la gestión de eventos entre módulos, facilitando así una comunicación interna clara y flexible, sin introducir dependencias entre módulos.
Componentes de interfaz	Provee elementos visuales comunes (botones, formularios, iconos), garantizando coherencia visual, reduciendo esfuerzo en desarrollo, y facilitando escalabilidad.
Gestor de Preferencias	Ofrece un mecanismo centralizado para gestionar las configuraciones y preferencias del usuario, simplificando su integración, acceso y almacenamiento persistente por parte de los distintos módulos.

Fuente: Elaboración propia, 2025.

Los módulos del frontend encapsulan características específicas dentro de su dominio de aplicación y se resumen en la Tabla 4.2:

Tabla 4.2: Módulos específicos del frontend.

Modulo	Descripción
Visualización	Encargado de gestionar la reproducción y visualización de sesiones de captura. Este módulo coordina el funcionamiento de diversos plugins visuales que muestran los datos capturados durante una sesión.
Captura	Responsable de las acciones de captura desde la interfaz. Permite configurar y controlar el estado de las capturas de datos mediante interacción directa con el usuario y comunicación con el backend.
Organización	Se ocupa de administrar elementos relacionados con los proyectos de investigación, participantes y protocolos. Ofrece una interfaz gráfica para realizar todas las operaciones necesarias para la gestión y ejecución de estos elementos, incluyendo la interacción con otros módulos y con el backend.

Fuente: Elaboración propia, 2025.

4.2.2 Backend

En el backend, existe nuevamente una separación entre núcleo y módulos específicos. El núcleo del backend reúne componentes transversales reutilizables esenciales para la operación del

sistema, los cuales se describen brevemente en la Tabla 4.3:

Tabla 4.3: Componentes del núcleo en el backend.

Componentes	Descripción
Gestor de Plugins	Administra dinámicamente plugins para la API, encargándose de descubrir, cargar, inicializar y gestionar estos componentes externos. Además, proporciona interfaces base para el desarrollo de plugins y gestión de configuraciones.
Internacionalización	Maneja mensajes del backend en múltiples idiomas, facilitando el desarrollo de plugins y módulos que requieran soporte multilinguaje.
Gestor de Archivos	Gestiona todas las operaciones relacionadas con archivos y directorios requeridas por la aplicación.
Almacenamiento de datos (JSON)	Proporciona almacenamiento persistente usando estructuras JSON, ofreciendo operaciones básicas de consulta, inserción, modificación y eliminación.

Fuente: Elaboración propia, 2025.

Los módulos específicos del backend implementan funcionalidades concretas de la lógica del negocio del sistema, resumidas en la Tabla 4.4:

Tabla 4.4: Módulos específicos del backend.

Modulo	Descripción
Captura	Administra y coordina el proceso completo de captura de datos, incluyendo la interacción con plugins especializados en captura multimodal. Este módulo también maneja la configuración específica del usuario respecto a los plugins de captura.
Visualización	Gestiona principalmente configuraciones de usuario relacionadas con los plugins visuales.
Organización	Contiene la lógica fundamental para la administración de proyectos de investigación, participantes y protocolos, incluyendo la ejecución controlada de protocolos.

Fuente: Elaboración propia, 2025.

4.2.3 Plugins

La capa superior se constituye por plugins, que son componentes externos que extienden y personalizan funcionalidades del sistema en áreas específicas. Estos plugins implementan interfaces definidas por los módulos correspondientes (captura o visualización) y utilizan recursos del núcleo, como la internacionalización, para adaptarse mejor a las necesidades de cada usuario y proyecto.

Esta organización general, caracterizada por la clara separación entre núcleo, módulos y plugins, así como por la división frontend-backend, permite que MO evolucione de manera flexible, facilite

la incorporación de nuevas funcionalidades y se adapte eficazmente a diversos contextos de investigación y observación multimodal.

4.3 VISTA LÓGICA

La vista lógica describe detalladamente la estructura interna del sistema, especificando las clases y componentes principales, sus responsabilidades, relaciones y aspectos importantes de implementación. Esto permite comprender cómo interactúan los elementos fundamentales del núcleo para mantener la modularidad y extensibilidad del sistema.

4.3.1 Núcleo

El núcleo contiene las clases esenciales y funcionalidades base utilizadas por todos los módulos. Proporciona estructuras y mecanismos comunes que garantizan la modularidad, extensibilidad y consistencia en el manejo de plugins y otros componentes fundamentales.

Se puede ver la representación del núcleo para el backend en la Figura 4.3, y para el frontend en la Figura 4.4. Además, los diagramas de clases detallados se encuentran en el Apéndice B, en las Figuras B.1 y B.2, respectivamente.

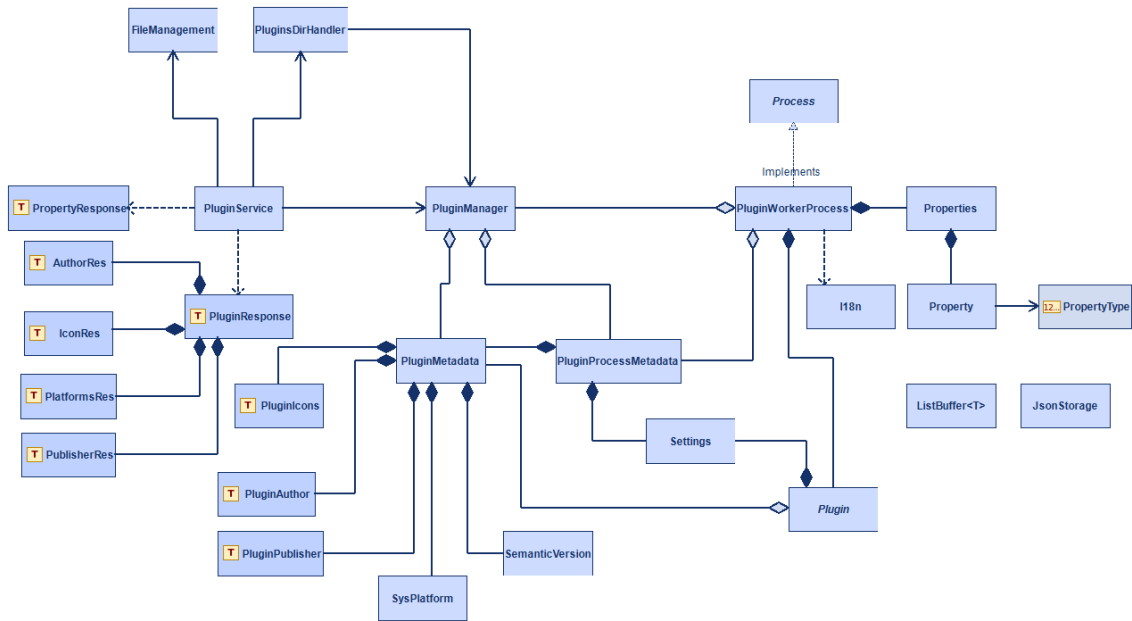


Figura 4.3: Diagrama de clases del Núcleo (backend).

Fuente: Elaboración propia, 2025.

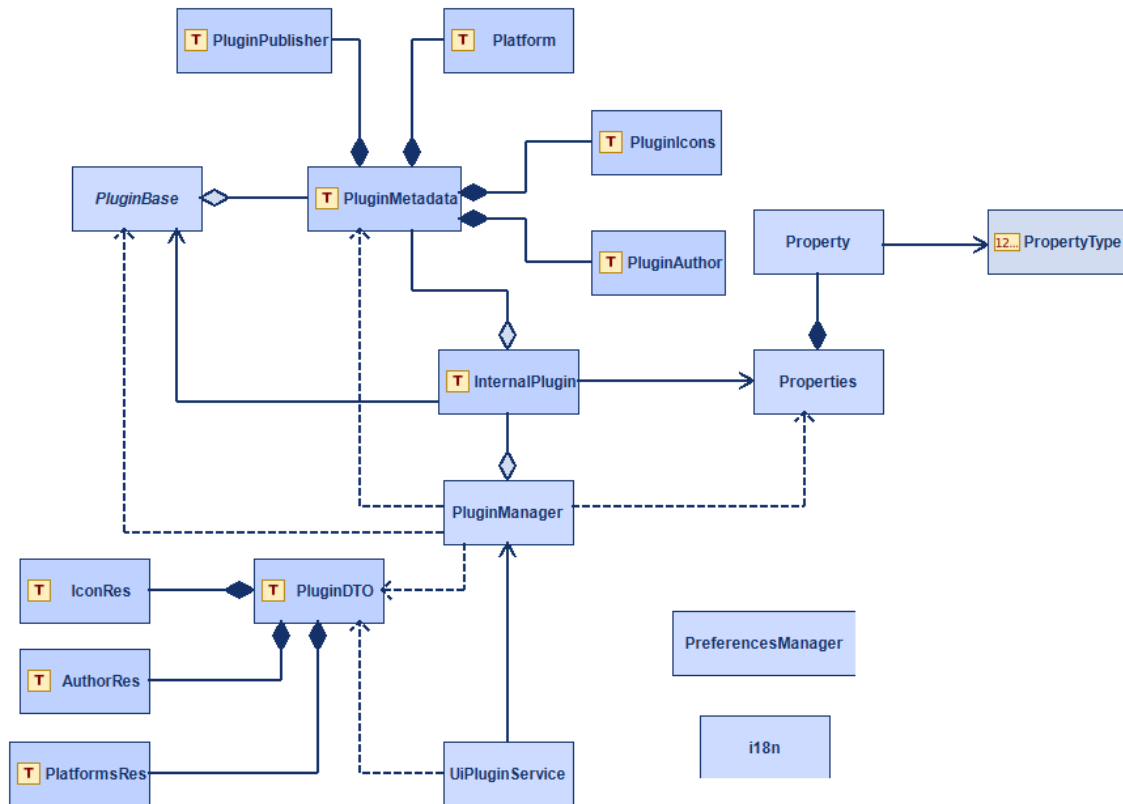


Figura 4.4: Diagrama de clases del Núcleo (frontend).

Fuente: Elaboración propia, 2025.

Además, en las Tablas 4.5 y 4.6 se presenta la descripción para cada una de las clases que se observan en los diagramas.

Tabla 4.5: Descripción de clases del núcleo (backend).

Clase	Descripción
PluginManager	Gestiona de forma centralizada el ciclo de vida de los plugins, incluyendo su carga, inicialización y disposición global. Se implementa como singleton para asegurar consistencia y disponibilidad única.
PluginProcessMetadata	Contiene información necesaria para identificar y gestionar procesos asociados a la ejecución de plugins.
PluginWorkerProcess	Ejecuta un plugin en un proceso separado, permitiendo crear instancias del plugin, gestionar su configuración y ejecutar callbacks de manera aislada.
PluginsDirHandler	Monitorea el directorio local de plugins para detectar incorporaciones manuales y facilitar su integración dinámica al sistema.
PluginAuthor	Estructura de datos que representa al autor de un plugin.
PluginIcons	Define las rutas a los íconos utilizados por el plugin.
PluginPublisher	Contiene información del publicador de un plugin.
PluginMetadata	Describe los metadatos clave de un plugin.
Plugin	Clase base abstracta que define la interfaz obligatoria de los plugins, como carga, descarga y configuración. Debe ser extendida por todos los plugins desarrollados para MO.
PropertyType	Enumeración que representa los distintos tipos de propiedades configurables en un plugin.
Property	Define una propiedad configurable individual dentro de un plugin.
Properties	Agrupación de múltiples propiedades utilizadas para configurar visualmente un plugin desde la interfaz.
SemanticVersion	Representa una versión semántica siguiendo el formato estándar <code>major.minor.patch</code> .
Settings	Permite almacenar, recuperar y actualizar configuraciones específicas de un plugin en formato JSON.
SysPlatform	Enumera las plataformas del sistema soportadas por un plugin, como Linux, Windows o macOS.
FileManagement	Encapsula operaciones de sistema de archivos, como creación, renombrado o eliminación de archivos y carpetas, soportando ejecución asíncrona.

Clase	Descripción
JsonStorage	Implementa almacenamiento simple basado en archivos JSON, incluyendo mecanismos de bloqueo para evitar condiciones de carrera.
ListBuffer	Buffer seguro para el almacenamiento temporal y concurrente de elementos en estructuras compartidas entre hilos y procesos.
I18n	Conjunto de funciones utilizadas para internacionalización mediante la librería i18nice, permitiendo la traducción dinámica según el idioma activo.
PlatformsRes	Modelo de datos que expone las plataformas soportadas por un plugin a través de la API.
AuthorRes	Modelo que representa los datos del autor de un plugin en las respuestas API.
PublisherRes	Modelo que contiene la información pública del publicador de un plugin.
PluginResponse	Modelo de respuesta que entrega toda la información de un plugin, incluyendo metadatos y estado.
PropertyResponse	Modelo que representa una propiedad de configuración en una respuesta de la API.
PluginService	Servicio centralizado que permite gestionar plugins mediante rutas expuestas en la API REST: instalación, obtención de propiedades, eliminación y lectura de metadatos.

Fuente: Elaboración propia, 2025.

Tabla 4.6: Descripción de clases del núcleo (frontend).

Clase	Descripción
PluginManager	Clase encargada de gestionar los plugins del frontend, incluyendo su registro y carga.
PluginMetadata	Contiene los metadatos esenciales de un plugin.
PluginPublisher	Representa los datos del publicador del plugin.
Platform	Define las plataformas en las que un plugin puede ser ejecutado (Windows, Linux, macOS).
PluginIcons	Representa las rutas a los íconos del plugin.
PluginAuthor	Contiene información sobre el autor del plugin.
Property	Representa una propiedad configurable individual.
Properties	Agrupación de múltiples propiedades utilizadas para configurar visualmente un plugin desde la interfaz.

Clase	Descripción
PropertyType	Enumeración de los tipos de propiedades disponibles.
PluginDTO	Objeto de transferencia de datos que encapsula la información del plugin.
IconRes	Recurso que define las rutas visuales de los íconos del plugin.
AuthorRes	Modelo de datos que representa al autor del plugin.
PlatformsRes	Modelo que describe las plataformas compatibles con el plugin.
InternalPlugin	Representa los datos internos utilizados por el sistema para el seguimiento y control del estado de plugins en ejecución.
I18n	Módulo de internacionalización del frontend, encargado de traducir la interfaz de usuario y los plugins, gestionando recursos en múltiples idiomas.
PreferencesManager	Responsable de gestionar las preferencias del usuario, incluyendo almacenamiento persistente, recuperación y modificación de configuraciones.
UiPluginService	Servicio encargado de gestionar plugins de interfaz de usuario, permitiendo su registro, carga, obtención, validación y eliminación mediante el uso del sistema de archivos y el gestor de plugins.

Fuente: Elaboración propia, 2025.

Aspectos del Sistema de Plugins

El sistema de plugins fue diseñado para proporcionar una arquitectura extensible y robusta, tanto en el frontend como en el backend. Uno de los pilares de esta implementación es el uso del patrón de diseño *Singleton* en las clases `PluginManager` y `PluginsDirHandler`, cuyo propósito es garantizar que exista una única instancia activa de estas a lo largo de toda la aplicación. Esto permite centralizar el estado y comportamiento asociados a la gestión de plugins, asegurando consistencia y evitando conflictos durante la carga, actualización o eliminación de los mismos. Esta estrategia resulta particularmente útil al mantener en memoria los datos de los plugins actualmente instalados y su estado, evitando así duplicaciones o desincronizaciones entre componentes del sistema.

Un componente clave que habilita la flexibilidad del sistema es el uso de entry points, definidos dentro del archivo `metadata.json` de cada plugin. Un entry point representa una referencia explícita al punto de entrada de una clase, función o componente específico que se debe

cargar dinámicamente. En el caso de los plugins del backend, estos entry points utilizan el formato estándar de Python (`modulo.ruta:NombreClase`), lo que permite al `PluginManager` y al `PluginWorkerProcess` localizar e instanciar las clases necesarias sin requerir una referencia directa en el código base de MO. Esta arquitectura no solo permite desacoplar el sistema principal de las implementaciones específicas de los plugins, sino que también facilita la compatibilidad futura: si en algún momento se requieren nuevas versiones o estructuras de plugins, se pueden introducir nuevos entry points (por ejemplo, `mo.plugin.v2`) sin afectar los ya existentes. De este modo, el sistema puede evolucionar sin romper la compatibilidad con plugins previamente desarrollados.

En relación con la ejecución de los plugins del backend, se utiliza la clase `PluginWorkerProcess`, la cual hereda de `Process` del módulo `multiprocessing` de Python. Esta clase permite ejecutar cada plugin en un proceso aislado, lo que proporciona dos ventajas principales: por un lado, evita conflictos entre dependencias cuando distintos plugins requieren versiones diferentes de una misma librería, y por otro, aísla posibles errores o caídas que puedan afectar al sistema principal. El método `run()` es el encargado de iniciar el ciclo de vida del proceso, el cual se mantiene vivo a la espera de instrucciones. Para facilitar la comunicación entre el proceso principal y el creado, se utilizan los objetos `parent_conn` y `child_conn`, que establecen un canal bidireccional entre ambas instancias (proceso hijo y padre). Gracias a este mecanismo, es posible invocar funciones dentro del proceso aislado, como por ejemplo `add_plugin_instance`, que permite registrar nuevas instancias de un plugin en ejecución.

Una funcionalidad particularmente relevante en este contexto es la capacidad del `PluginWorkerProcess` para ejecutar callbacks sobre las instancias de plugin previamente cargadas. Estas funciones permiten reaccionar ante eventos o comandos específicos, ejecutando lógica dentro del contexto del proceso del plugin. Junto a esto, se puede pasar una cola de comunicación (`Queue`) que permite a los procesos e incluso a hilos internos enviar datos de forma segura y asíncrona al exterior. Este mecanismo será especialmente importante en el módulo de captura, donde se requiere transmitir eventos en tiempo real.

El diseño del archivo `metadata.json` utilizado por cada plugin también sigue una estructura inspirada en el sistema de extensiones de Visual Studio Code, anticipando la posible creación futura de un repositorio centralizado de plugins. Este diseño define un identificador único para cada plugin basado en dos partes: el identificador del publicador y el identificador del plugin, dando como resultado un ID compuesto en formato "`publisher_id.plugin_id`" (Code, 2025). Esta convención favorece la organización, evita colisiones de nombres y facilita la trazabilidad de origen de cada plugin.

La configuración de los plugins se gestiona mediante un sistema de propiedades (`Properties`), este sistema está basado en el que implementa OBS Studio, y permite generar automáticamente

interfaces de configuración dinámicas (Bailey, s.f.). Cada propiedad (Property) representa un campo configurable (como texto, número, opción booleana, lista desplegable, entre otros), y puede incluir un callback de modificación (`modified_callback`) que permite actualizar dinámicamente otras propiedades en respuesta a un cambio. Esta capacidad reactiva hace que los formularios de configuración de los plugins puedan adaptarse en tiempo real al flujo de configuración del usuario. El sistema está diseñado para ser extensible, de modo que si se requiere un nuevo tipo de propiedad, basta con extender la enumeración `PropertyType` y registrar la lógica correspondiente en la clase `Properties`.

Por su parte, el sistema de plugins del frontend mantiene una estructura similar, compartiendo el uso del archivo `metadata.json` y el campo `target` para indicar si el plugin está destinado a la API ("api") o a la interfaz de usuario ("ui"). A diferencia del backend, los plugins de frontend no se ejecutan en procesos separados, sino que son cargados dentro del contexto de ejecución de la interfaz. Esto implica que todas sus dependencias deben estar incluidas o importadas dentro del archivo `.js` generado. En cuanto a los entry points, la estructura cambia levemente: en este caso, se utiliza el formato `ruta/del/archivo.js#ExportName` para especificar el componente exportado a utilizar. Si el export es por defecto (default), el símbolo `#` no es necesario. Esta convención asegura una carga clara y precisa de los componentes que serán utilizados por la aplicación.

Otros aspectos de implementación

Además de los componentes encargados directamente del sistema de plugins, existen otras clases y utilidades dentro del núcleo que cumplen funciones esenciales para garantizar el buen funcionamiento, la eficiencia del sistema y la experiencia del usuario. Uno de los elementos fundamentales es la clase `FileManagement`, responsable de ejecutar operaciones sobre el sistema de archivos, como la creación, eliminación, renombrado y organización de carpetas y archivos dentro del entorno de la aplicación. Esta clase ha sido diseñada utilizando las capacidades asincrónicas de Python (`async/await`), lo que permite realizar operaciones de entrada/salida (I/O) sin bloquear el hilo principal de la API. Esta característica es aprovechada gracias a la compatibilidad de FastAPI con operaciones asincrónicas, permitiendo mejorar el rendimiento del backend durante tareas intensivas en disco. De esta forma, el sistema puede atender múltiples peticiones concurrentes mientras realiza procesos de escritura o lectura, evitando caídas de rendimiento o bloqueos innecesarios. En lo referente a la internacionalización, MO implementa mecanismos tanto en el backend como en la interfaz de usuario para garantizar una experiencia multilingüe. En el backend, se emplea la librería `i18nice`, que ofrece funciones

de traducción de textos mediante archivos estructurados por lenguaje y namespace. Cada plugin define en su metadata la ruta a su carpeta de traducciones (*locales*) y, a partir de ahí, el `PluginWorkerProcess` se encarga de cargar estos recursos en función del idioma activo. La estructura esperada para los archivos de traducción es `path/to/locales/lang/namespace.json`, permitiendo al plugin invocar la función `translate` para recuperar los textos localizados de forma contextual. En el frontend, el sistema de internacionalización opera bajo una lógica similar, pero con una distinción importante: dado que tanto el proceso principal de Electron (`main`) como el renderizador (`renderer`) coexisten en el mismo entorno de ejecución, se implementaron dos instancias separadas de carga de recursos (i18n) que se comunican entre sí. El proceso principal se encarga de cargar todos los archivos de traducción (tanto del sistema como de los plugins), y posteriormente comparte estos recursos con el renderizador mediante mecanismos de comunicación explicados en la sección 4.1. Para evitar colisiones entre distintos archivos de traducción, se estableció como convención que cada plugin del frontend utilice como namespace el valor `"publisher_id-plugin_id"`, y que sus traducciones estén contenidas en un archivo único llamado `translations.json` ubicado dentro de la carpeta correspondiente al idioma. Esta estrategia asegura consistencia, escalabilidad y prevención de conflictos entre plugins que eventualmente utilicen un mismo namespace.

4.3.2 Módulo de Organización

El módulo de organización se encarga de administrar los elementos esenciales para definir la estructura inicial de una sesión de captura, permitiendo la creación, edición y configuración de proyectos, participantes y protocolos. La implementación del módulo se construye principalmente sobre servicios que hacen uso de clases del núcleo para almacenamiento, bloqueo y manipulación segura de datos.

En la Figura 4.5 se presenta un diagrama de clases que muestra las entidades y relaciones involucradas en este módulo, incluyendo tanto clases propias como aquellas reutilizadas desde el núcleo (divididas por colores como se ve en la leyenda incluida en el diagrama). Para una vista detallada del diseño, se puede consultar el Apéndice B, Figura B.3.

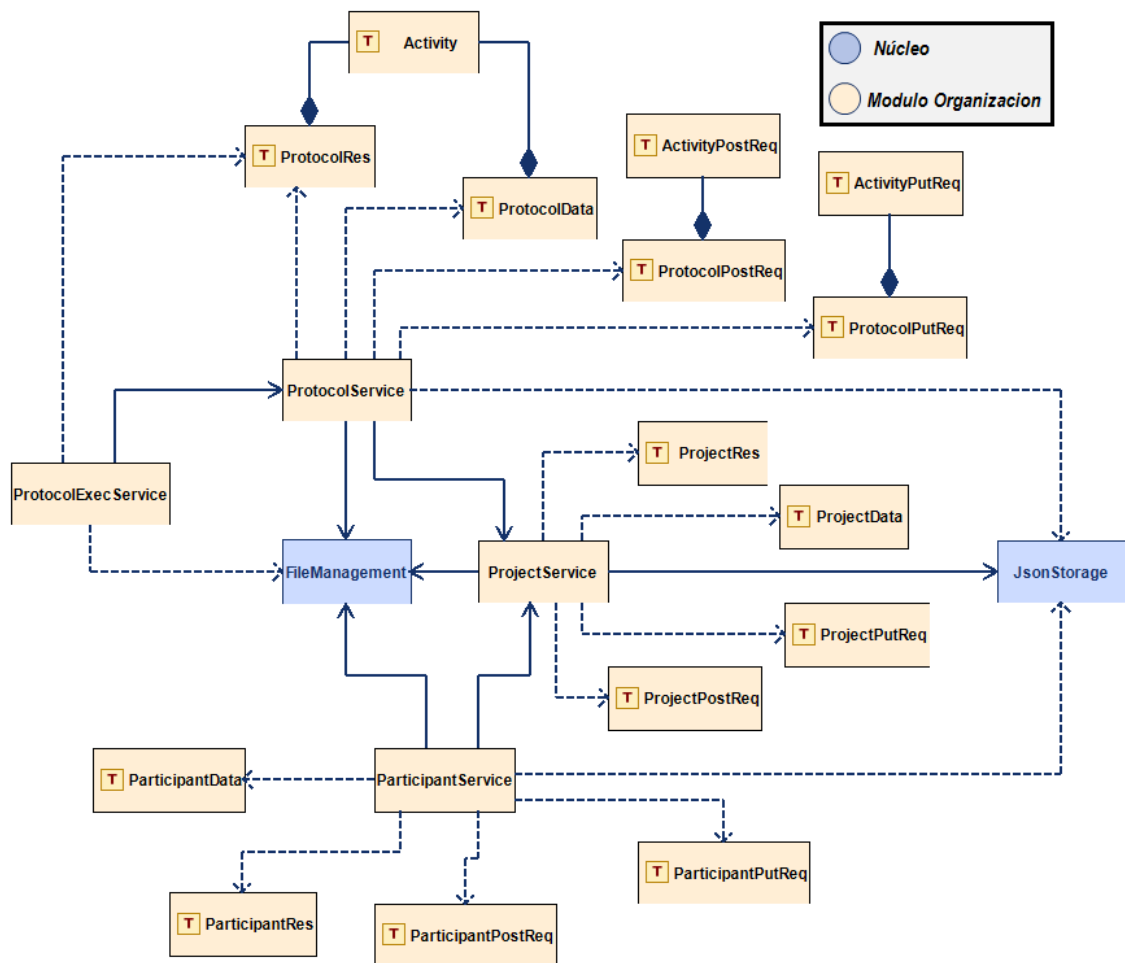


Figura 4.5: Diagrama de clases del módulo de Organización.

Fuente: Elaboración propia, 2025.

Además, en la Tabla 4.7 se presentan las principales clases que conforman este módulo y sus respectivas descripciones

Tabla 4.7: Descripción de clases del módulo de organización.

Clase	Descripción
ProjectService	Servicio que permite gestionar proyectos: creación, edición, eliminación y bloqueo. Los proyectos se almacenan como carpetas en el sistema de archivos, acompañados de archivos JSON que contienen su metadata.

Clase	Descripción
ParticipantService	Servicio encargado de gestionar participantes asociados a un proyecto, incluyendo operaciones CRUD y bloqueo. Cada participante es representado como un directorio con su metadata asociada en formato JSON.
ProtocolService	Permite crear, editar, eliminar y bloquear protocolos de investigación. Los protocolos se almacenan como archivos JSON dentro del directorio del proyecto.
ProtocolExecService	Servicio responsable de la ejecución secuencial de las actividades definidas en un protocolo, notificando el avance al frontend mediante WebSocket.
ProjectData	Estructura que representa internamente los datos de un proyecto, utilizada para carga, persistencia y manipulación de su estado.
ProjectRes	Modelo utilizado para retornar información estructurada de un proyecto en las respuestas de la API.
ProjectPostReq	Modelo que representa el cuerpo de las solicitudes de creación de un proyecto.
ProjectPutReq	Modelo que representa las solicitudes de actualización parcial o completa de un proyecto existente.
ParticipantData	Modelo que describe los datos internos que representan a un participante dentro de un proyecto.
ParticipantRes	Modelo de respuesta utilizado por la API para representar un participante.
ParticipantPostReq	Modelo que representa el cuerpo de las solicitudes para crear un nuevo participante.
ParticipantPutReq	Modelo utilizado en solicitudes de edición de participantes.
ProtocolData	Clase que encapsula los datos de un protocolo de investigación, como sus actividades, configuración y restricciones.
ProtocolRes	Modelo de respuesta que expone un protocolo y sus detalles a través de la API.
ProtocolPostReq	Modelo que representa los datos requeridos para la creación de un protocolo.
ProtocolPutReq	Modelo para actualizar los datos de un protocolo existente.
Activity	Clase que representa una actividad que forma parte de un protocolo. Puede incluir configuraciones y parámetros específicos

Clase	Descripción
ActivityPostReq	Modelo de solicitud para crear una nueva actividad dentro de un protocolo.
ActivityPutReq	Modelo para editar una actividad específica.
ProtocolExecMsg	Estructura de datos utilizada para enviar mensajes al frontend durante la ejecución de un protocolo (por WebSocket), informando tiempo y avance.

Fuente: Elaboración propia, 2025.

Aspectos de Implementación

El módulo de organización se construye sobre servicios expuestos mediante la API, donde cada uno de los servicios (ProjectService, ParticipantService, ProtocolService, ProtocolExecService) actúa como capa de lógica de negocio para las rutas REST definidas en FastAPI. Estos servicios utilizan componentes del núcleo, como FileManagement y JsonStorage, para realizar operaciones sobre el sistema de archivos y gestionar el almacenamiento de los datos asociados al módulo.

Un aspecto distintivo del módulo es el servicio de ejecución de protocolos (ProtocolExecService), que opera sobre una conexión WebSocket en lugar de HTTP convencional. Esta elección permite una ejecución continua, controlada y en tiempo real de las actividades del protocolo, comunicando constantemente el estado y el tiempo transcurrido de cada actividad hacia el frontend. En casos donde la actividad lo permita, el servidor envía actualizaciones periódicas sobre el tiempo restante, y avanza automáticamente a la siguiente etapa del protocolo según las condiciones definidas. Este proceso completo será descrito con mayor detalle en la sección 4.4.

Por el lado del frontend, las operaciones relacionadas con el módulo de organización se realizan principalmente a través de solicitudes REST a las rutas de la API, las cuales invocan los servicios descritos previamente. Para el caso de la ejecución de protocolos, se establece una conexión WebSocket que permite recibir en tiempo real las actualizaciones del estado de cada actividad durante su ejecución. La lógica de presentación y la interacción con el usuario en este módulo serán abordadas en la sección 4.4.

4.3.3 Módulo de Captura

El módulo de captura se encarga de gestionar la recolección de datos durante sesiones de estudio, operando en conjunto con el sistema de plugins definido en el núcleo para facilitar la incorporación de fuentes de captura especializadas. Además, colabora estrechamente con el módulo de organización para registrar correctamente la información contextual de cada sesión en el respectivo proyecto y participante asociados.

La Figura 4.6 presenta el diagrama de clases del módulo, en el que se distinguen las clases propias del módulo de captura y aquellas reutilizadas desde el núcleo o el módulo de organización. Para una vista detallada del diseño, se puede consultar el Apéndice B, Figura B.4. En complemento, la Tabla 4.8 presenta una descripción de las clases que conforman este módulo.

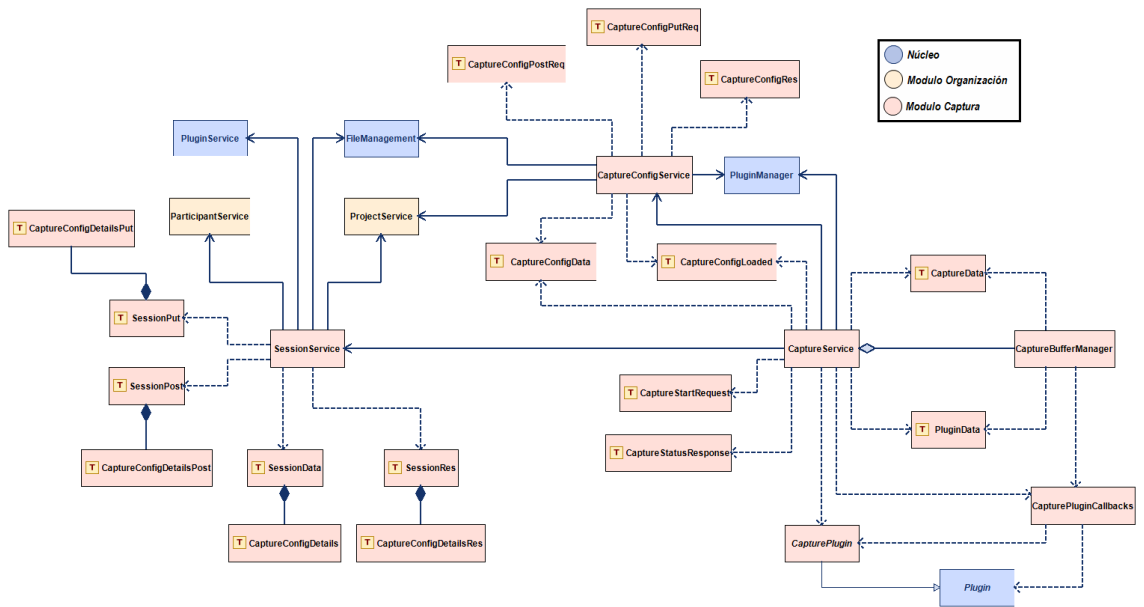


Figura 4.6: Diagrama de clases del módulo de Captura.

Fuente: Elaboración propia, 2025.

Tabla 4.8: Descripción de clases del módulo de captura.

Clase	Descripción
SessionService	Servicio encargado de gestionar las sesiones de captura dentro de un proyecto y participante. Administra la creación, actualización y eliminación de sesiones, así como sus datos asociados.

Clase	Descripción
CaptureConfigService	Servicio para gestionar configuraciones de captura. Permite agregar, actualizar, eliminar y recuperar configuraciones, las cuales se almacenan por proyecto.
CaptureService	Servicio singleton que gestiona el ciclo de vida completo de la captura: inicialización, pausa, reanudación y detención de la sesión. Controla la ejecución de plugins y coordina el almacenamiento de datos.
CapturePlugin	Clase abstracta base que define la interfaz para todos los plugins de captura. Establece métodos obligatorios como <code>prepare</code> , <code>start</code> , <code>pause</code> , <code>resume</code> , <code>stop</code> y <code>save</code> .
CapturePluginCallbacks	Módulo que define callbacks para la ejecución del ciclo de vida de plugins. Estos son utilizados entre procesos y permiten modularizar la lógica asociada a cada estado del plugin.
CaptureBufferManager	Administra los buffers de datos provenientes de múltiples plugins. Se encarga de recibir, almacenar temporalmente y enviar los datos capturados en intervalos definidos.
CaptureData	Representa una unidad de dato capturado, compuesta por un timestamp y un contenido serializable.
PluginData	Estructura que agrega a <code>CaptureData</code> información del plugin de origen, permitiendo su trazabilidad.
CaptureConfigData	Representa la información de configuración activa utilizada en una sesión de captura.
CaptureConfigDetails	Estructura interna que detalla cada configuración aplicada durante una sesión.
CaptureConfigLoaded	Clase interna que representa la configuración de captura.
SessionData	Modelo que contiene los metadatos de una sesión: tiempo de inicio, fin, pausas, configuraciones utilizadas y otros datos relevantes.
SessionRes	Respuesta serializada con los detalles principales de una sesión de captura.
CaptureStartRequest	Modelo utilizado para iniciar una sesión de captura desde la interfaz.
CaptureStatusResponse	Modelo de respuesta que informa el estado de la sesión de captura.
CaptureConfigRes	Modelo de respuesta que encapsula la configuración de captura.
CaptureConfigPostReq	Modelo de solicitud utilizado para registrar una nueva configuración de captura en un proyecto.

Clase	Descripción
CaptureConfigPutReq	Modelo de solicitud usado para actualizar una configuración de captura existente.
SessionPost	Modelo de solicitud que permite registrar una nueva sesión de captura, incluyendo la configuración de captura a utilizar.
SessionPut	Modelo de solicitud que permite actualizar una sesión de captura existente, incluyendo cambios en su configuración asociada.

Fuente: Elaboración propia, 2025.

Aspectos de implementación

El proceso de captura es gestionado por la clase `CaptureService`, encargado de coordinar la ejecución, el almacenamiento y el control de estado de las sesiones de captura. Dentro de esta arquitectura, la clase abstracta `CapturePlugin` define la interfaz obligatoria que deben implementar todos los plugins de captura para integrarse adecuadamente con el ciclo de vida de las sesiones. Esta clase extiende, a su vez, de `Plugin`, heredando los métodos de carga, configuración y descarga definidos en el núcleo del sistema.

Cada plugin debe implementar los métodos fundamentales que definen este ciclo de vida. En primer lugar, `prepare(path, file_name)` es invocado para establecer el entorno de grabación, como las rutas y nombres de archivo que se utilizarán para almacenar los datos. Luego, `start(start_ts, get_timestamp, on_data)` inicia el proceso de recolección, siendo responsable de capturar activamente los datos y reportarlos usando la función `on_data`. Esta función (`on_data`) debe ser invocada por el plugin cada vez que se genera un nuevo dato, empaquetándolo en una instancia de `CaptureData` que contiene un timestamp y el dato en sí. Para garantizar la coherencia temporal entre múltiples fuentes de datos, se proporciona al plugin el parámetro `get_timestamp`, una función que retorna el tiempo actual del sistema con precisión. Esta función debe ser utilizada por el plugin tanto al generar `CaptureData` como en situaciones donde se requiera sincronización con dispositivos externos, asegurando así que todos los plugins trabajen bajo una misma referencia temporal unificada.

Los métodos `pause(ts)` y `resume(ts)` permiten suspender y reanudar temporalmente la captura, conservando el contexto de ejecución. Finalmente, `stop(ts)` detiene por completo la operación del plugin, permitiendo realizar tareas de limpieza o finalización de recursos. Para el almacenamiento de los datos capturados, el método `save(data, end_of_data=False)` es llamado periódicamente por el sistema. Este método recibe una lista de instancias de

CaptureData agrupadas temporalmente, y es invocado repetidamente durante la captura, además de una última vez con `end_of_data=True` para garantizar que todos los datos fueron persistidos. Por último, `get_file_extension()` retorna la extensión del archivo que será utilizado para guardar los datos recolectados.

Un aspecto técnico relevante es la estructura de los datos reportados por los plugins. Cada elemento capturado se transmite como una instancia de `CaptureData`, que agrupa el contenido con su respectivo *timestamp*. El contenido debe cumplir con el tipo `PickleableType`, el cual representa estructuras de datos que pueden ser serializadas utilizando el módulo `pickle` de Python. Esta restricción es importante, ya que garantiza que los datos pueden ser enviados correctamente entre procesos, permitiendo su transporte y posterior manipulación sin pérdida de información o estructura. Un tipo `pickleable` en Python incluye tipos como enteros, cadenas, listas, diccionarios, tuplas y otros objetos anidados que pueden ser descompuestos y reconstruidos automáticamente por el sistema de serialización (GeeksforGeeks, 2024).

El componente encargado de gestionar y coordinar la captura a nivel global es `CaptureService`. Esta clase se implementa bajo el patrón *singleton*, asegurando que exista una única instancia responsable de controlar los estados de captura y distribuir los recursos necesarios. Internamente, `CaptureService` delega la administración de buffers a la clase `CaptureBufferManager`, que se encarga de recibir los datos generados por los distintos plugins, almacenarlos temporalmente en memoria y entregarlos periódicamente a los métodos `save()` de cada plugin correspondiente para su persistencia.

El sistema de captura también incorpora el módulo `CapturePluginCallbacks`, que contiene las funciones necesarias para gestionar cada fase del ciclo de vida del plugin (preparación, inicio, pausa, reanudación, detención y guardado). Estas funciones no están definidas como una clase propiamente tal, ya que deben ser serializables y compartidas entre procesos mediante referencias, permitiendo que el núcleo controle de forma segura la ejecución remota de los plugins desde procesos hijos.

La clase `SessionData` modela toda la información relevante asociada a una sesión de captura. Esta incluye los *timestamps* de inicio y término, los períodos de pausa, las configuraciones utilizadas y los tiempos en los que se activó cada fuente de captura. Esta información se va actualizando dinámicamente durante la ejecución de la sesión y es persistida una vez que esta finaliza, sirviendo como un registro completo de metadatos para su posterior visualización o análisis.

Finalmente, la interacción con este módulo por parte del frontend consiste en el envío de solicitudes REST hacia el backend, ejecutando las rutas definidas que derivan en los servicios descritos en el diagrama. Esto incluye operaciones como crear sesiones, iniciar o detener capturas, o recuperar información registrada. La lógica visual e interacción con el usuario, así

como el flujo detallado de ejecución, serán abordados en la sección correspondiente a la Vista de Procesos (sección 4.4).

4.3.4 Módulo de Visualización

El módulo de visualización se encarga de gestionar las configuraciones y vistas necesarias para la reproducción de sesiones capturadas, permitiendo así la exploración y análisis de los datos capturados. En el lado del backend, se centraliza en la definición y manejo de configuraciones de visualización a nivel de proyecto. Por su parte, en el frontend se enfoca en proveer a los plugins de reproducción de sesiones (playback plugins) con toda la información contextual y de control necesaria para renderizar correctamente los datos.

En la Figura 4.7 se presenta el diagrama de clases correspondiente al backend, mientras que la Figura 4.8 expone la lógica interna del sistema de visualización en la interfaz gráfica. En ambos diagramas se distinguen las clases propias del módulo y aquellas reutilizadas desde otros módulos, como el núcleo o el módulo de organización. Para una vista detallada del diseño, se puede consultar el Apéndice B, en las Figuras B.5 y B.6.

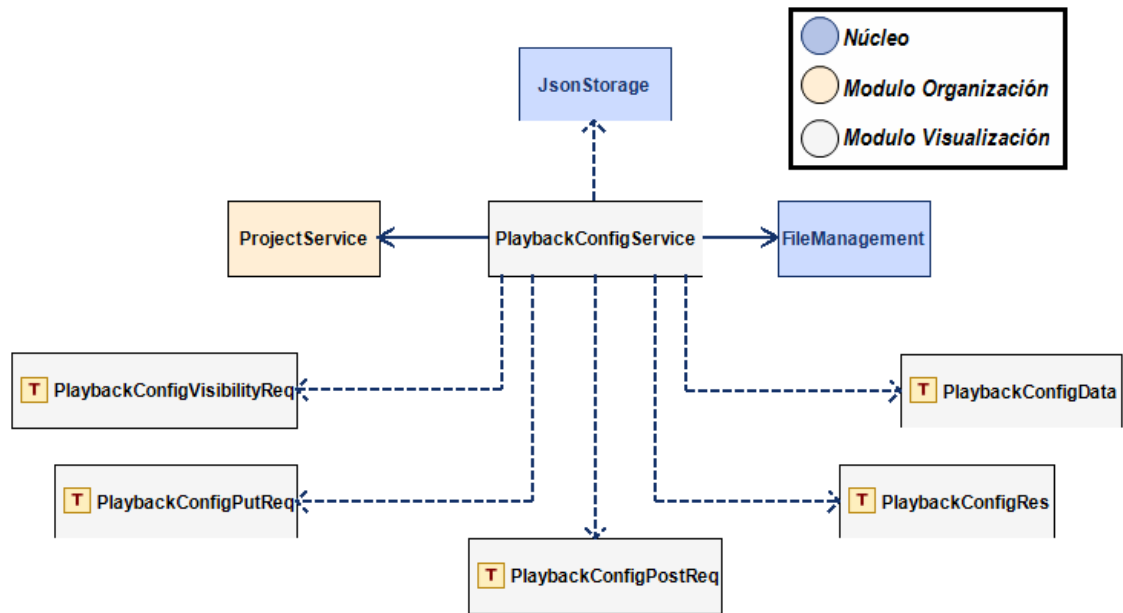


Figura 4.7: Diagrama de clases del módulo de Visualización (backend).

Fuente: Elaboración propia, 2025.

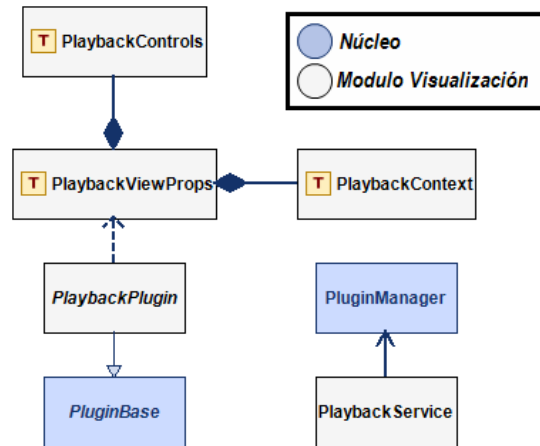


Figura 4.8: Diagrama de clases del módulo de Visualización (frontend).

Fuente: Elaboración propia, 2025.

Cabe mencionar que la lógica asociada al componente de reproducción de sesiones no se muestra explícitamente en estos diagramas, dado que no se implementa mediante clases, sino a través de una combinación de componentes de React y eventos que interactúan con el proceso principal y el preload. Esta lógica se explicará en profundidad en la sección 4.4 correspondiente a la Vista de Procesos.

Finalmente, en la Tabla 4.9 se resumen las clases que conforman el backend de este módulo, mientras que la Tabla 4.10 presenta las clases clave utilizadas en el frontend.

Tabla 4.9: Descripción de clases del módulo de visualización (backend).

Clase	Descripción
PlaybackConfigService	Servicio encargado de gestionar configuraciones de visualización (playback) a nivel de proyecto. Permite agregar, actualizar, eliminar y recuperar configuraciones asociadas a la reproducción de sesiones.
PlaybackConfigPostReq	Modelo de datos utilizado al registrar una nueva configuración de visualización.
PlaybackConfigPutReq	Modelo utilizado para actualizar una configuración de visualización existente.
PlaybackConfigVisibilityReq	Modelo que permite modificar el estado de visibilidad de una configuración.
PlaybackConfigRes	Modelo de respuesta que encapsula los datos de una configuración de visualización al ser solicitada.

Clase	Descripción
PlaybackConfigData	Modelo de datos interno que representa la estructura completa de una configuración de visualización.

Fuente: Elaboración propia, 2025.

Tabla 4.10: Descripción de clases del módulo de visualización (frontend).

Clase	Descripción
PlaybackContext	Contiene la información contextual necesaria para visualizar una sesión.
PlaybackControls	Interfaz que expone métodos de suscripción a eventos del reproductor: reproducción, pausa, cambio de tiempo (seek) y sincronización.
PlaybackViewProps	Objeto que agrupa los datos entregados a un plugin de visualización: controles de reproducción, contexto de sesión y configuraciones.
PlaybackPlugin	Clase abstracta base que deben extender los plugins de visualización. Define métodos clave como getView, getPreview y validExtensions.
PlaybackService	Se encarga de gestionar los plugins de reproducción, permitiendo obtener sus metadatos, instancias y extensiones válidas, así como filtrar configuraciones de captura compatibles con cada plugin.

Fuente: Elaboración propia, 2025.

Aspectos de implementación

En el backend, el componente central es la clase `PlaybackConfigService`, encargada de gestionar las configuraciones de visualización utilizadas específicamente durante la reproducción de sesiones. Cada configuración puede asociarse a una configuración de captura mediante el campo `capture_config_id`, lo que permite vincular automáticamente los datos capturados con los plugins correspondientes al momento de visualizar una sesión. Esta relación facilita una experiencia de reproducción coherente y automatizada, sin necesidad de reasignar manualmente qué plugin debe encargarse de cada archivo generado.

Adicionalmente, `PlaybackConfigService` permite almacenar el layout visual generado en el frontend, correspondiente a la distribución de los distintos plugins de reproducción en pantalla. Este layout es definido por el usuario a través de la interfaz gráfica, utilizando la biblioteca `dockview`, seleccionada por sus capacidades clave para esta tarea: soporte para cambiar

dinámicamente el tamaño de los paneles, reordenarlos, abrirlos en nuevas ventanas, y lo más importante, persistir esta estructura entre sesiones.

En el frontend, los plugins de visualización extienden la clase abstracta `PlaybackPlugin`, la cual hereda a su vez de `PluginBase`. Esta clase define la interfaz obligatoria que deben implementar todos los plugins orientados a representar datos capturados de manera visual. Su propósito principal es renderizar los datos de forma sincronizada y coherente, respondiendo apropiadamente a los eventos del sistema de reproducción.

La clase `PlaybackPlugin` requiere la implementación de tres métodos fundamentales para asegurar su integración con el reproductor general. El primero de ellos es `getView(props)`, el cual recibe como parámetro un objeto de tipo `PlaybackViewProps` y debe retornar un componente de React encargado de renderizar la visualización principal durante la reproducción de una sesión. Este componente debe utilizar los eventos de control provistos para mantenerse sincronizado con el estado del reproductor.

El segundo método, `getPreview()`, permite al plugin definir una vista previa que será utilizada en la pantalla principal. Esta vista puede ser una representación estática, un gráfico resumido o cualquier elemento visual que anticipe el tipo de contenido que será mostrado durante la reproducción.

Finalmente, el método `validExtensions()` permite indicar las extensiones de archivo compatibles con el plugin. Esto es clave para que el sistema pueda asociar correctamente los archivos generados durante la captura con los plugins capaces de visualizarlos, evitando incompatibilidades y mejorando la automatización del proceso de selección.

El objeto `PlaybackViewProps` que se entrega a los plugins contiene tres elementos esenciales. Por un lado, el objeto `PlaybackControls` proporciona una interfaz para suscribirse a los eventos del reproductor: `onPlay`, `onPause`, `onSeek` y `onSync`. Cada uno de estos eventos permite al plugin reaccionar ante acciones del usuario o del sistema, ya sea iniciando animaciones, pausando la visualización, sincronizando su estado interno o moviéndose a un instante específico de la línea de tiempo. Cada función de suscripción retorna un mecanismo de desuscripción que asegura una gestión eficiente de los recursos al desmontar el componente.

En segundo lugar, el objeto `PlaybackContext` entrega toda la información contextual de la sesión. Incluye la ruta al archivo que contiene los datos, el *timestamp* global en el que comenzó la captura (`captureStartTimestamp`), el *timestamp* correspondiente al primer dato del archivo (`fileCaptureStartTimestamp`), y una lista de intervalos de pausa. Esta información permite que el plugin interprete correctamente los datos y los sincronice con la línea de tiempo general, evitando desfases o errores de reproducción.

Por último, el atributo `settings` permite al plugin acceder a los parámetros de configuración definidos previamente por el usuario en la etapa de creación de la configuración de visualización.

Estos valores pueden incluir opciones específicas de visualización, filtros o preferencias que modifican el comportamiento del plugin.

Este enfoque modular y desacoplado asegura que los plugins de visualización puedan integrarse de manera sencilla en la aplicación. Cada uno actúa de manera autónoma pero coordinada, ofreciendo una experiencia de reproducción personalizada y extensible.

4.4 VISTA DE PROCESOS

Esta vista describe los principales flujos de interacción que se producen durante el uso de la aplicación, desde la configuración inicial de un proyecto hasta la ejecución de sesiones de captura y visualización. A través de diagramas de secuencia, se detallan las acciones llevadas a cabo por los usuarios y su correspondencia con los distintos módulos y servicios que conforman MO. Esta perspectiva permite comprender el comportamiento dinámico del sistema, facilitando la identificación de dependencias, sincronización entre componentes y puntos críticos del flujo de trabajo.

4.4.1 Configuración inicial de un proyecto

El proceso de configuración inicial corresponde al conjunto de pasos necesarios para preparar un nuevo proyecto dentro de la aplicación, permitiendo que este quede completamente habilitado para la ejecución de protocolos, la captura de datos y su posterior visualización. Este flujo constituye la base para cualquier actividad posterior, y se apoya principalmente en los servicios del módulo de organización, aunque también integra funcionalidades de los módulos de captura y visualización.

Como se puede observar en la Figura 4.9, el proceso comienza con la creación del proyecto, seguido por la incorporación de al menos un participante asociado. Posteriormente, el usuario puede agregar cuantas configuraciones desee tanto para plugins de captura como de visualización, así como definir el protocolo a ser ejecutado. Si bien en el diagrama no se explicita la repetición de cada uno de estos pasos, cabe destacar que cada elemento (participantes, configuraciones o protocolos) puede añadirse múltiples veces según las necesidades del estudio en cuestión.

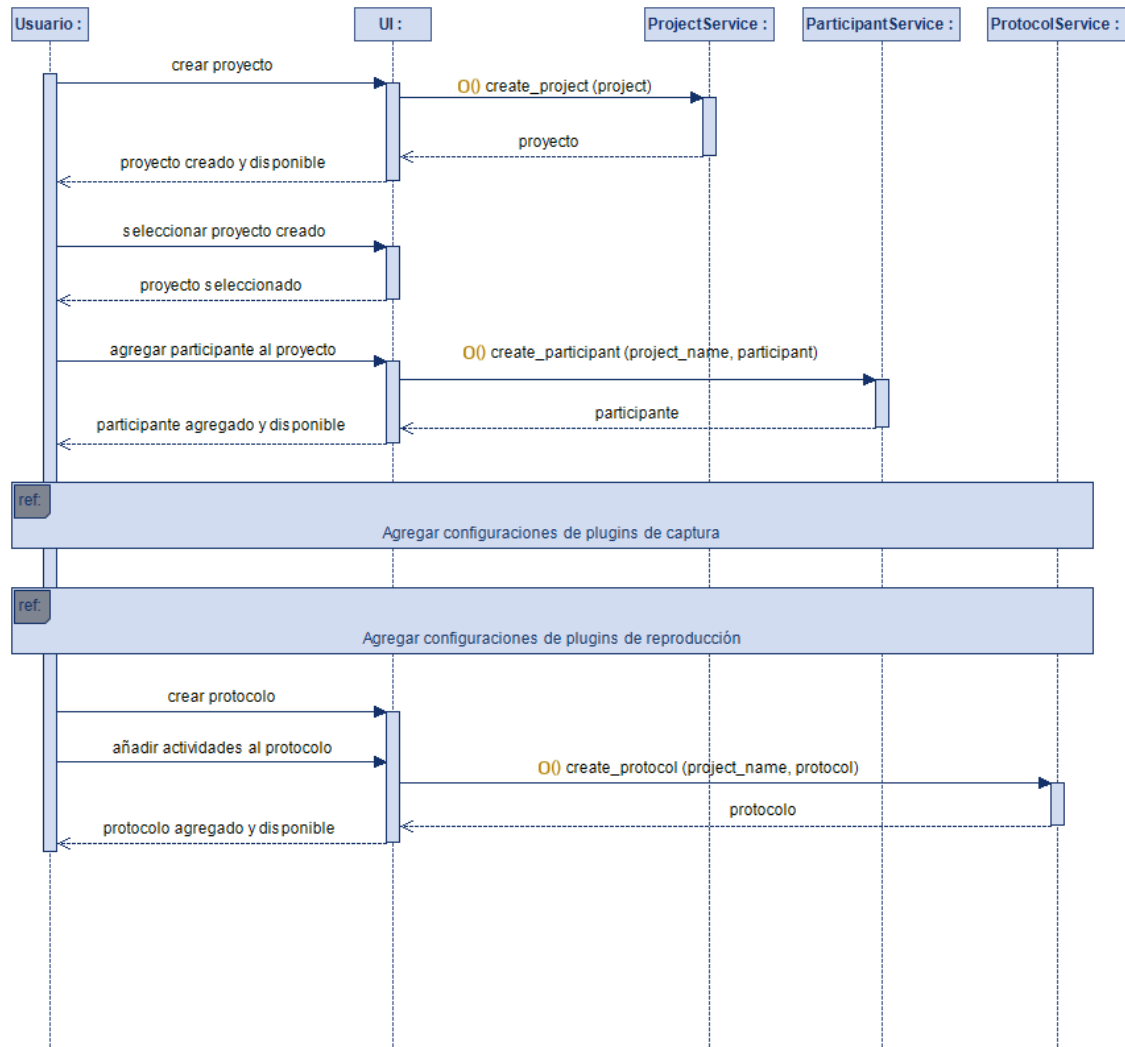


Figura 4.9: Diagrama de secuencia - Configuración inicial de un proyecto.

Fuente: Elaboración propia, 2025.

El primer paso es la creación del proyecto, a través del servicio `ProjectService`, el cual se encarga de generar la estructura de almacenamiento correspondiente en el sistema de archivos. Una vez creado, se selecciona el proyecto y se procede a la creación del participante mediante el `ParticipantService`, este participante quedara dentro de la carpeta asociada al proyecto creado anteriormente, donde luego se almacenarán las sesiones de captura asociadas a este participante.

Tras esto, se incluyen dos referencias a procesos complementarios que están desglosados en los diagramas de las Figuras 4.10 y 4.11, correspondientes a la configuración de plugins de captura y reproducción respectivamente. En cada uno de estos procesos, el usuario selecciona un plugin disponible, accede a sus propiedades de configuración, y finalmente guarda dicha configuración para que quede disponible en el proyecto.

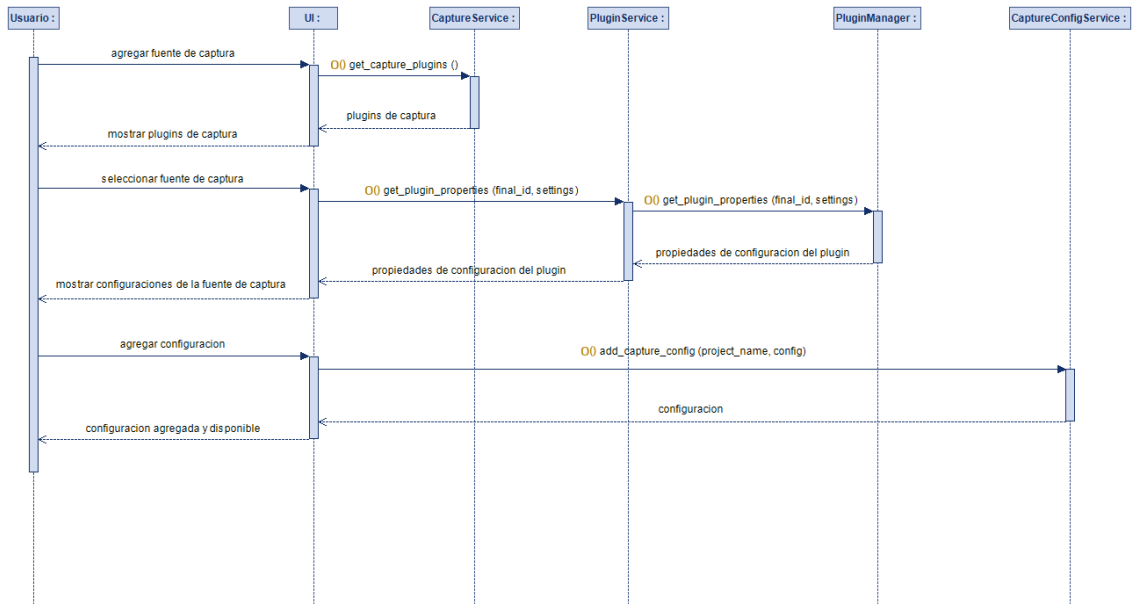


Figura 4.10: Diagrama de secuencia - Agregar configuraciones de plugins de captura.
Fuente: Elaboración propia, 2025.

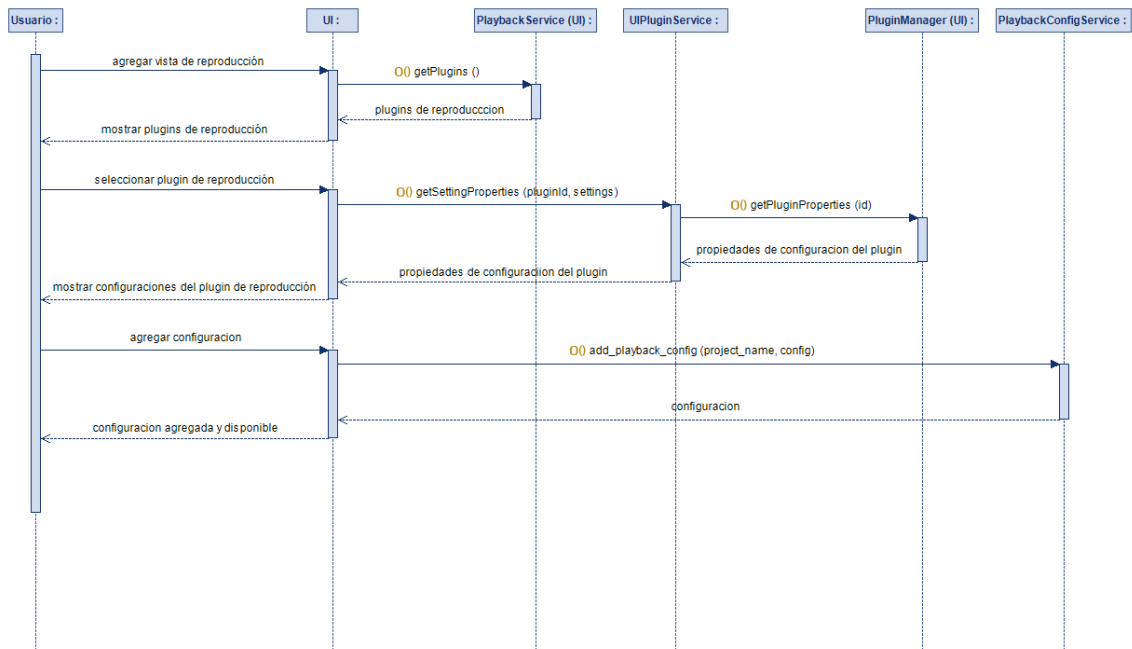


Figura 4.11: Diagrama de secuencia - Agregar configuraciones de plugins de reproducción.
Fuente: Elaboración propia, 2025.

Finalmente, el flujo concluye con la creación del protocolo mediante el servicio `ProtocolService`, lo que permite definir las actividades que estructuran una sesión de estudio. Este protocolo podrá luego ser ejecutado durante una sesión de captura o reproducción, actuando como guía para el

estudio en cuestión.

4.4.2 Ejecución de un protocolo de investigación

El proceso de ejecución de un protocolo de investigación corresponde al flujo que permite guiar al usuario a través de una serie de actividades previamente definidas dentro de un protocolo asociado a un proyecto determinado. Esta funcionalidad es esencial para estudios que requieren una secuencia estructurada de pasos, ya sea para presentar instrucciones, mostrar materiales, ejecutar tareas, iniciar procesos, entre otros. La Figura 4.12 muestra el diagrama de secuencia que representa este proceso. Este proceso se inicia cuando el usuario selecciona un proyecto y, posteriormente, uno de los protocolos disponibles asociados a este. Al activar la opción de ejecución, se establece una conexión mediante WebSocket con el backend, permitiendo mantener una comunicación bidireccional y en tiempo real durante todo el flujo de ejecución.

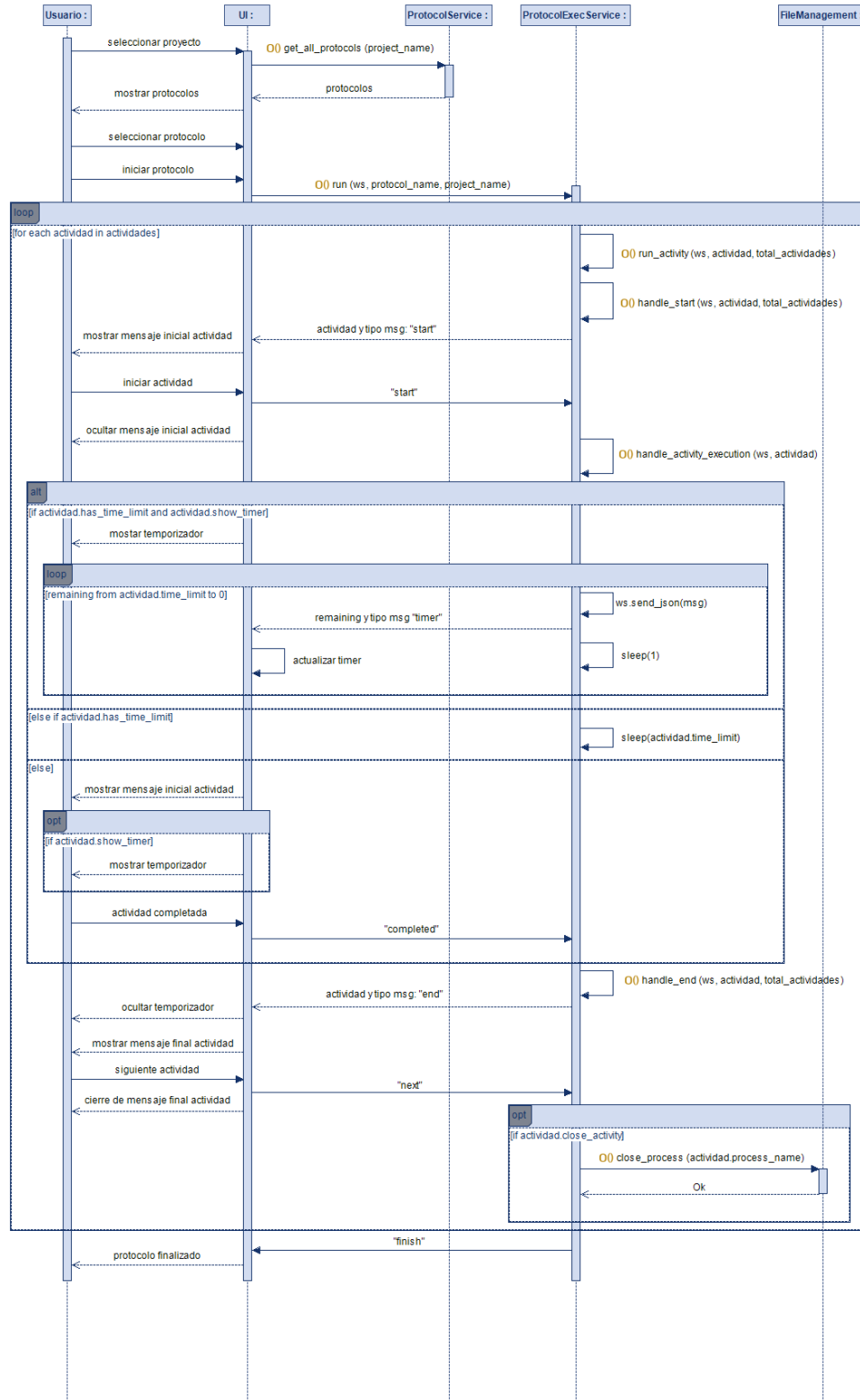


Figura 4.12: Diagrama de secuencia - Ejecución de un protocolo de investigación.

Fuente: Elaboración propia, 2025.

La conexión entre el frontend y el backend se realiza mediante un WebSocket proporcionado por FastAPI, el cual permite establecer un canal persistente de comunicación una vez aceptada la conexión. Este canal es utilizado para enviar y recibir mensajes en formato JSON, facilitando una interacción continua durante la ejecución del protocolo sin necesidad de múltiples solicitudes HTTP. Al establecer la conexión, se inicia el flujo controlado por el backend, gestionado por la clase `ProtocolExecService`.

Desde el backend, `ProtocolExecService` se encarga de cargar el protocolo seleccionado e iterar sobre su lista de actividades ordenadas, gestionando en cada paso los mensajes que se envían al cliente para coordinar la ejecución de cada actividad.

El ciclo para cada actividad comienza con el envío de un mensaje de tipo `start`, el cual contiene información como el nombre de la actividad, un mensaje introductorio, indicadores de visibilidad del temporizador y si esta cuenta con una duración límite. La interfaz gráfica, al recibir este mensaje, muestra un cuadro con el mensaje correspondiente, solicitando la confirmación del usuario para comenzar. La respuesta esperada por parte del cliente es el texto `"start"`, lo que permite continuar con la ejecución.

Si la actividad posee un archivo asociado, este se abre automáticamente en el sistema. Posteriormente, si la actividad tiene una duración límite y el temporizador debe mostrarse, el backend envía mensajes periódicos de tipo `timer`, conteniendo el número de segundos restantes, que son utilizados por la interfaz para actualizar visualmente un temporizador flotante. En caso de no mostrar el temporizador pero sí tener un límite de tiempo, el backend simplemente espera el tiempo correspondiente. Finalmente, si no existe límite de tiempo, la interfaz permanece a la espera de una interacción del usuario para marcar la actividad como completada mediante el mensaje `"completed"`.

Una vez finalizada la ejecución de la actividad, el backend envía un mensaje de tipo `end`, junto con un mensaje de cierre asociado a dicha actividad. La interfaz lo interpreta, mostrando nuevamente un cuadro con el mensaje final, esperando la confirmación del usuario mediante el mensaje `"next"` para avanzar a la siguiente actividad. Si la actividad tiene definido que debe cerrarse un proceso externo, el backend ejecuta esta acción mediante el servicio `FileManagement`, cerrando la aplicación o archivo correspondiente.

Este flujo se repite por cada una de las actividades del protocolo, manteniendo en todo momento una comunicación sincronizada entre el backend y la interfaz de usuario. Al completarse la última actividad, el servidor envía un mensaje de tipo `"finish"`, tras lo cual se cierra la conexión WebSocket y se da por terminada la ejecución del protocolo.

4.4.3 Proceso de captura de datos

El proceso de captura de datos es el flujo central del módulo de captura y uno de los más relevantes de la aplicación, dicho proceso permite recolectar, almacenar y gestionar los datos provenientes de las distintas fuentes de captura configuradas. Este proceso es orquestado principalmente por la clase `CaptureService`, que coordina tanto la inicialización de los plugins de captura como el almacenamiento progresivo de los datos, apoyándose en estructuras multiproceso y multihilo para asegurar el rendimiento y la estabilidad del sistema.

En la Figura 4.13 se puede observar el flujo general del proceso, que está compuesto por distintas etapas que se describen a continuación. Con el fin de mejorar la legibilidad, las fases internas más complejas han sido separadas en diagramas complementarios: preparación de la captura (Figura 4.14), inicio de los plugins (Figura 4.15), procesos de buffering (Figura 4.16), y operaciones de pausa, reanudación y detención (Figuras 4.17, 4.18 y 4.19).

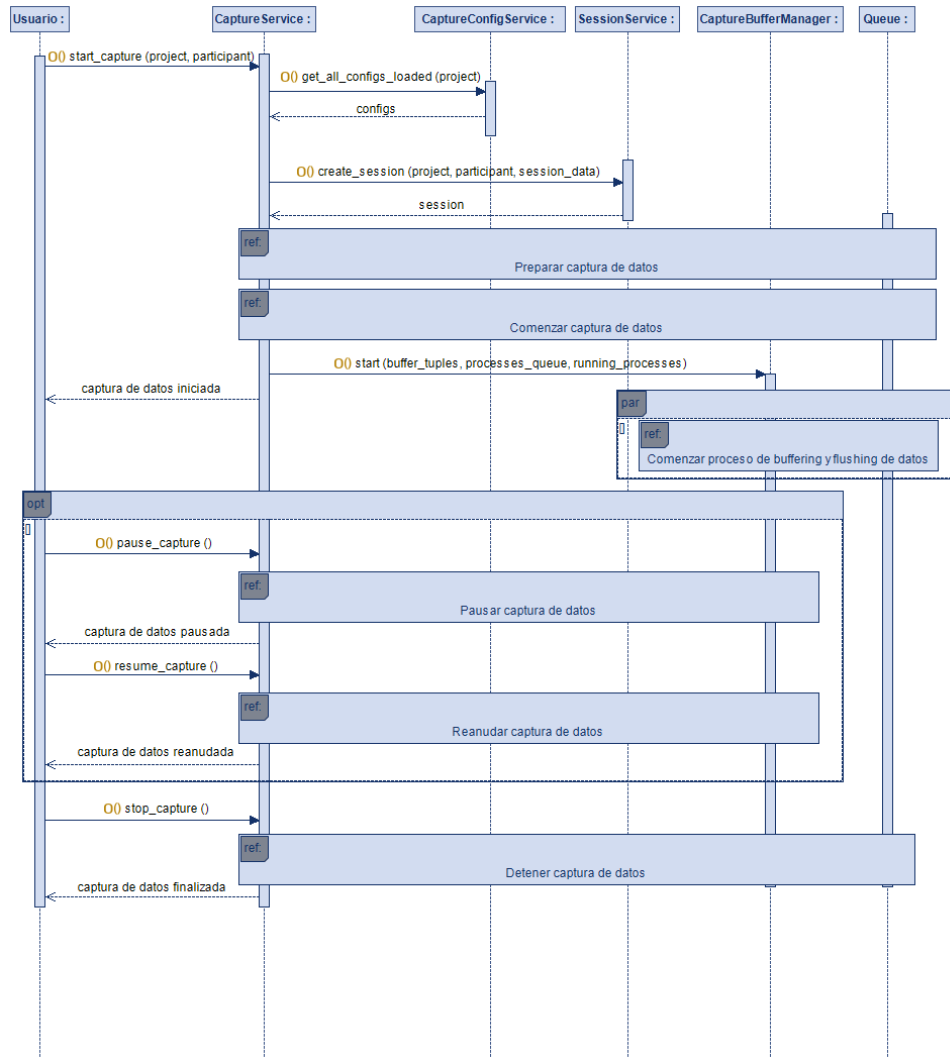


Figura 4.13: Diagrama de secuencia - Captura de datos.

Fuente: Elaboración propia, 2025.

Preparación del proceso de captura

El flujo inicia cuando el usuario solicita comenzar una captura para un proyecto y un participante determinados. `CaptureService` recupera todas las configuraciones de captura asociadas al proyecto y crea una nueva sesión, persistiendo su metadata inicial mediante `SessionService`. Esta sesión quedará asociada al participante y contendrá información como el *timestamp* de inicio, la fecha, y los plugins utilizados.

A partir de las configuraciones obtenidas, se procede a inicializar los procesos necesarios para la captura, lo cual se detalla en la Figura 4.14. Para cada tipo de plugin utilizado en las

configuraciones, se verifica si ya existe un proceso activo (`PluginWorkerProcess`); en caso contrario, se instancia uno nuevo. Durante su creación, este proceso se encarga de cargar el plugin correspondiente, junto con sus dependencias, quedando completamente preparado para gestionar las instancias asociadas a dicho plugin.

Una vez levantado (o reutilizado) el proceso del plugin, se agregan las instancias necesarias según las configuraciones definidas en el proyecto. Cada instancia representa una configuración concreta del plugin de captura y es añadida al proceso mediante la llamada a `add_plugin_instance()`, que la construye y la configura con los parámetros definidos. Finalmente, para cada instancia creada, se ejecuta el callback de `prepare()`, entregando como argumentos la ruta de la sesión y el nombre del archivo donde se almacenarán los datos capturados. Cabe destacar que, para que el proceso hijo ejecute correctamente el método `prepare()` en la instancia específica del plugin, se utiliza una función intermedia que encapsula el callback y sus argumentos. Esta función es enviada desde el proceso principal y resuelta internamente en el proceso hijo, siguiendo el mecanismo de comunicación ya descrito en la sección 4.3.1, correspondiente al núcleo del sistema.

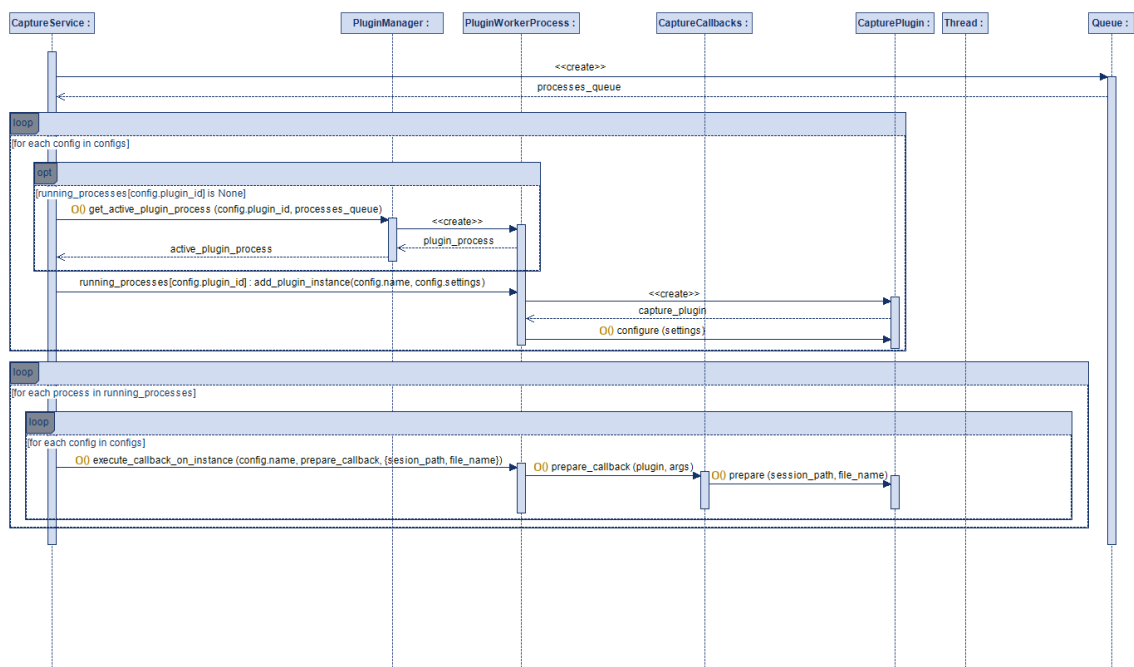


Figura 4.14: Diagrama de secuencia - Preparar captura de datos.

Fuente: Elaboración propia, 2025.

Ejecución de la captura

Una vez finalizada la preparación de los plugins, el servicio `CaptureService` registra el *timestamp* de inicio y procede a ejecutar el callback `start()` en cada instancia de plugin de captura. Este flujo se detalla en la Figura 4.15, donde se muestra cómo, por cada configuración activa, se llama a la función `start_callback()` a través del método `execute_callback_on_instance()` del proceso correspondiente.

La función `start_callback()` crea un hilo por instancia de plugin, dentro del cual se ejecuta el método `start()` definido por el plugin. Este método implementado por el plugin de captura debería iniciar un loop interno que se mantiene activo mientras no se invoque explícitamente la función `stop()`, y su propósito es capturar y transmitir continuamente los datos que el plugin genera durante la sesión.

Cada vez que el plugin captura un nuevo dato, se ejecuta la función `on_data()`, la cual es definida dentro de `start_callback()` y actúa como intermediaria entre el plugin y el sistema de buffering. Esta función toma la instancia de `CaptureData` generada por el plugin y encapsula la información junto con el identificador del plugin y de la configuración en un objeto `PluginData`. Una vez construido este objeto, se envía a la cola compartida de comunicación entre procesos (`multiprocessing.Queue`), desde donde será posteriormente procesado por el sistema de buffering. Esto permite que el sistema pueda identificar correctamente a qué proceso e instancia corresponde cada dato recibido.

Es importante recordar que los datos enviados deben ser picklable, es decir, serializables para ser transmitidos entre procesos mediante la cola compartida (`multiprocessing.Queue`).

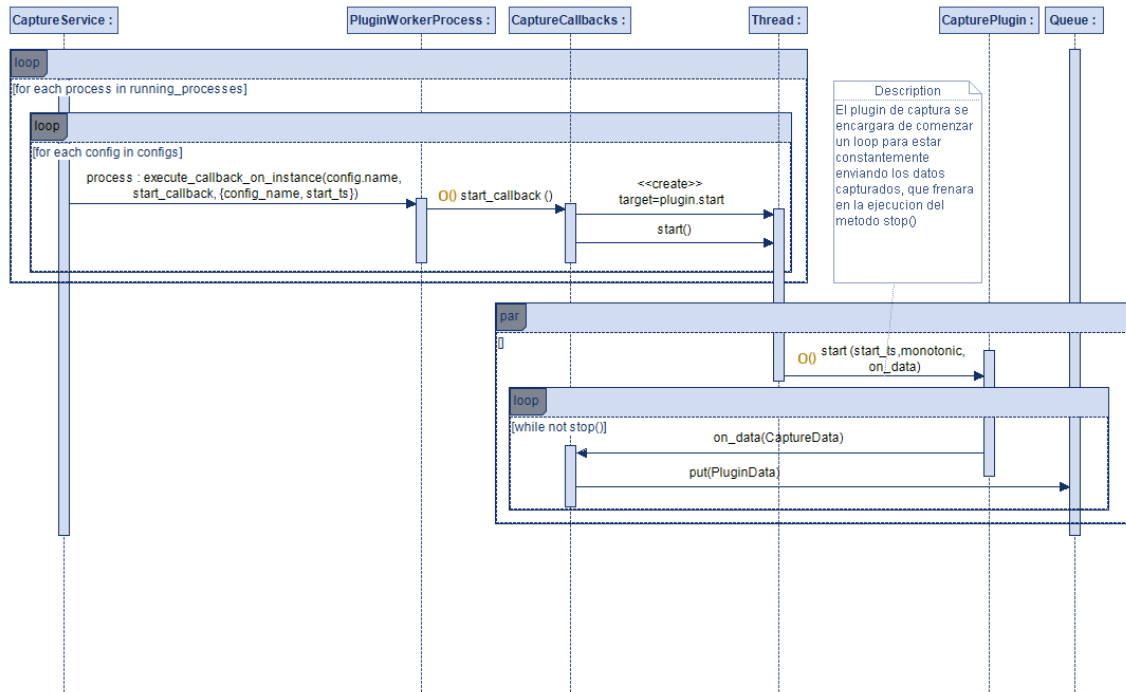


Figura 4.15: Diagrama de secuencia - Comenzar captura de datos.

Fuente: Elaboración propia, 2025.

Buffering y almacenamiento de datos

Los datos capturados por los plugins se transmiten a través de una cola (queue) hacia el componente `CaptureBufferManager`, cuya tarea es gestionar la persistencia de los datos sin bloquear el ciclo de captura. Como se muestra en la Figura 4.16, este componente crea tres hilos concurrentes que permiten mantener la operación fluida del sistema.

El primer hilo es responsable de consumir continuamente los datos desde la cola compartida. Por cada elemento recibido, identifica la configuración a la que pertenece y lo agrega a un *buffer*, una estructura de almacenamiento temporal que mantiene los datos en memoria antes de ser escritos a disco. El uso de buffers permite desacoplar la frecuencia con la que se generan los datos de la velocidad con la que pueden ser almacenados de forma segura, evitando cuellos de botella durante la captura.

El segundo hilo se encarga de realizar el vaciado de estos buffers (proceso conocido como *flush*) a intervalos regulares. Para ello, utiliza el callback `save()`, definido por cada plugin, el cual se invoca sobre las instancias respectivas y permite almacenar los datos acumulados de manera persistente.

Finalmente, el tercer hilo actúa como monitor del uso de recursos, evaluando periódicamente el estado de la memoria RAM y de la memoria swap (memoria virtual utilizada por el sistema operativo cuando la RAM se encuentra saturada). Si detecta un estado de sobrecarga (según límites configurados en el CaptureService), fuerza un vaciado inmediato de los buffers para liberar memoria y evitar interrupciones en la captura.

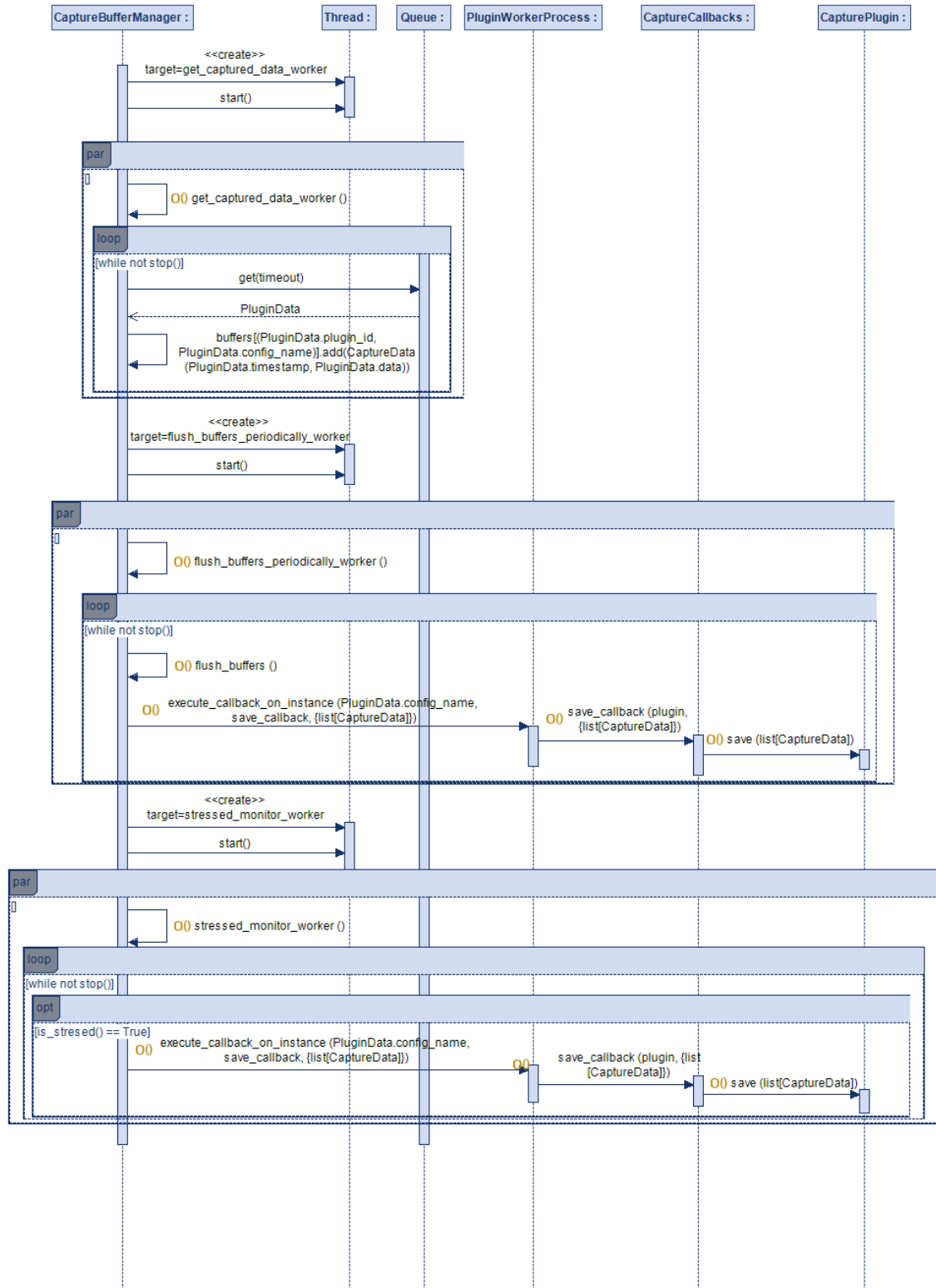


Figura 4.16: Diagrama de secuencia - Comenzar proceso de buffering y almacenamiento de datos.

Fuente: Elaboración propia, 2025.

Pausa y reanudación de la captura

El sistema también permite pausar y reanudar una sesión sin reiniciar el flujo. Estas operaciones son gestionadas por `CaptureService`, que al recibir una orden de pausa registra el *timestamp* y notifica a todos los plugins mediante el callback `pause()`. El componente de buffers registra el intervalo de pausa para evitar guardar datos en ese lapso. Al reanudar, se calcula el tiempo total pausado y se propaga la señal `resume()` a todos los plugins, como se muestra en las Figuras 4.17 y 4.18.

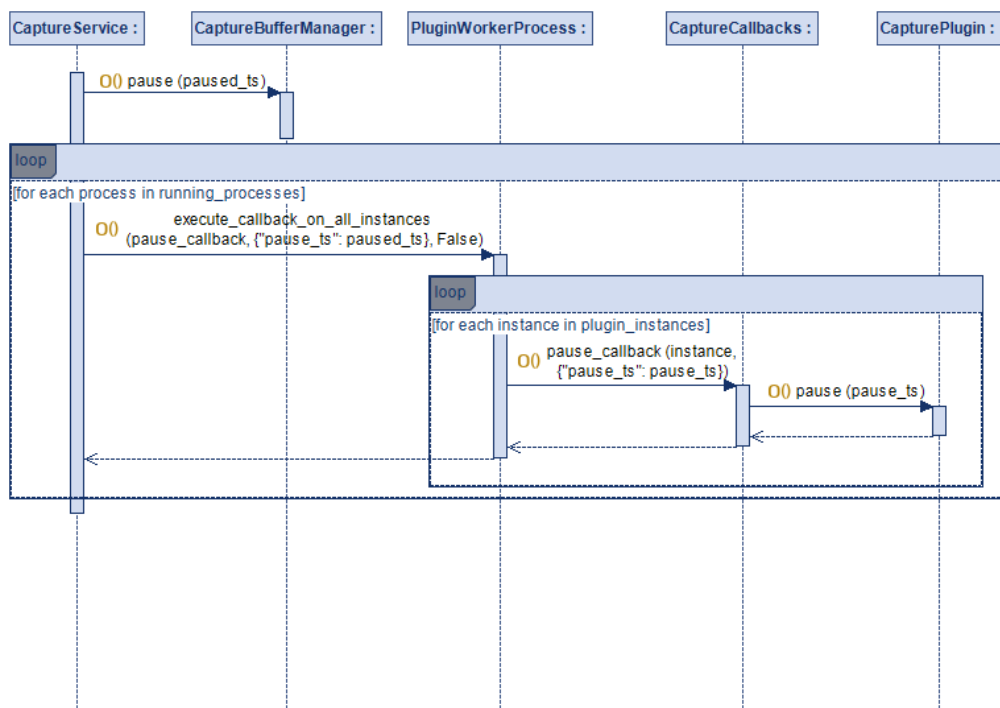


Figura 4.17: Diagrama de secuencia - Pausar captura de datos.

Fuente: Elaboración propia, 2025.

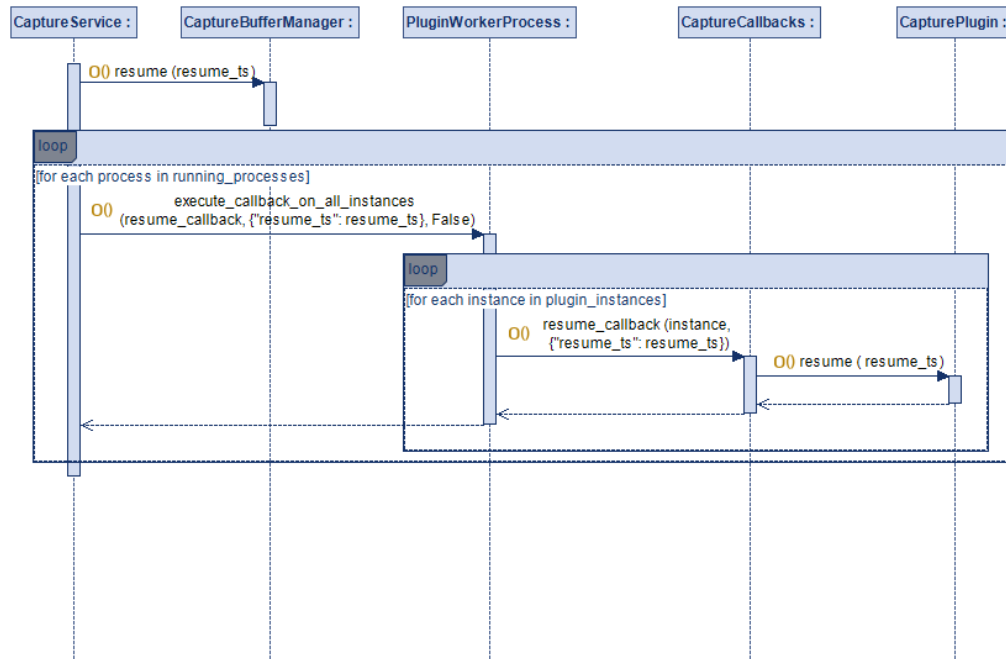


Figura 4.18: Diagrama de secuencia - Reanudar captura de datos.

Fuente: Elaboración propia, 2025.

Finalización de la sesión

Finalmente, al detener la sesión de captura, el servicio `CaptureService` coordina una serie de acciones que aseguran el cierre ordenado de los procesos involucrados y la persistencia completa de los datos. Este flujo se ilustra en la Figura 4.19.

En primer lugar, se ejecuta el callback `stop()` sobre cada instancia de plugin activa, permitiendo que cada uno finalice su ejecución interna y detenga la producción de datos. A continuación, se agregan a la sesión los metadatos faltantes, tales como el *timestamp* de finalización, la duración efectiva de la sesión (considerando los tiempos pausados), la fecha de cierre, y el detalle de los intervalos de pausa ocurridos durante la captura. Estos datos son esenciales tanto para mantener un historial coherente de la sesión como para la posterior sincronización durante la visualización de los datos.

Un paso clave antes de proceder al vaciado final de los datos es el agregado de un nuevo intervalo de pausa al `CaptureBufferManager`, que solo especifica el *timestamp* de finalización como punto de inicio del intervalo. Esto garantiza que los datos que lleguen después de ese instante (producto de desfases naturales entre procesos) no sean almacenados, preservando así la coherencia temporal de la sesión de captura.

Al detener el `CaptureBufferManager`, se interrumpen los hilos activos (lectura, vaciado periódico y monitoreo), tras lo cual se inicia el proceso de cierre. En esta fase, se transfieren todos los datos remanentes en la cola hacia sus buffers correspondientes. Durante esta transferencia, el sistema espera breves intervalos entre lecturas para asegurar que no queden datos rezagados llegando desde los procesos de captura. Solo cuando se ha vaciado completamente la cola, se procede a realizar el vaciado final de todos los buffers.

Este vaciado se efectúa invocando nuevamente el callback `save()` sobre cada plugin, pero esta vez con el flag `end_of_data = True`, lo cual permite a los plugins ejecutar tareas finales como el cierre de archivos, consolidación de registros, o liberación explícita de recursos.

Como último paso, el servicio detiene todos los procesos de captura (uno por tipo de plugin), forzando su finalización si exceden un tiempo de espera definido para garantizar que el cierre del sistema ocurra de forma segura y controlada.

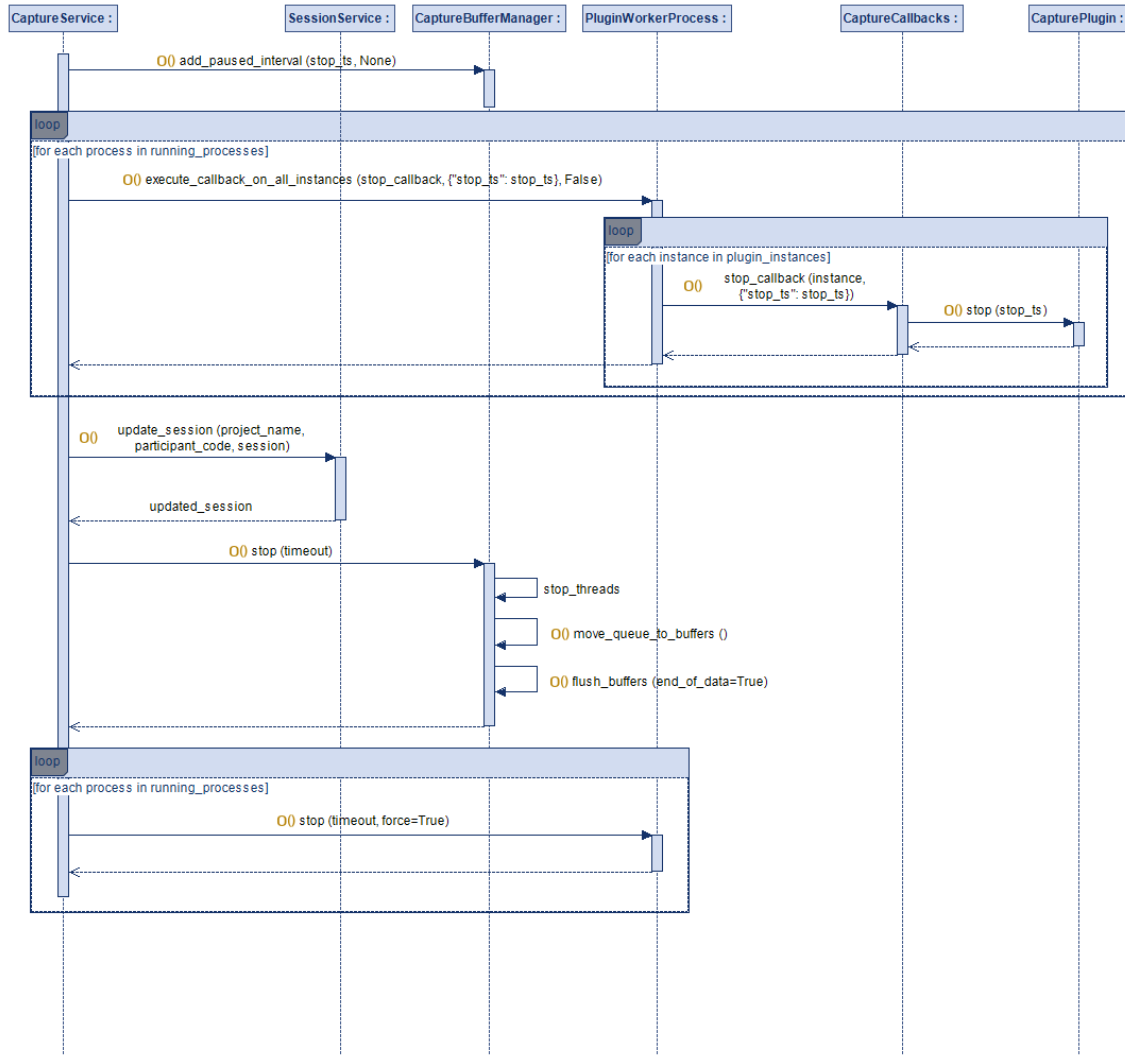


Figura 4.19: Diagrama de secuencia - Detener captura de datos.

Fuente: Elaboración propia, 2025.

4.4.4 Reproducción de una sesión

El proceso de reproducción de una sesión en la aplicación ocurre completamente del lado del cliente, sin intervención activa del backend, salvo para obtener la información de la sesión y el layout a visualizar. La Figura 4.20 ilustra este flujo general de reproducción, el cual inicia cuando el usuario selecciona un proyecto y un participante desde la interfaz gráfica. A partir de esta selección, la página de reproducción (PlaybackPage) consulta las sesiones disponibles mediante el servicio SessionService, utilizando el método `get_all_sessions()` para ese proyecto y participante. En paralelo, también se solicita el layout de reproducción guardado para el proyecto

mediante `get_playback_layout()`.

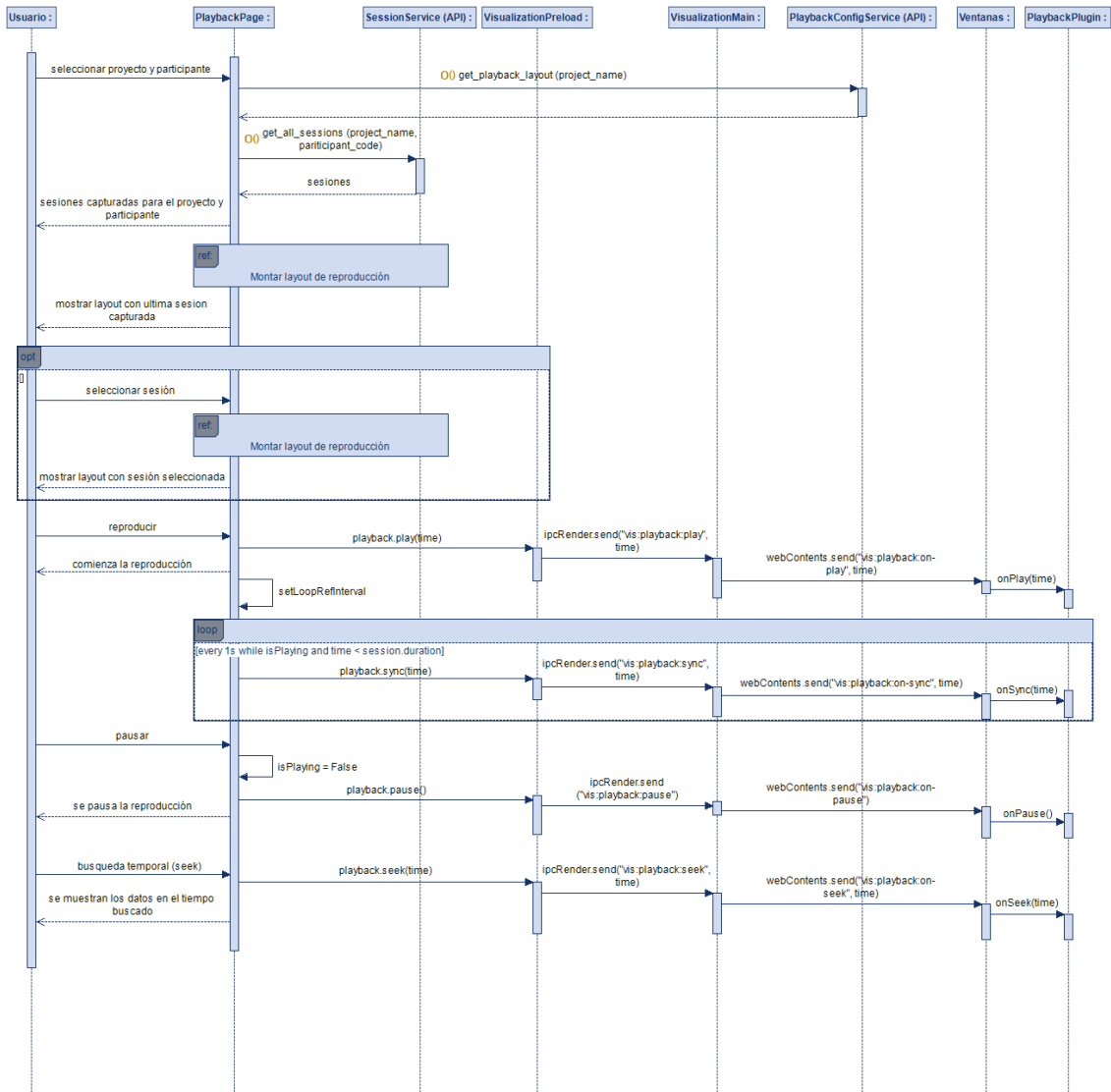


Figura 4.20: Diagrama de secuencia - Reproducción de una sesión.

Fuente: Elaboración propia, 2025.

Una vez recibida esta información, la aplicación monta automáticamente el layout correspondiente a la última sesión capturada, generando los paneles de reproducción configurados. Este procedimiento puede repetirse si el usuario selecciona manualmente otra sesión desde la interfaz, lo cual fuerza una reconstrucción del layout utilizando los datos actualizados de la sesión seleccionada. Esta fase resulta fundamental para entregar a los plugins el contexto necesario, como la ubicación del archivo a visualizar y los *timestamps* relevantes para la sesión.

El montaje del layout se detalla en la Figura 4.21. Esta tarea es ejecutada por el componente `PlaybackDock`, que recibe como entrada las configuraciones de reproducción obtenidas del

layout. Por cada configuración, se invoca la función `getPlaybackPanel()`, la cual inicia el flujo de carga del plugin correspondiente. En este proceso, se obtiene la instancia del plugin mediante `getPluginInstanceById()` desde el `PlaybackService`, se configura con los parámetros definidos por el usuario a través del método `configure()`, y finalmente se genera la vista llamando a `getView()`. Esta función recibe un conjunto de controles (`controls`) para reaccionar a eventos de reproducción como `play`, `pause`, `seek` y `sync`, un contexto (`context`) con información relevante como la ruta del archivo, los *timestamps* de captura y los intervalos de pausa, y los ajustes propios de la configuración. Como resultado, se obtiene un componente React que representa visualmente al plugin (`PlaybackPlugin`) y que es renderizado en pantalla junto con el resto de los paneles.

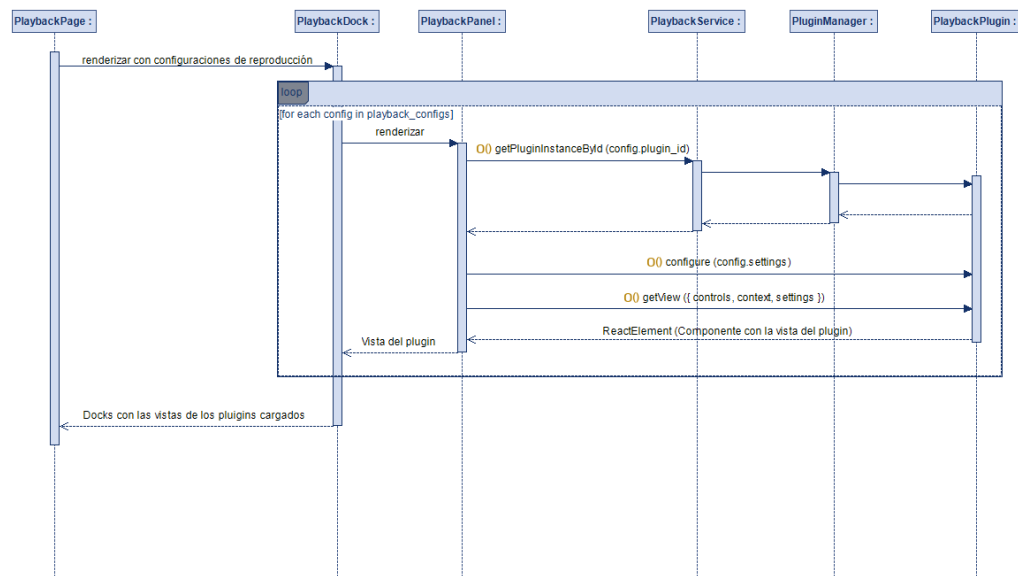


Figura 4.21: Diagrama de secuencia - Montar layout de reproducción.

Fuente: Elaboración propia, 2025.

Una vez montadas las vistas, el usuario puede iniciar la reproducción de la sesión. Este proceso implica la coordinación entre los distintos contextos de ejecución de Electron: `renderer`, `preload` y `main`. Cuando el usuario interactúa con la interfaz, por ejemplo presionando el botón de reproducción, la `PlaybackPage` llama a `window.visualization.playback.play(time)`. Esta función, definida en el contexto `preload` (`VisualizationPreload`), utiliza `ipcRenderer.send()` para enviar el evento `vis:playback:play` al proceso principal. Desde allí, el módulo `VisualizationMain` intercepta el mensaje y lo propaga a todas las ventanas abiertas usando `broadcast()`, que internamente recurre a `webContents.send()` para distribuir el evento. Cada ventana que tenga un plugin de reproducción activo escucha el evento mediante `ipcRenderer.on()` y ejecuta la función correspondiente a `onPlay()`, activando la reproducción en cada uno de los plugins involucrados.

Dicho patrón de comunicación se replica para otros eventos como `pause()`, `seek(time)` y `sync(time)`. Durante la reproducción, se ejecuta un bucle interno cada 10 ms para calcular el avance del tiempo, y adicionalmente, cada 1 segundo se emite un evento `sync()` para mantener sincronizadas todas las vistas. Este bucle se interrumpe automáticamente si se alcanza la duración completa de la sesión o si el usuario la pausa manualmente, garantizando un comportamiento coherente y sincronizado entre todos los componentes visuales.

4.5 ESCENARIOS

Esta sección presenta los principales escenarios de uso del sistema desde la perspectiva del usuario, con el objetivo de ilustrar cómo se integran las funcionalidades clave en situaciones reales de operación. A través de diagramas de casos de uso se representan las interacciones entre el actor principal del sistema y las funcionalidades disponibles.

4.5.1 Configuración inicial de un proyecto

Este escenario describe el conjunto de acciones necesarias para que un investigador pueda preparar el entorno de trabajo en el sistema antes de iniciar procesos de captura de datos. Corresponde a la configuración inicial requerida para registrar los elementos fundamentales de un estudio, incluyendo la estructura organizativa del proyecto, los participantes involucrados y las herramientas empleadas para la recolección y reproducción de la información.

La Figura 4.22 ilustra los casos de uso involucrados en este escenario, representando las interacciones del investigador con el sistema y sus respectivas dependencias.

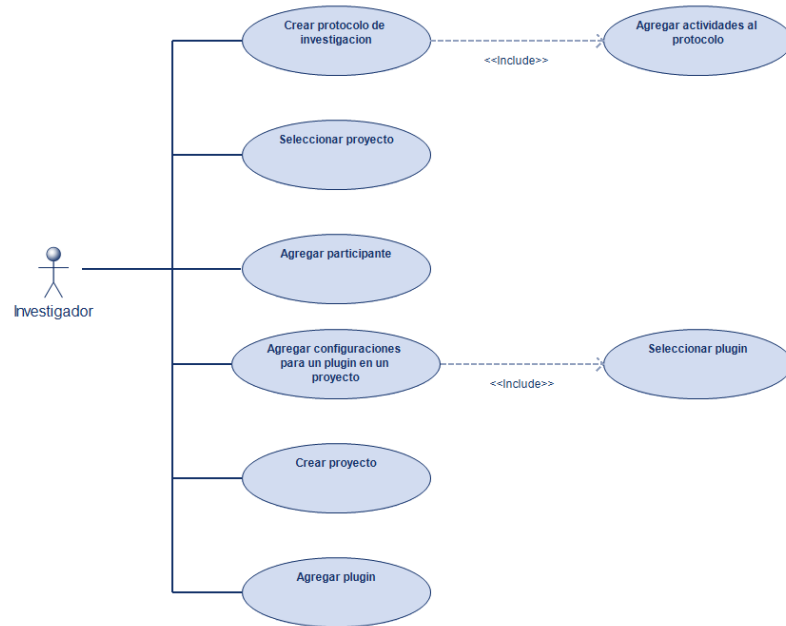


Figura 4.22: Diagrama de casos de uso - Configuración inicial de un proyecto.
Fuente: Elaboración propia, 2025.

La Tabla 4.11 presenta una descripción de los casos de uso involucrados en el escenario de configuración inicial de un proyecto. Cada caso de uso representa una acción clave que puede ejecutar el investigador dentro del sistema, y se indica explícitamente cualquier precondition necesaria para su ejecución.

Tabla 4.11: Casos de uso del escenario de configuración inicial de un proyecto.

Caso de uso	Descripción	Precondición
Crear proyecto	Permite al investigador registrar un nuevo proyecto en el sistema, el cual servirá como contenedor principal para las demás entidades del estudio.	Ninguna
Seleccionar proyecto	Define el contexto de trabajo activo sobre el cual se aplicarán las acciones siguientes.	Haber creado al menos un proyecto
Agregar participante	Registra un nuevo participante dentro del proyecto seleccionado, asociando sus datos con futuras sesiones de captura.	Haber seleccionado un proyecto
Agregar plugin	Permite registrar en el sistema un nuevo plugin de captura o visualización, para que pueda ser configurado y utilizado en proyectos.	Ninguna

Caso de uso	Descripción	Precondición
Agregar configuraciones para un plugin en un proyecto	Permite definir los parámetros específicos de funcionamiento de un plugin en el contexto de un proyecto determinado.	Haber seleccionado un proyecto y haber agregado al menos un plugin
Crear protocolo de investigación	Permite estructurar y registrar una secuencia de actividades a seguir durante una sesión de estudio.	Haber seleccionado un proyecto
Agregar actividades al protocolo	Permite definir las distintas tareas o etapas que componen el protocolo de investigación.	Haber iniciado la creación de un protocolo
Seleccionar plugin	Permite elegir un plugin disponible en el sistema para asociarlo a una configuración dentro del proyecto.	Haber agregado al menos un plugin

Fuente: Elaboración propia, 2025.

4.5.2 Ejecución de un protocolo

Este escenario representa el flujo de trabajo que permite al investigador llevar a cabo un protocolo de investigación previamente definido. El protocolo, compuesto por una secuencia ordenada de actividades, actúa como guía estructurada para dirigir sesiones de estudio dentro del sistema. La ejecución del protocolo se realiza de forma interactiva y puede involucrar tanto acciones manuales por parte del usuario como eventos automáticos gestionados por el sistema.

La Figura 4.23 muestra los casos de uso asociados a este escenario.

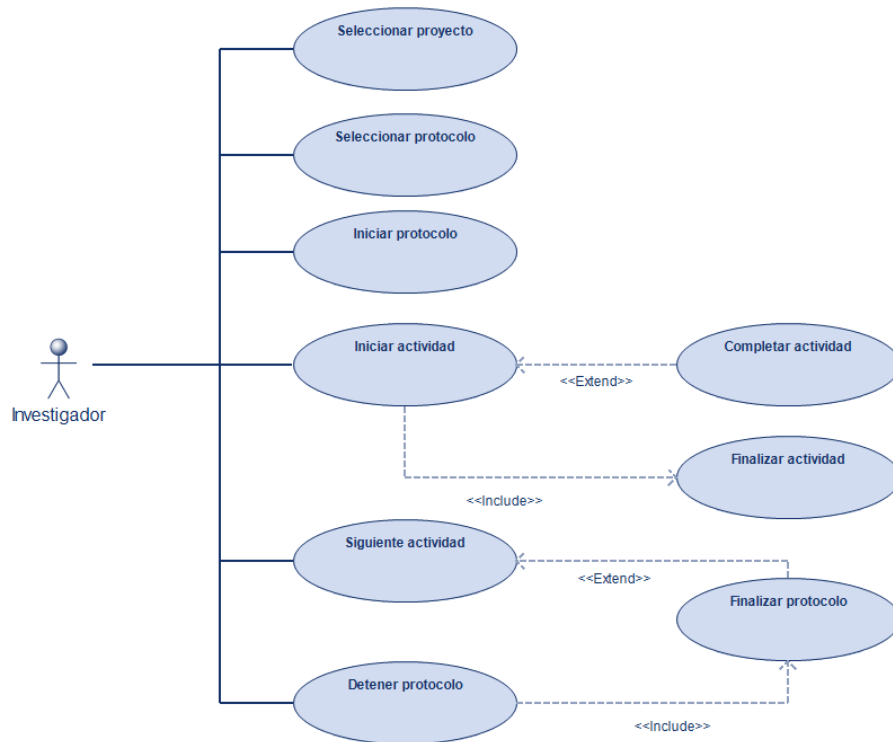


Figura 4.23: Diagrama de casos de uso - Ejecución de un protocolo.
Fuente: Elaboración propia, 2025.

La Tabla 4.12 presenta una descripción detallada de cada caso de uso involucrado en este escenario, indicando también sus respectivas precondiciones.

Tabla 4.12: Casos de uso del escenario de ejecución de un protocolo.

Caso de uso	Descripción	Precondición
Seleccionar proyecto	Define el contexto sobre el cual se ejecutará el protocolo.	Haber creado al menos un proyecto
Seleccionar protocolo	Permite elegir un protocolo definido previamente dentro del proyecto activo.	Haber seleccionado un proyecto y haber creado al menos un protocolo
Iniciar protocolo	Da comienzo a la ejecución secuencial de las actividades que componen el protocolo.	Haber seleccionado un protocolo
Iniciar actividad	Permite iniciar una actividad específica del protocolo en ejecución.	Haber iniciado el protocolo

Caso de uso	Descripción	Precondición
Completar actividad	Opción utilizada para finalizar manualmente una actividad que no posee tiempo límite.	Haber iniciado una actividad sin tiempo límite
Finalizar actividad	Marca el término de una actividad, ya sea de forma manual o automática.	Haber iniciado una actividad
Siguiente actividad	Avanza a la siguiente actividad del protocolo, si la actual ha finalizado.	Haber finalizado la actividad actual
Finalizar protocolo	Marca la conclusión del protocolo, ya sea porque se completaron todas las actividades o porque se detuvo manualmente.	Todas las actividades han finalizado o el protocolo ha sido detenido
Detener protocolo	Permite interrumpir la ejecución del protocolo en cualquier momento.	Haber iniciado el protocolo

Fuente: Elaboración propia, 2025.

4.5.3 Captura de datos

Este escenario contempla las acciones que puede realizar un investigador para capturar datos de forma multimodal. Involucra la gestión completa del proceso de captura, desde su inicio hasta su detención, permitiendo también pausar y reanudar la recolección de datos según sea necesario. La Figura 4.24 presenta el conjunto de casos de uso involucrados en este escenario.

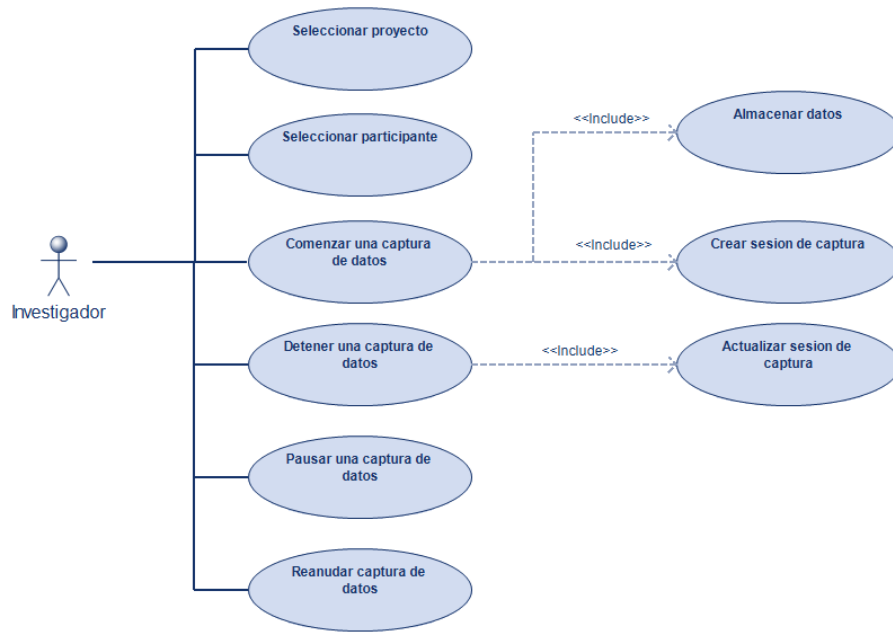


Figura 4.24: Diagrama de casos de uso - Captura de datos.
Fuente: Elaboración propia, 2025.

La Tabla 4.13 resume cada uno de los casos de uso que componen este escenario, junto con sus respectivas precondiciones.

Tabla 4.13: Casos de uso del escenario de captura de datos.

Caso de uso	Descripción	Precondición
Seleccionar proyecto	Permite definir el proyecto activo en el cual se realizará la captura de datos.	Haber creado al menos un proyecto
Seleccionar participante	Permite establecer el participante que estará asociado a la sesión de captura.	Haber seleccionado un proyecto y haber creado al menos un participante
Comenzar una captura de datos	Inicia una sesión de captura, asociando las configuraciones de plugins definidas previamente y generando una sesión de captura que será registrada por el sistema.	Haber seleccionado un proyecto y participante, y tener configuraciones de plugins de captura asociadas al proyecto
Pausar una captura de datos	Detiene temporalmente la captura, sin finalizar la sesión.	Haber comenzado una captura de datos

Caso de uso	Descripción	Precondición
Reanudar captura de datos	Permite continuar una captura que había sido pausada previamente.	Haber pausado una captura de datos
Detener una captura de datos	Finaliza la sesión de captura en curso, registrando los datos faltantes para la sesión.	Haber comenzado una captura de datos
Crear sesión de captura	Registra una nueva sesión de captura asociada al participante seleccionado.	Haber comenzado una captura de datos
Actualizar sesión de captura	Permite completar y actualizar los metadatos de una sesión luego de haber sido detenida.	Haber creado la sesión a actualizar
Almacenar datos	Representa el almacenamiento continuo de datos generados por los plugins durante la sesión de captura.	Haber comenzado una captura de datos

Fuente: Elaboración propia, 2025.

4.5.4 Reproducción de una sesión

Este escenario permite al investigador revisar sesiones previamente capturadas para analizar los datos recolectados a través de los distintos plugins de visualización disponibles. A través de la interfaz, el usuario puede seleccionar una sesión específica de un participante dentro de un proyecto determinado, iniciar su reproducción, pausar en cualquier momento o navegar libremente por la línea de tiempo para revisar los datos registrados. La Figura 4.25 presenta los casos de uso involucrados en este escenario, mientras que la Tabla 4.14 describe cada uno con sus respectivas precondiciones.

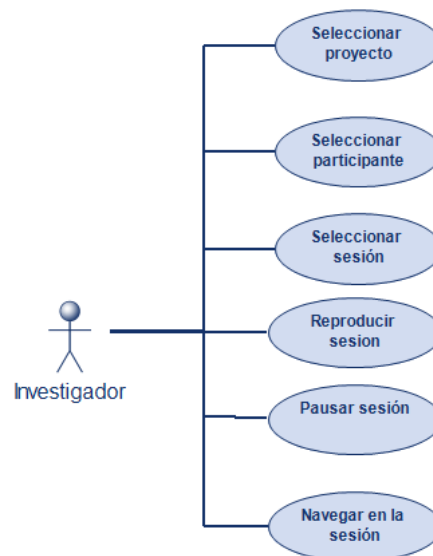


Figura 4.25: Diagrama de casos de uso - Reproducción de una sesión.
Fuente: Elaboración propia, 2025.

Tabla 4.14: Casos de uso del escenario de reproducción de una sesión.

Caso de uso	Descripción	Precondición
Seleccionar proyecto	Define el proyecto desde el cual se desea visualizar una sesión previamente capturada.	Haber creado al menos un proyecto
Seleccionar participante	Permite restringir las sesiones visibles a las asociadas a un participante específico del proyecto.	Haber seleccionado un proyecto y contar con al menos un participante
Seleccionar sesión	Define la sesión capturada que será reproducida, incluyendo su archivo y metadatos asociados.	Haber seleccionado un proyecto, un participante, y que existan sesiones capturadas
Reproducir sesión	Inicia la visualización de los datos multimodales correspondientes a la sesión seleccionada.	Estar en la pantalla de reproducción y haber seleccionado una sesión
Pausar sesión	Permite detener temporalmente la reproducción de la sesión en curso, sin perder el punto de reproducción.	Estar reproduciendo una sesión
Navegar en la sesión	Permite moverse libremente a un punto temporal específico dentro de la sesión en reproducción.	Estar en la pantalla de reproducción y haber seleccionado una sesión

Caso de uso	Descripción	Precondición
-------------	-------------	--------------

Fuente: Elaboración propia, 2025.

4.6 RESUMEN

En este capítulo se abordaron los aspectos fundamentales del diseño e implementación de la nueva versión de MO, utilizando el modelo de vistas 4+1 para organizar la arquitectura desde distintas perspectivas. Se detallaron la estructura física, los componentes de desarrollo, la lógica interna del sistema, los procesos clave de operación y los principales escenarios de uso. Esta visión integral permite comprender cómo las distintas partes del sistema interactúan entre sí, evidenciando su modularidad, extensibilidad y alineación con los objetivos del proyecto.

CAPÍTULO 5. EVALUACIÓN

Este capítulo presenta la evaluación del sistema desarrollado, detallando las distintas pruebas realizadas para asegurar su correcto funcionamiento, robustez y adecuación a los objetivos planteados. La evaluación se estructuró en distintos tipos de pruebas, abarcando aspectos tanto técnicos como funcionales del software.

En primer lugar, se realizaron pruebas unitarias y de integración para verificar el comportamiento correcto de los módulos individuales y su interacción dentro del sistema. También se incluyeron pruebas de análisis estático de código (*Static Code Analysis*, SCA), orientadas a detectar errores comunes, posibles vulnerabilidades y problemas de mantenimiento. A nivel de desempeño, se aplicaron pruebas de rendimiento para evaluar la eficiencia del sistema bajo diferentes condiciones de carga.

Adicionalmente, se evaluaron la validez y confiabilidad de los datos generados por los plugins de captura, con el fin de asegurar la calidad y precisión de la información recolectada. Finalmente, se llevaron a cabo pruebas de aceptación para verificar el cumplimiento de los requerimientos funcionales y no funcionales definidos.

Cabe señalar que no se realizaron pruebas de usabilidad durante el proceso de evaluación. Esta decisión se justifica debido a que la interfaz gráfica de usuario fue diseñada tomando como referencia herramientas consolidadas en el ámbito de la captura y sincronización de datos multimodales, como *OBS Studio* y *Noldus Hub*. Además, la evaluación de aspectos de experiencia de usuario no constituye el foco principal de este proyecto, cuyo objetivo principal se centró en la arquitectura y funcionamiento técnico del sistema.

5.1 AMBIENTE DE EJECUCIÓN DE PRUEBAS

Con el objetivo de obtener una evaluación más completa y representar distintos contextos de uso, las pruebas fueron realizadas en tres entornos de ejecución diferenciados, descritos a continuación:

- **Ambiente 1 — PC de escritorio (Windows 11):**

- Procesador: Intel Core i5-9400F
- Memoria RAM: 16 GB
- Tarjeta gráfica: NVIDIA GeForce GTX 1660
- Almacenamiento: SSD de 480 GB

- Sistema operativo: Windows 11

- **Ambiente 2 — Notebook (Windows 11):**

- Procesador: Intel Core i5-1135G7
- Memoria RAM: 8 GB
- Gráficos: integrados Intel Iris Xe
- Almacenamiento: SSD de 240 GB
- Sistema operativo: Windows 11

- **Ambiente 3 — PC de escritorio (Ubuntu 24.04 LTS):**

- Procesador: Intel Core i5-9400F
- Memoria RAM: 16 GB
- Tarjeta gráfica: NVIDIA GeForce GTX 1660
- Almacenamiento: Disco duro (HDD) de 1 TB
- Sistema operativo: Ubuntu Linux 24.04 LTS

Todas las pruebas, con excepción de las de rendimiento y aceptación, fueron ejecutadas exclusivamente en el primer ambiente, el cual representó el ambiente principal de desarrollo y validación del sistema. Esto incluye las pruebas unitarias, de integración, análisis estático de código (SCA), de cobertura, así como las pruebas de validez y confiabilidad de los plugins.

Por otro lado, las pruebas de rendimiento fueron replicadas en los otros ambientes definidos, con el propósito de observar el comportamiento del sistema bajo diferentes condiciones de hardware y sistema operativo. Las pruebas de aceptación, en cambio, no se encuentran consideradas dentro de estos ambientes, ya que fueron ejecutadas directamente por el profesor guía del proyecto, Roberto González-Ibáñez.

5.2 PRUEBAS UNITARIAS, DE INTEGRACIÓN Y DE ANÁLISIS ESTÁTICO

Las pruebas unitarias, de integración y de análisis estático de código fueron desarrolladas al finalizar cada iteración en el desarrollo de la aplicación, y permitieron verificar distintos aspectos del comportamiento y calidad del sistema. Las métricas obtenidas se organizaron de acuerdo con los criterios establecidos por la herramienta **SonarQube**, tanto para el backend como para el frontend.

A continuación, se describen los aspectos evaluados, **basados en las métricas y criterios definidos por SonarQube (2025):**

- **Seguridad** (*Security*): Evalúa el nivel de protección del sistema frente a accesos no autorizados, vulnerabilidades o destrucción de información.
- **Confiabilidad** (*Reliability*): Mide la capacidad del sistema para mantener un comportamiento correcto bajo condiciones esperadas durante un periodo determinado.
- **Mantenibilidad** (*Maintainability*): Estima la facilidad con que el código puede ser entendido, reparado o mejorado. Se reporta además el *Índice de deuda técnica* (*Debt Ratio*), que representa el esfuerzo técnico necesario para resolver los problemas detectados como proporción del tiempo requerido para reescribir el código afectado.
- **Cobertura** (*Coverage*): Corresponde al porcentaje de líneas de código cubiertas por las pruebas (unitarias e integración). Mientras mayor sea este valor, mayor garantía de que el comportamiento del sistema ha sido verificado.
- **Líneas duplicadas** (*Duplications*): Indica la proporción de líneas duplicadas dentro del código base.
- **Puntos críticos de seguridad** (*Security Hotspots*): Refiere a posibles puntos sensibles que deben revisarse manualmente, aunque no representan fallas confirmadas.

La escala de calificación utilizada por SonarQube para los tres primeros aspectos mencionados (seguridad, confiabilidad y mantenibilidad) se detalla a continuación:

Tabla 5.1: Criterios de calificación en SonarQube.

Calificación	Criterio
A	Sin errores o solo con observaciones informativas
B	Al menos un problema de severidad baja
C	Al menos un problema de severidad media
D	Al menos un problema de severidad alta
E	Al menos un problema de severidad crítica (bloqueante)

Fuente: SonarQube (2025).

Backend

Para el backend se utilizó **pytest** como framework principal de pruebas, acompañado de los módulos `pytest-cov` y `pytest-asyncio`. Las pruebas unitarias se enfocaron principalmente en validar la lógica de negocio de los distintos servicios, cubriendo la mayoría de los escenarios previstos. Por su parte, las pruebas de integración se aplicaron sobre las rutas expuestas por la API, verificando su funcionamiento principalmente en casos exitosos, sin contemplar pruebas de borde o errores intencionados.

Una prueba clave desarrollada para el núcleo del sistema fue la verificación de compatibilidad con plugins de captura. Esta se basa en un *fixture* con un plugin válido, y su propósito es garantizar que futuras modificaciones en la plataforma no rompan la compatibilidad con plugins desarrollados previamente. Esta prueba debe mantenerse vigente en todo momento.

Los resultados generales obtenidos se presentan en la Tabla 5.2. Se observa un alto nivel de calidad en todos los aspectos evaluados. En particular, el sistema no presentó ningún problema de seguridad, confiabilidad o mantenibilidad según el análisis de SonarQube, obteniendo calificación A en todos los indicadores. La cobertura de pruebas alcanzó un 95.2%, evidenciando que la gran mayoría del código ha sido verificado mediante pruebas unitarias y de integración. No se detectaron líneas duplicadas ni puntos críticos de seguridad.

Tabla 5.2: Resultados de pruebas unitarias, de integración y SCA para el backend.

Métrica	Valor	Calificación
Problemas de seguridad	0	A
Problemas de confiabilidad	0	A
Problemas de mantenibilidad	0 (Deuda técnica: 0%)	A
Cobertura de pruebas	95.2% (2017/2118 líneas cubiertas)	—
Líneas duplicadas	0% (0 líneas duplicadas)	—
Puntos críticos de seguridad	0	A

Fuente: Elaboración propia, 2025.

Frontend

Para el frontend se utilizaron las herramientas **Vitest**, **Playwright** y **Storybook**. En particular, Vitest fue utilizado para realizar pruebas unitarias e integración sobre componentes, integrándolos directamente en sus correspondientes *stories* de Storybook, que funcionan como representaciones visuales y casos de uso interactivos de los componentes.

Una limitación encontrada fue la imposibilidad de probar de forma unitaria componentes que interactúan con funcionalidades específicas de Electron, especialmente aquellas que dependen del *preload script*.

Para las pruebas de integración *end-to-end* se utilizó **Playwright**, acompañado de la librería `electron-playwright-helpers`. Esta configuración permitió lanzar la aplicación ya construida y verificar, por ejemplo, la carga de plugins o el inicio de una sesión de captura. Estas pruebas permiten validar la correcta interacción entre frontend y backend, aunque no se reflejan directamente en el reporte de cobertura.

Los resultados obtenidos se presentan en la Tabla 5.3. Al igual que en el backend, se obtuvo calificación A en todos los indicadores de calidad (seguridad, confiabilidad y mantenibilidad), sin

problemas ni puntos críticos. No obstante, la cobertura de pruebas alcanzó un 55.21%, debido principalmente a las limitaciones para testear componentes dependientes de Electron y a que las pruebas de integración no son reflejadas por el sistema de cobertura. Las duplicaciones fueron mínimas (0.7%).

Tabla 5.3: Resultados de pruebas unitarias, de integración y SCA para el frontend.

Métrica	Valor	Calificación
Problemas de seguridad	0	A
Problemas de confiabilidad	0	A
Problemas de mantenibilidad	0 (Deuda técnica: 0%)	A
Cobertura de pruebas	55.21% (1894/3430 líneas cubiertas)	–
Líneas duplicadas	0.7% (128 líneas duplicadas)	–
Puntos críticos de seguridad	0	A

Fuente: Elaboración propia, 2025.

5.3 PRUEBAS DE RENDIMIENTO

Las pruebas de rendimiento se enfocaron en evaluar el comportamiento del sistema bajo condiciones de uso intensivo, específicamente durante la ejecución de sesiones de captura de datos. Esta elección se justifica debido a que la captura constituye una de las funcionalidades principales de la aplicación y representa una de las operaciones más exigentes en términos de procesamiento y uso de memoria, al involucrar la ejecución concurrente de múltiples plugins, el manejo de eventos en tiempo real y la persistencia de datos.

Estas pruebas fueron de carácter semi-automatizado. La parte no automatizada consistió en preparar previamente el entorno, cargando la aplicación con los plugins de captura requeridos y asociándolos a una configuración válida dentro de un proyecto con su respectivo participante. Por otro lado, el proceso automatizado se implementó utilizando las herramientas **Playwright** y **Nut.js**. Este flujo automatizado permitió abrir la aplicación, verificar que los plugins estuvieran correctamente cargados y comenzar una sesión de captura. Durante dicha sesión, se simulaban movimientos del mouse y escritura mediante el teclado con patrones predefinidos, deterministas, para asegurar la repetibilidad y consistencia de las pruebas.

Cada iteración consistió en una sesión de captura de un minuto de duración, y se realizaron un total de 10 iteraciones por cada prueba, permitiendo obtener resultados estables y representativos.

Las métricas utilizadas para evaluar el rendimiento fueron el uso promedio de CPU (expresado en porcentaje) y el uso promedio de memoria RAM (expresado en megabytes). Estas métricas fueron recolectadas cada segundo durante la ejecución, mediante la biblioteca `systeminformation`.

La medición se centró en el conjunto de procesos correspondientes a la aplicación cliente y al servidor de backend (API), ejecutados localmente como procesos independientes.

Se diseñaron y ejecutaron cuatro pruebas distintas en cada uno de los ambientes definidos, variando los plugins de captura activos en cada caso:

- Captura multimodal: incluye los plugins de captura de teclado, mouse y pantalla.
- Captura solo con plugin de teclado.
- Captura solo con plugin de mouse.
- Captura solo con plugin de pantalla.

Para todas estas pruebas, se registraron todos los eventos disponibles ofrecidos por los plugins de entrada, incluyendo las pulsaciones y liberaciones de teclas, así como los movimientos, clics y desplazamientos del mouse. En el caso del plugin de captura de pantalla, se utilizó una configuración de resolución de 1920x1080 píxeles, a 30 fotogramas por segundo, con codificación en formato .mp4 mediante el códec H264.

Estos escenarios permiten analizar el impacto individual y combinado de los distintos tipos de captura soportados por el sistema, proporcionando así una visión más completa sobre el uso de recursos y la escalabilidad del sistema en condiciones reales.

Ambiente 1 - PC de escritorio (Windows 11)

Los resultados de las pruebas de rendimiento, resumidos en la Tabla 5.4, permiten analizar el comportamiento de la aplicación bajo distintas condiciones de carga asociadas a los tipos de captura.

Tabla 5.4: Resumen de resultados de las pruebas de rendimiento (Ambiente 1).

Tipo de prueba	Uso CPU promedio (%)	Uso RAM promedio (MB)
Captura multimodal	40.94 (0.51)	1913.79 (34.36)
Captura con plugin de teclado	4.09 (0.15)	533.23 (1.53)
Captura con plugin de mouse	8.67 (0.27)	529.61 (4.52)
Captura con plugin de pantalla	37.82 (0.42)	1634.76 (28.59)

Fuente: Elaboración propia, 2025.

La prueba con mayor exigencia fue la captura multimodal, que combina la ejecución simultánea de los plugins de teclado, mouse y pantalla. Esta configuración arrojó un uso promedio de CPU de aproximadamente 40.94 % y un uso promedio de memoria de 1913.79 MB, evidencian el carácter intensivo de esta modalidad de captura, que implica operaciones concurrentes de monitoreo

de eventos, procesamiento de imágenes (por parte del plugin de pantalla) y almacenamiento constante de datos. Los detalles por iteración se presentan en la Tabla 5.5.

Tabla 5.5: Resultados prueba de rendimiento (Ambiente 1): captura multimodal.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	41.37	1933.34
2	41.23	1871.21
3	41.44	1899.47
4	40.12	1926.05
5	40.94	1876.78
6	40.83	1954.67
7	41.67	1965.89
8	40.80	1925.60
9	41.20	1915.31
10	39.84	1869.55
Promedio (DE)	40.94 (0.51)	1913.79 (34.36)

Fuente: Elaboración propia, 2025.

En contraste, las pruebas individuales muestran un uso mucho más reducido de recursos. El plugin de teclado presentó el menor consumo, con un uso promedio de CPU de 4.09 % y memoria de 533.23 MB (ver Tabla 5.6). El plugin de mouse mostró un comportamiento ligeramente más demandante, con un uso de CPU de 8.67 % y memoria de 529.61 MB, atribuyéndose esto a la mayor frecuencia de eventos asociados al movimiento del puntero (ver Tabla 5.7). Por su parte, el plugin de pantalla alcanzó un uso de CPU promedio de 37.82 % y de memoria de 1634.76 MB, posicionándose como el componente más exigente a nivel individual, debido al procesamiento y almacenamiento continuo de capturas de imagen a lo largo de la sesión (ver Tabla 5.8).

Tabla 5.6: Resultados prueba de rendimiento (Ambiente 1): captura con plugin de teclado.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	4.02	537.24
2	4.11	532.15
3	4.15	532.09
4	4.19	533.37
5	4.10	533.22
6	4.05	532.61
7	3.83	532.45
8	3.97	531.65
9	4.12	533.22
10	4.38	534.28
Promedio (DE)	4.09 (0.15)	533.23 (1.53)

Fuente: Elaboración propia, 2025.

Tabla 5.7: Resultados prueba de rendimiento (Ambiente 1): captura con plugin del mouse.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	8.79	544.19
2	8.91	527.90
3	8.64	527.80
4	8.25	527.31
5	8.11	528.54
6	8.62	528.51
7	8.77	528.88
8	8.70	527.78
9	9.12	527.10
10	8.82	528.05
Promedio (DE)	8.67 (0.27)	529.61 (4.52)

Fuente: Elaboración propia, 2025.

Tabla 5.8: Resultados prueba de rendimiento (Ambiente 1): captura con plugin de pantalla.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	37.08	1652.01
2	37.94	1681.24
3	38.05	1641.90
4	37.75	1589.06
5	37.58	1598.39
6	38.06	1646.50
7	37.85	1649.19
8	37.94	1614.56
9	38.52	1657.24
10	37.38	1617.54
Promedio (DE)	37.82 (0.42)	1634.76 (28.59)

Fuente: Elaboración propia, 2025.

Estos resultados permiten identificar oportunidades concretas de optimización a futuro, especialmente en lo relativo al plugin de captura de pantalla, que representa el mayor impacto individual en el uso de recursos del sistema. Actualmente, este plugin utiliza bibliotecas multiplataforma para realizar la captura de pantalla, lo cual, si bien asegura compatibilidad entre sistemas operativos, introduce una sobrecarga adicional al no estar optimizado para cada plataforma específica. Una posible mejora consiste en implementar la captura de forma nativa para cada sistema operativo, utilizando interfaces propias que aprovechen mejor las capacidades del entorno en que se ejecuta.

Por otro lado, el proceso de codificación y almacenamiento del video se realiza actualmente utilizando `opencv-python`. Si bien esta biblioteca entrega una solución integrada para captura y codificación, se podría evaluar el uso directo de herramientas más especializadas como FFmpeg, que permitirían un mayor control sobre el rendimiento, la compresión y el uso eficiente de recursos. Adicionalmente, se podría considerar la incorporación de soporte para aceleración por hardware mediante GPU como una opción configurable, lo cual podría reducir significativamente

la carga sobre la CPU durante sesiones de alta demanda.

Cabe señalar que estas optimizaciones no fueron abordadas durante el desarrollo del presente proyecto, ya que su enfoque principal se centró en la arquitectura y funcionamiento general del sistema, y no en la optimización de rendimiento a bajo nivel.

Ambiente 2 - Notebook (Windows 11)

En este ambiente de pruebas, los resultados obtenidos permiten comparar el desempeño del sistema en un entorno de hardware distinto. La Tabla 5.9 presenta un resumen general de las métricas obtenidas para cada tipo de captura.

Tabla 5.9: Resumen de resultados de las pruebas de rendimiento (Ambiente 2).

Tipo de prueba	Uso CPU promedio (%)	Uso RAM promedio (MB)
Captura multimodal	35.38 (1.60)	1626.86 (93.39)
Captura con plugin de teclado	2.68 (0.23)	526.35 (6.24)
Captura con plugin de mouse	5.57 (0.11)	533.67 (1.97)
Captura con plugin de pantalla	31.66 (0.36)	1245.32 (37.13)

Fuente: Elaboración propia, 2025.

El uso promedio de CPU y memoria en la prueba de captura multimodal fue de 35.38% y 1626.86MB respectivamente (Tabla 5.10), cifras algo inferiores a las registradas en el ambiente de escritorio, lo cual sugiere una buena adaptación del sistema a contextos menos potentes, sin que se hayan identificado interrupciones ni pérdidas de datos.

Tabla 5.10: Resultados prueba de rendimiento (Ambiente 2): captura multimodal.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	35.06	1683.29
2	34.97	1735.80
3	34.78	1571.18
4	34.80	1563.16
5	33.85	1566.35
6	34.48	1662.92
7	33.60	1566.18
8	35.93	1689.88
9	38.13	1738.28
10	38.18	1491.53
Promedio (DE)	35.38 (1.60)	1626.86 (93.39)

Fuente: Elaboración propia, 2025.

En las pruebas individuales, el plugin de teclado presentó el menor uso de recursos, con 2.68% de CPU y 526.35MB de memoria (Tabla 5.11). Le siguió el plugin de mouse con 5.57% de CPU

y 533.67MB (Tabla 5.12). Por último, el plugin de pantalla volvió a ser el más exigente a nivel individual, alcanzando 31.66% de CPU y 1245.32MB de RAM (Tabla 5.13).

Tabla 5.11: Resultados prueba de rendimiento (Ambiente 2): captura con plugin de teclado.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	3.09	535.71
2	3.11	530.44
3	2.50	529.37
4	2.47	529.56
5	2.70	530.85
6	2.54	528.50
7	2.53	523.15
8	2.47	517.61
9	2.58	518.81
10	2.77	519.51
Promedio (DE)	2.68 (0.23)	526.35 (6.24)

Fuente: Elaboración propia, 2025.

Tabla 5.12: Resultados prueba de rendimiento (Ambiente 2): captura con plugin del mouse.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	5.46	538.70
2	5.43	533.63
3	5.77	533.28
4	5.70	533.50
5	5.51	532.31
6	5.59	533.90
7	5.52	534.30
8	5.47	531.81
9	5.66	531.83
10	5.56	533.40
Promedio (DE)	5.57 (0.11)	533.67 (1.97)

Fuente: Elaboración propia, 2025.

Tabla 5.13: Resultados prueba de rendimiento (Ambiente 2): captura con plugin de pantalla.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	31.13	1308.26
2	31.68	1296.21
3	31.61	1275.25
4	31.87	1238.29
5	31.14	1235.27
6	31.35	1242.33
7	31.76	1254.86
8	31.86	1212.43
9	31.78	1185.71
10	32.42	1204.60
Promedio (DE)	31.66 (0.36)	1245.32 (37.13)

Fuente: Elaboración propia, 2025.

Si bien los resultados globales muestran un comportamiento estable y aceptable, el uso de recursos sigue siendo elevado en el caso del plugin de pantalla. Esto evidencia que aún existen oportunidades de mejora en términos de eficiencia.

Ambiente 3 — PC de escritorio (Ubuntu 24.04)

Este ambiente corresponde al mismo equipo utilizado en el Ambiente 1, pero con una configuración distinta en cuanto al sistema operativo y medio de almacenamiento. En este caso, MO fue ejecutado sobre el sistema operativo Ubuntu 24.04 LTS, instalado en un disco duro mecánico (HDD) de 1 TB, en lugar de una unidad SSD. Este entorno permite observar el desempeño de la aplicación en un sistema Linux, con diferencias relevantes respecto a la gestión de procesos, uso de memoria y rendimiento de disco.

Los resultados generales se presentan en la Tabla 5.14. En comparación con los ambientes anteriores, se observa una tendencia consistente a un menor uso de CPU, pero un mayor uso de memoria RAM, particularmente en las pruebas que incluyen el plugin de captura de pantalla. En la prueba de captura multimodal, por ejemplo, el uso promedio de CPU fue de 25.00 % (frente a 40.94 % en Windows), pero el uso de RAM alcanzó los 2625.33 MB, superando ampliamente los registros de los otros ambientes.

Tabla 5.14: Resumen de resultados de las pruebas de rendimiento (Ambiente 3).

Tipo de prueba	Uso CPU promedio (%)	Uso RAM promedio (MB)
Captura multimodal	25.00 (0.88)	2625.33 (110.63)
Captura con plugin de teclado	1.51 (0.06)	680.94 (1.58)
Captura con plugin de mouse	6.00 (0.35)	687.77 (2.23)
Captura con plugin de pantalla	23.16 (0.54)	2513.55 (347.51)

Fuente: Elaboración propia, 2025.

La Tabla 5.15 presenta el detalle por iteración de la prueba multimodal. Se observa una mayor dispersión en el uso de memoria en comparación con los otros ambientes, con picos que superan los 3100 MB. Este comportamiento sugiere una menor estabilidad en la demanda de recursos durante la captura, posiblemente influenciado por el uso de un disco HDD, que puede afectar la eficiencia en el manejo de archivos de salida.

Tabla 5.15: Resultados prueba de rendimiento (Ambiente 3): captura multimodal.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	24.04	2549.28
2	26.07	2819.49
3	25.71	2550.97
4	24.03	2538.32
5	23.97	2533.19
6	25.28	2569.24
7	24.62	2793.91
8	25.44	2693.34
9	26.34	2540.08
10	24.52	2665.47
Promedio (DE)	25.00 (0.88)	2625.33 (110.63)

Fuente: Elaboración propia, 2025.

En las pruebas individuales, el plugin de teclado mantuvo su bajo nivel de consumo, con un uso promedio de CPU de 1.51 % y 680.94 MB de memoria RAM (Tabla 5.16). El plugin de mouse presentó un uso de CPU de 6.00 % y un consumo de 687.77 MB de RAM (Tabla 5.17). Por su parte, el plugin de pantalla volvió a destacar como el más exigente dentro de las capturas unimodales, registrando un uso de CPU de 23.16 % y un consumo promedio de memoria de 2513.55 MB, valor superior al observado en los ambientes anteriores (Tabla 5.18).

Tabla 5.16: Resultados prueba de rendimiento (Ambiente 3): plugin de captura del teclado.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	1.34	678.60
2	1.51	677.41
3	1.53	681.44
4	1.54	681.44
5	1.55	681.42
6	1.53	681.58
7	1.52	681.83
8	1.51	681.75
9	1.51	682.03
10	1.54	681.86
Promedio (DE)	1.51 (0.06)	680.94 (1.58)

Fuente: Elaboración propia, 2025.

Tabla 5.17: Resultados prueba de rendimiento (Ambiente 3): plugin de captura del mouse.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	5.09	684.89
2	6.18	683.97
3	6.07	684.89
4	6.17	689.08
5	5.89	688.78
6	6.43	688.77
7	5.92	689.04
8	6.04	689.32
9	6.06	689.48
10	6.10	689.48
Promedio (DE)	6.00 (0.35)	687.77 (2.23)

Fuente: Elaboración propia, 2025.

Tabla 5.18: Resultados prueba de rendimiento (Ambiente 3): plugin de captura de pantalla.

Iteración	Uso CPU promedio (%)	Uso RAM promedio (MB)
1	21.65	2274.38
2	23.18	2347.31
3	23.27	2292.89
4	23.40	2293.38
5	23.39	2292.86
6	23.31	2311.93
7	23.43	2289.06
8	23.21	2957.78
9	23.38	3132.16
10	23.39	2943.77
Promedio (DE)	23.16 (0.54)	2513.55 (347.51)

Fuente: Elaboración propia, 2025.

Este comportamiento indica que, en entornos Linux, MO presenta una menor carga de CPU pero un mayor consumo de memoria en comparación con Windows, posiblemente debido a diferencias en la gestión del sistema operativo y al uso de un disco HDD. Aun así, durante las sesiones de captura no se observaron interrupciones ni pérdida de datos, lo que confirma que MO puede ejecutarse correctamente en Linux, manteniendo su compatibilidad multiplataforma.

5.4 PRUEBAS DE VALIDEZ Y CONFIABILIDAD

Para la evaluación de la validez y confiabilidad de los datos generados por los plugins de captura desarrollados para MO, se utilizó como referencia el marco metodológico propuesto por Mauricio Soto Piña Soto Piña (2021). Dicho marco fue concebido específicamente para esta herramienta, abordando la necesidad de sistematizar los procesos evaluativos de las extensiones desarrolladas

para MO.

Este marco establece una metodología clara y replicable para la evaluación de extensiones de captura, basada en dos dimensiones fundamentales: validez y confiabilidad. La validez se evalúa a través de la comparación con una línea de referencia validada (*ground truth*), mientras que la confiabilidad se analiza mediante pruebas estadísticas que permiten determinar el grado de consistencia en las mediciones realizadas bajo condiciones constantes.

Dado que el presente trabajo consiste en una actualización tecnológica de la plataforma MO, incluyendo cambios en su arquitectura e interfaces, así como la migración de sus componentes a nuevas tecnologías, resulta necesario revalidar el comportamiento de los plugins de captura del mouse, teclado y pantalla implementados, frente a las nuevas condiciones de ejecución.

Se utilizaron las mismas métricas e indicadores definidos en el marco de evaluación de Soto Piña para los respectivos plugins, manteniendo la lógica metodológica de validación y confiabilidad propuesta por el autor. La única diferencia introducida en esta implementación fue la sustitución de la línea de referencia (*ground truth*) para los plugins de captura de mouse y teclado. En lugar de la herramienta externa MacroRecorder, se optó por el uso de la biblioteca **Nut.js**, la cual fue integrada directamente en el código de pruebas automatizadas desarrolladas con **Playwright**.

5.4.1 Evaluación del plugin de captura de mouse

Para el plugin de captura de mouse, las métricas de evaluación definidas por Soto Piña (2021) son: número total de clics, cantidad de clics izquierdos y derechos, y coordenadas del puntero al inicio y término de la ejecución. Estas métricas permiten evaluar tanto eventos discretos como trayectorias espaciales, cubriendo así los principales tipos de información generada por esta extensión.

La validación se realizó comparando los datos recolectados por MO contra los valores generados por la biblioteca Nut.js, la cual actuó como *ground truth* al emitir de manera programada las acciones de mouse. Los datos fueron capturados a lo largo de cinco ejecuciones repetidas del mismo script automatizado.

Como puede observarse, los valores entregados por MO coinciden exactamente con los generados por Nut.js en todas las métricas consideradas. Esto se refleja en un error relativo igual a 0% para todos los casos, lo que indica una validez perfecta respecto a la referencia utilizada.

Tabla 5.19: Mediciones de las métricas en la captura de mouse con MO y Nut.js.

Fuente	<i>Clicks totales</i>	<i>Clicks izquierdos</i>	<i>Clicks derechos</i>	X Inicial	Y Inicial	X Final	Y Final
Prueba 1							
MO	41	23	18	528	977	767	789
Nut.js	41	23	18	528	977	767	789
Prueba 2							
MO	41	23	18	528	977	767	789
Nut.js	41	23	18	528	977	767	789
Prueba 3							
MO	41	23	18	528	977	767	789
Nut.js	41	23	18	528	977	767	789
Prueba 4							
MO	41	23	18	528	977	767	789
Nut.js	41	23	18	528	977	767	789
Prueba 5							
MO	41	23	18	528	977	767	789
Nut.js	41	23	18	528	977	767	789
Error relativo (%)	0	0	0	0	0	0	0

Fuente: Elaboración propia, 2025.

En cuanto a la confiabilidad, se evaluó la estabilidad de las mediciones en ejecuciones repetidas del mismo escenario, calculando el coeficiente de variación (CV_{RMS}), que mide la dispersión relativa para cada métrica. Los resultados, mostrados en la Tabla 5.20, también reflejan un desempeño excelente: todas las métricas obtuvieron un coeficiente de variación de 0, lo que evidencia una consistencia total en las mediciones realizadas por el plugin de MO.

Tabla 5.20: Coeficiente de varianza por métrica de mouse.

	<i>Clicks totales</i>	<i>Clicks izquierdos</i>	<i>Clicks derechos</i>	X Inicial	Y Inicial	X Final	Y Final
CV_{RMS}	0	0	0	0	0	0	0

Fuente: Elaboración propia, 2025.

Estos resultados se resumen gráficamente en la Figura 5.1, a través de un gráfico tipo diana que representa el desvío porcentual de cada métrica respecto al ground truth. Todas las métricas se alinean exactamente con la línea base de referencia (0%), reforzando tanto la precisión como la estabilidad del plugin.

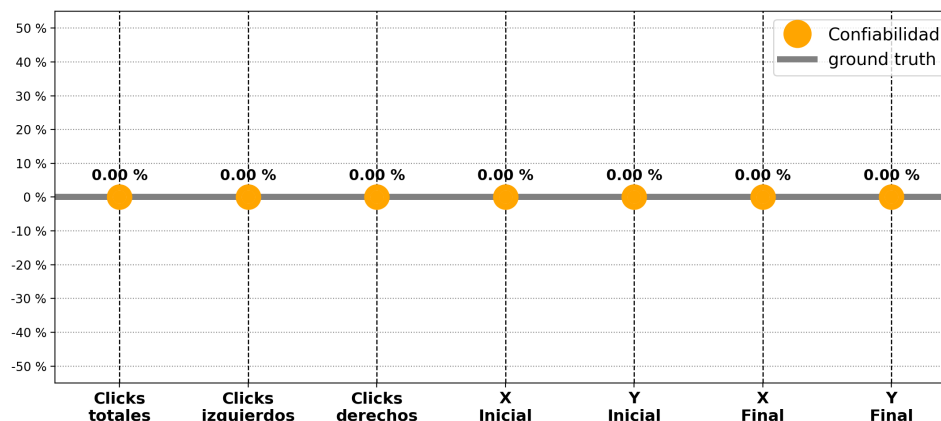


Figura 5.1: Gráfico de diapa para la captura de mouse.
Fuente: Elaboración propia, 2025.

En conjunto, los resultados obtenidos permiten concluir que el plugin de captura de mouse implementado en MO es completamente confiable y válido en el contexto de ejecución evaluado, cumpliendo con todos los criterios establecidos en el marco metodológico de evaluación utilizado.

5.4.2 Evaluación del plugin de captura de teclado

Para el plugin de captura de teclado, las métricas de evaluación consideradas por Soto Piña (2021) son: número total de pulsaciones (*keystrokes*), cantidad de palabras correctamente tipeadas y la distancia de Levenshtein entre el texto esperado y el texto registrado. Estas métricas permiten evaluar tanto la precisión a nivel de eventos individuales como la fidelidad del contenido textual completo generado a partir de las pulsaciones capturadas.

Al igual que en el caso anterior, la validación se llevó a cabo comparando los datos registrados por MO con los generados por la biblioteca `Nut.js`, que operó como línea base (*ground truth*). En este caso, el texto fue tipeado automáticamente en cinco ejecuciones repetidas de un mismo script controlado por Playwright, garantizando la consistencia del contenido ingresado en cada iteración.

Los resultados obtenidos se muestran en la Tabla 5.21. Al analizar las tres métricas consideradas, se observa una coincidencia exacta entre los valores registrados por MO y aquellos entregados por `Nut.js`, lo que equivale a un error relativo de 0% para todos los casos. Este resultado indica que el plugin fue capaz de capturar correctamente cada pulsación de teclado y reconstruir con total fidelidad el texto ingresado.

Tabla 5.21: Mediciones de las métricas en la captura de teclado con MO y Nut.js.

Fuente	Keystrokes	Palabras correctas	Levenshtein Distance
Prueba 1			
MO	123	16	0
Nut.js	123	16	0
Prueba 2			
MO	123	16	0
Nut.js	123	16	0
Prueba 3			
MO	123	16	0
Nut.js	123	16	0
Prueba 4			
MO	123	16	0
Nut.js	123	16	0
Prueba 5			
MO	123	16	0
Nut.js	123	16	0
Error relativo (%)	0	0	0

Fuente: Elaboración propia, 2025.

En términos de confiabilidad, la Tabla 5.22 presenta los coeficientes de variación calculados a partir de las cinco repeticiones realizadas. Tal como se evidencia, todas las métricas alcanzaron un valor de CV_{RMS} igual a 0, demostrando una estabilidad total entre ejecuciones equivalentes del mismo escenario de entrada.

Tabla 5.22: Coeficiente de variación por métrica de teclado.

	Keystrokes	Palabras correctas	Levenshtein Distance
CV_{RMS}	0	0	0

Fuente: Elaboración propia, 2025.

Estos resultados son complementados por el gráfico tipo diana de la Figura 5.2, el cual representa visualmente el desvío porcentual respecto al *ground truth*. Tal como se muestra, todos los puntos coinciden con la referencia (0%), lo que refuerza la validez y consistencia del comportamiento del plugin evaluado.

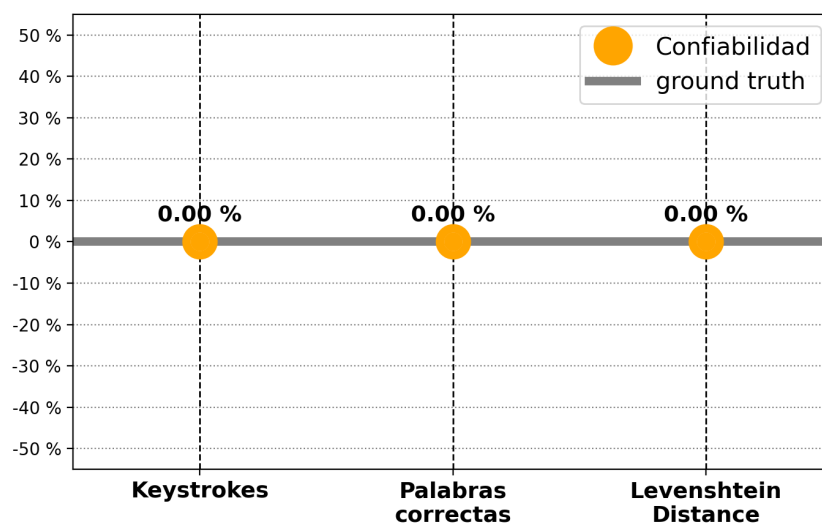


Figura 5.2: Gráfico de diana para la captura de teclado.
Fuente: Elaboración propia, 2025.

En síntesis, los resultados permiten concluir que el plugin de captura de teclado implementado en MO presenta un nivel de desempeño válido y completamente confiable, siendo capaz de registrar con precisión y estabilidad las entradas de texto generadas.

5.4.3 Evaluación del plugin de captura de pantalla

Para evaluar la validez y confiabilidad del plugin de captura de pantalla desarrollado para MO, estas pruebas se realizaron comparando su rendimiento frente a la herramienta de referencia OBS Studio, utilizada como línea base para establecer los valores esperados. Se mantuvieron las mismas métricas definidas por Soto Piña (2021), a saber: tasa promedio de cuadros por segundo (FPS), duración de la sesión de captura (en segundos) y cantidad de cuadros perdidos (*frame drops*) durante la grabación. Estas métricas fueron evaluadas en tres escenarios distintos, definidos por tasas de captura de 15, 30 y 45 FPS, manteniendo constante la resolución en 1920x1080 píxeles. Para cada escenario se realizaron tres ejecuciones controladas mediante scripts automatizados con Playwright, garantizando la consistencia de las condiciones de prueba. Los resultados obtenidos se presentan en la Tabla 5.23. Como puede observarse, MO mantuvo una duración de captura constante en todos los casos (30 segundos), replicando correctamente el valor de referencia. Sin embargo, se observaron diferencias significativas en las tasas de cuadros por segundo, especialmente a medida que aumentaba la exigencia del escenario. Para el caso de 15 FPS, el error relativo fue de 2.87%, mientras que para 30 FPS alcanzó un 19.77%, y en

el escenario de 45 FPS se elevó hasta 44.69% (ver Tabla 5.24). Este comportamiento se tradujo también en una creciente cantidad de cuadros perdidos, desde menos de un cuadro en promedio en el primer escenario, hasta más de 20 cuadros perdidos por prueba a 45 FPS.

Tabla 5.23: Mediciones de las métricas en la captura de pantalla con MO y OBS.

	Prueba 1			Prueba 2			Prueba 3		
	FPS	Duración Captura [s]	Drops [frame]	FPS	Duración Captura [s]	Drops [frame]	FPS	Duración Captura [s]	Drops [frame]
15fps									
MO	14.71	30	0.29	14.55	30	0.45	14.45	30	0.55
OBS	15	30	0	15	30	0	15	30	0
30fps									
MO	24.07	30	5.93	24.13	30	5.87	24	30	6
OBS	30	30	0	30	30	0	30	30	0
45fps									
MO	24.77	30	20.23	24.97	30	20.03	24.93	30	20.07
OBS	45	30	0	45	30	0	45	30	0

Fuente: Elaboración propia, 2025.

Tabla 5.24: Resumen y errores relativos entre MO y OBS para distintas tasas de FPS.

	FPS	Duración Captura [s]	Drops [frame]
15fps			
MO	14.57	30.00	0.43
OBS	15.00	30.00	0
Error Relativo (%)	2.87%	0%	-
30fps			
MO	24.07	30.00	5.93
OBS	30.00	30.00	0
Error Relativo (%)	19.77%	0%	-
45fps			
MO	24.89	30.00	20.11
OBS	45.00	30.00	0
Error Relativo (%)	44.69%	0%	-

Fuente: Elaboración propia, 2025.

En cuanto a la confiabilidad de las mediciones, la Tabla 5.25 muestra los coeficientes de variación obtenidos para las distintas métricas. Los valores registrados reflejan una estabilidad completa en la duración de la captura (CV = 0 en todos los casos), y una variación relativamente baja en las tasas de FPS (0.90 para 15 FPS, 0.28 para 30 FPS y 0.41 para 45 FPS), lo cual indica

una reproducción estable de las condiciones dentro de cada escenario, pese a las limitaciones observadas a nivel de rendimiento.

Tabla 5.25: Coeficiente de variación por métrica de pantalla.

	15fps		30fps		45fps	
	FPS	Duración Captura	FPS	Duración Captura	FPS	Duración Captura
CV_{RMS}	0.90	0	0.28	0	0.41	0

Fuente: Elaboración propia, 2025.

Estos resultados sugieren que, si bien el plugin logra mantener la duración de las sesiones de manera precisa y consistente, su desempeño se ve comprometido en escenarios de captura con altas exigencias de tasa de cuadros por segundo. Esto implica que, bajo condiciones de captura más demandantes (como 30 FPS o más), el plugin no logra sostener el rendimiento esperado, lo cual afecta directamente la validez de los datos generados.

Este comportamiento se representa visualmente en la Figura 5.3, mediante un gráfico tipo diana que muestra el desvío porcentual de las métricas registradas por MO respecto al *ground truth* establecido por OBS Studio. Se puede observar que, mientras la duración de captura se mantiene alineada con la referencia (0% de desvío), la tasa de FPS muestra una desviación creciente conforme aumenta la exigencia del escenario, lo que refuerza las conclusiones previamente descritas.

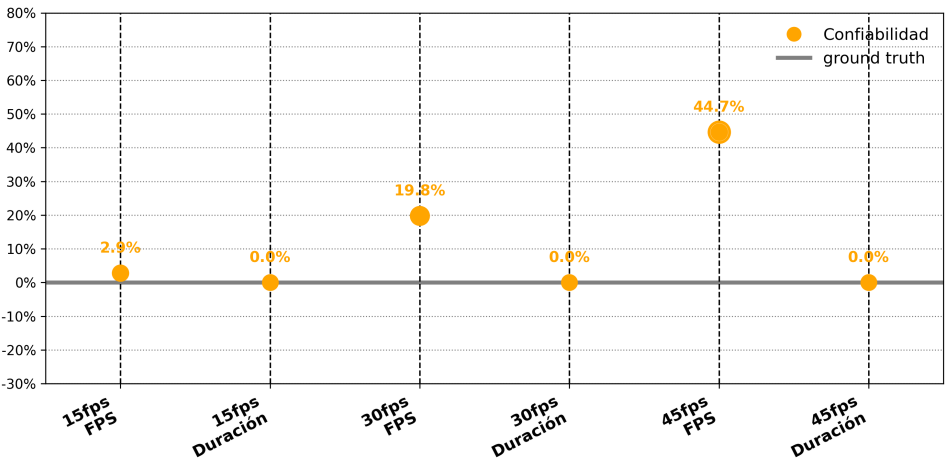


Figura 5.3: Gráfico de diana para la captura de pantalla.
Fuente: Elaboración propia, 2025.

5.5 PRUEBAS DE ACEPTACIÓN

Las pruebas de aceptación fueron aplicadas con el objetivo de validar que el sistema cumpliera con los requisitos funcionales y no funcionales definidos, desde una perspectiva de usuario. Estas pruebas fueron ejecutadas por el profesor guía del proyecto, Roberto González-Ibáñez, quien actuó como evaluador externo del sistema desarrollado. Se definieron un total de ocho pruebas de aceptación, cada una correspondiente a un conjunto de funcionalidades clave de la aplicación. Cada prueba consistió en una secuencia detallada de pasos, incluyendo precondiciones, requisitos abordados y resultados esperados. Posteriormente, se asignó una calificación conforme al nivel de cumplimiento observado, siguiendo la escala de evaluación mostrada en la Tabla 5.26.

Tabla 5.26: Categorías de evaluación de las pruebas de aceptación.

Evaluación	Significado	Porcentaje de cumplimiento
A	Completa	85–100%
B	Aceptada con observaciones	55–84.9%
C	Incompleta	45–54.9%
D	Insuficiente	15–44.9%
E	No aceptada	0–14.9%

Fuente: Cáceres Rodríguez (2018).

Del total de pruebas realizadas, cuatro fueron evaluadas con una calificación de **A**, lo que indica un cumplimiento completo de los criterios definidos. Las otras cuatro pruebas recibieron una calificación de **B**, siendo aceptadas con observaciones menores, sin que estas afecten el correcto funcionamiento del sistema. Estos resultados, resumidos en la Tabla 5.27, reflejan un buen nivel de madurez del sistema en términos de funcionalidades clave, con oportunidades puntuales de mejora en aspectos de experiencia de usuario y consistencia. El detalle completo de cada prueba de aceptación se presenta en el Apéndice C.

Tabla 5.27: Resultados generales de las pruebas de aceptación.

ID	Título de la prueba	Evaluación
PA-01	Operaciones con plugins	A
PA-02	Configuración inicial de un proyecto	A
PA-03	Proceso de captura y ejecución de protocolo	B
PA-04	Reproducción de sesión capturada	B
PA-05	Operaciones extra participante	A
PA-06	Operaciones extra protocolo	B
PA-07	Operaciones extra plugin configurable	B
PA-08	Operaciones extra proyecto	A

Fuente: Elaboración propia, 2025.

CAPÍTULO 6. CONCLUSIONES

Este capítulo presenta las conclusiones obtenidas a partir del trabajo realizado, analizando el grado de cumplimiento de los objetivos planteados inicialmente. Asimismo, se abordan las implicancias, alcances y limitaciones del proyecto, y se deja espacio para propuestas de trabajo futuro y reflexiones personales.

6.1 OBJETIVOS

Durante el desarrollo del proyecto se abordaron los cinco objetivos específicos establecidos en la etapa inicial. En las siguientes secciones se presenta una evaluación de cada uno en relación con los resultados obtenidos.

6.1.1 Objetivos específicos

- **Diagnosticar el estado actual de MO en sus distintas versiones identificando componentes críticos y la compatibilidad con los plugins vigentes de MO:** Este objetivo fue plenamente alcanzado. Se realizó un diagnóstico mediante la comparación entre versiones del sistema, análisis de compatibilidad de plugins y detección de fallas recurrentes. Los hallazgos identificaron fallos estructurales en la evolución del núcleo, como el incremento del acoplamiento, la pérdida de compatibilidad con plugins anteriores y problemas derivados de la migración a JavaFX. Este análisis sirvió como base sólida para justificar la necesidad de una reestructuración completa del sistema.
- **Rediseñar el núcleo de MO preservando su estructura modular y extensible:** El rediseño del núcleo se realizó en base a principios de modularidad, separación de responsabilidades y extensibilidad. Se introdujo una arquitectura dividida entre frontend y backend, facilitando la evolución independiente de cada capa. Además, se implementó un sistema de plugins claramente desacoplado, que preserva y mejora la capacidad de MO de adaptarse a nuevas funcionalidades.
- **Implementar el núcleo de MO a partir del rediseño, reutilización de componentes y el uso de tecnologías actualizadas:** Este objetivo se cumple con éxito mediante la reimplementación del sistema utilizando tecnologías modernas como Electron, React y

Python. Se reutilizaron conceptos funcionales del sistema original y se integraron buenas prácticas actuales. La implementación dio origen a un núcleo operativo que sirve como base para la extensión mediante módulos y plugins.

- **Implementar un instalador para el núcleo de MO y el conjunto de plugins creados:**

Se desarrolló un instalador gráfico funcional que permite a los usuarios seleccionar plugins, definir el directorio de instalación y ejecutar la aplicación una vez completado el proceso. Esta herramienta facilita la distribución y adopción de la nueva versión de MO.

- **Demostrar la funcionalidad de la nueva versión mediante la creación de al menos 5 plugins:**

El objetivo se supera, desarrollando un conjunto de plugins que cubren funcionalidades de captura (teclado, mouse, pantalla) y reproducción (video, teclado, mouse). Estos plugins permiten validar el funcionamiento completo del sistema, desde la configuración hasta la visualización de datos, cumpliendo el propósito de demostrar la extensibilidad del nuevo núcleo.

6.1.2 Objetivo general

Actualizar el núcleo de MO a través de un proceso integral de reimplementación, refactorización y migración tecnológica para permitir la extensibilidad del software en base a plugins que dan soporte a nuevos sensores.

Este objetivo general se considera completamente cumplido. A lo largo del proyecto se realizó una actualización profunda del núcleo de MO, abordando tanto el rediseño conceptual como la implementación técnica. La nueva arquitectura permite la incorporación sencilla de nuevos sensores y fuentes de datos mediante un sistema de plugins robusto y bien documentado. Además, se migraron todas las funcionalidades clave a tecnologías modernas, mejorando la mantenibilidad y la capacidad de evolución del sistema.

6.2 IMPLICANCIAS Y ALCANCES

6.2.1 Implicancias

La reestructuración del núcleo de MO implica una transformación importante en su forma de uso, mantenimiento y evolución. Esta nueva versión permite a investigadores y desarrolladores

adaptar el sistema a una gran variedad de contextos de captura y análisis multimodal, gracias a su enfoque extensible y modular. Asimismo, la estandarización de los mecanismos de configuración, internacionalización y empaquetamiento de plugins facilita la creación y distribución de nuevas extensiones.

6.2.2 Alcances

El alcance del proyecto contempló el rediseño y la implementación completa del núcleo de MO, incluyendo la separación entre frontend y backend, la definición de un sistema de plugins para captura y reproducción, la implementación de un conjunto representativo de plugins funcionales y la creación de un instalador.

Sin embargo, algunos elementos quedaron fuera del alcance, entre ellos: la migración de todos los plugins históricos, la implementación de un repositorio centralizado de plugins, la actualización de los módulos de análisis, comunicación y actualización.

6.3 TRABAJO FUTURO

El trabajo realizado en este proyecto sienta una base sólida para continuar con la evolución de MO, permitiendo que sus características históricas y nuevas funcionalidades puedan seguir migrándose y consolidándose sobre esta nueva versión. A partir de esta base, se identifican diversas líneas de trabajo futuro que permitirán ampliar el alcance y mejorar el desempeño del sistema:

- Migración de plugins desarrollados en versiones anteriores: A pesar de que se implementaron plugins clave para validar el nuevo núcleo, existen múltiples extensiones previamente desarrolladas que aún no han sido adaptadas. La migración de estos plugins permitiría recuperar funcionalidades valiosas y enriquecer el ecosistema de plugins disponibles.
- Implementación de módulos no abordados en este proyecto: Durante este trabajo no se abordaron los módulos de análisis, comunicación y actualización, los cuales son parte del diseño modular de MO. La implementación de estos módulos permitiría enriquecer las funcionalidades de la aplicación, con las características clave con las que contaba MO.
- Desarrollo de un repositorio centralizado de plugins: Una mejora clave sería implementar un sistema de distribución de plugins, que incluya una página web para registrar y compartir

extensiones, y una interfaz dentro de MO para explorar e instalar plugins disponibles. Se sugiere seguir el enfoque de Visual Studio Code, donde los plugins pueden buscarse, visualizarse y gestionarse desde la propia aplicación.

- Optimización del plugin de captura de pantalla: Aunque funcional, el plugin de pantalla mostró limitaciones en tasas de cuadros elevadas. Como parte del trabajo futuro se propone explorar implementaciones específicas para cada sistema operativo, aprovechar herramientas como FFmpeg de forma directa y evaluar el uso de la GPU como opción configurable, lo que podría mejorar el rendimiento y reducir el consumo de recursos.
- Mejoras en la lógica de buffering y persistencia de datos: Se recomienda optimizar el sistema de almacenamiento temporal, especialmente en lo relativo al vaciado periódico de datos hacia disco. Esto incluye ajustar los parámetros de uso de memoria, el tiempo periódico de vaciado definido, evaluar esquemas adaptativos de escritura y explorar alternativas para mejorar la eficiencia en escenarios de alta demanda.

6.4 REFLEXIONES PERSONALES

El desarrollo de este proyecto representó una experiencia significativa tanto en el ámbito técnico como personal. Uno de los principales desafíos fue trabajar en el desarrollo de una aplicación de escritorio, un tipo de sistema con el que no había tenido experiencia previa en profundidad. A esto se sumó la complejidad de comprender el funcionamiento de la versión original de MO, cuyo análisis implicó estudiar distintas etapas de su evolución, evaluar código desarrollado por terceros y detectar con precisión los factores que condujeron a los problemas presentados. Esta etapa fue particularmente formativa, ya que nunca antes había enfrentado un proceso tan riguroso de revisión crítica sobre un sistema preexistente.

Otro reto importante fue entender y rediseñar el sistema de plugins, buscando una arquitectura capaz de sostener la extensibilidad del software en el largo plazo. La implementación requirió anticipar posibles problemas futuros y definir mecanismos de transición que permitieran conservar la compatibilidad. A nivel técnico, el desarrollo del sistema de captura en entornos multiproceso y multihilo también implicó superar barreras complejas en cuanto a concurrencia, sincronización y manejo de datos en tiempo real.

Durante el proceso, fue necesario aprovechar múltiples conocimientos adquiridos a lo largo de la carrera. Sentí que la formación recibida me preparó adecuadamente para enfrentar estos desafíos. El proyecto me permitió fortalecer habilidades técnicas, especialmente aquellas vinculadas a aplicaciones de escritorio, y profundizar en buenas prácticas de documentación,

entendiendo que este aspecto es esencial para facilitar la evolución futura de un sistema.

Si bien existieron momentos exigentes en términos de diseño e implementación, el proceso resultó enriquecedor. Me sentí satisfecho con el resultado final, ya que logré desarrollar un producto de software que no solo es funcional, sino que además capitaliza y mejora la lógica de versiones anteriores, respetando su enfoque modular y extensible.

A nivel personal, considero que este trabajo me permitió crecer y reafirmar mi interés por enfrentar desafíos complejos que representen oportunidades reales de aprendizaje. Elegí este tema sabiendo que sería exigente, precisamente porque buscaba una experiencia que me empujara a mejorar y a prepararme de mejor forma para mi desarrollo profesional.

GLOSARIO

- API (Application Programming Interface): Interfaz de programación que define un conjunto de reglas y protocolos que permiten la comunicación entre diferentes componentes de software. Facilita la integración de funcionalidades sin exponer la lógica interna del sistema.
- Plugin: Componente de software que se integra de forma modular a una aplicación principal, permitiendo extender sus funcionalidades sin necesidad de modificar su núcleo.
- Interfaz de Usuario (UI): Conjunto de elementos visuales e interactivos mediante los cuales el usuario se comunica con una aplicación. Incluye botones, menús, formularios y demás componentes que facilitan la interacción.
- SDK (Software Development Kit): Conjunto de herramientas, librerías, documentación y ejemplos que facilitan el desarrollo de aplicaciones para una plataforma específica. Un SDK permite a los desarrolladores integrar funcionalidades, utilizar interfaces y compilar aplicaciones de forma más eficiente y estructurada.
- Frontend: Parte del software con la que interactúa directamente el usuario. Se encarga de la presentación visual y la experiencia de uso.
- Backend: Componente del software que gestiona la lógica de negocio, acceso a bases de datos, procesamiento de datos y comunicación con otros sistemas. Opera en segundo plano y expone funcionalidades a través de APIs u otros mecanismos de integración.
- Modularidad: Principio de diseño de software que promueve la organización de un sistema en módulos independientes. Cada módulo cumple una función específica y puede desarrollarse, modificarse o sustituirse sin afectar al resto del sistema.
- Extensibilidad: Capacidad de un sistema de software para incorporar nuevas funcionalidades o componentes (como plugins) sin alterar su estructura original. Permite que la aplicación evolucione de forma sostenible ante nuevos requerimientos.
- Datos Multimodales: Conjunto de datos recolectados desde múltiples fuentes o modalidades (como teclado, mouse, video, señales fisiológicas, etc.), que se integran para obtener una visión más completa y precisa del fenómeno observado.

REFERENCIAS BIBLIOGRÁFICAS

- Bailey, L. (2025). OBS Studio API Documentation.
URL <https://docs.obsproject.com/>
- Bailey, L. (s.f.). Plugins — OBS Studio 32.0.1 documentation.
URL <https://docs.obsproject.com/plugins#properties>
- BasuMallick, C. (2024). What Are Plugins? Working, Importance, and Best Practices | Spiceworks.
URL <https://www.spiceworks.com/tech/tech-general/articles/what-are-plugins/>
- Bogers, N. (2024). The power of multimodal research in human behavior studies.
URL <https://www.noldus.com/blog/multimodal-research>
- Brategren, K. (s.f.). Software modularity.
URL <https://www.modularmanagement.com/blog/software-modularity>
- Cáceres Rodríguez, C. F. (2018). Desarrollo de un módulo para la interacción, control y monitoreo remoto en el software multimodal observer.
- Code, V. S. (2025). Publishing extensions.
URL <https://code.visualstudio.com/api/working-with-extensions/publishing-extension#create-a-publisher>
- Curifuta Poblete, G. A. (2017). **Extensiones de MO para el análisis de datos de rastreo ocular y expresiones faciales.**
- Díaz Sandoval, F. I. (2021). Rediseño de interfaz de usuario y modelo de interacción de Multimodal Observer.
- Electron (s.f.). Build cross-platform desktop apps with JavaScript, HTML, and CSS.
URL <https://www.electronjs.org/>
- GeeksforGeeks (2024). Understanding Python Pickling with example.
URL <https://www.geeksforgeeks.org/python/understanding-python-pickling-example/>
- Gutiérrez Gálvez, C. J. (2016). MO: un software modular y extensible para la captura y visualización de datos multimodales.
- Huincahue Azócar, G. (2024). Paquete de extensión de Multimodal Observer para la captura, visualización y análisis de datos de rastreo ocular y fisiológicos.
- iMotions (s.f.a). iMotions Lab - Powering Human Insight. Web page.
URL <https://imotions.com/products/imotions-lab/>
- iMotions (s.f.b). iMotions Lab Modules. Web page.
URL <https://imotions.com/products/imotions-lab/modules/>
- iMotions (s.f.c). iMotions Lab Pricing. Web page.
URL <https://imotions.com/products/pricing/>
- InTeractiOn (s.f.). InTeractiOn.
URL <https://interaction-lab.info/>
- IONOS (2020). ¿Qué es un plugin y para qué se usa?
URL <https://www.ionos.com/es-us/digitalguide/servidores/know-how/que-es-un-plugin/>
- Jaramillo Vallejos, B. A. (2024). Mejora de la usabilidad de Multimodal Observer a través de la homogeneización de interfaces y la integración de sus componentes.

- Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software*, 12, 45–50.
- Martin, J. (1991). *Rapid Application Development*. The James Martin productivity series. Macmillan Publishing Company.
URL <https://books.google.cl/books?id=o6FQAAAAAAAJ>
- Martins, J. (2024). ¿Qué es la metodología Kanban y cómo funciona? [2024] • Asana.
URL <https://asana.com/es/resources/what-is-kanban>
- Mateusz (2024). What is the Virtual DOM in React?
URL <https://www.freecodecamp.org/news/what-is-the-virtual-dom-in-react/>
- MDN (s.f.). <script type="importmap">.
URL <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script/type/importmap>
- Meta Platforms (s.f.). React – Una biblioteca de JavaScript para crear interfaces de usuario.
<https://es.react.dev/>.
- Noldus (s.f.a). Choose the software licensing that fits your needs. Web page.
URL <https://noldus.com/licensing>
- Noldus (s.f.b). Explore observational data in detail The Observer XT. Web page.
URL <https://www.noldus.com/observer-xt-human>
- Noldus (s.f.c). How do I use NoldusHub? Web page.
URL <https://noldus.com/noldushub/set-up>
- Noldus (s.f.d). Simplify your multimodal research NoldusHub. Web page.
URL <https://noldus.com/noldushub>
- Noldus (s.f.e). The Observer XT | Expand your lab with add-ons. Web page.
URL <https://www.noldus.com/observer-xt-human/add-ons>
- Noldus (s.f.f). The Observer XT 15 - Technical Specifications. PDF Document.
URL <https://delarosaresearch.com/uploads/Technical%20Specifications%20-The%20observer%20XT%2015.pdf>
- Noldus (s.f.g). Using The Observer XT for animal behavior research. Web page.
URL <https://noldus.com/observer-xt-animal>
- Noldus (s.f.h). Why use NoldusHub? Web page.
URL <https://www.noldus.com/noldushub/benefits>
- OBS (2022). Plugins Guide.
URL <https://obsproject.com/kb/plugins-guide>
- Obsproject (s.f.). GitHub - obsproject/obs-studio: OBS Studio - Free and open source software for live streaming and screen recording.
URL <https://github.com/obsproject/obs-studio>
- OpenJS, & Electron (s.f.). Process Model.
URL <https://www.electronjs.org/docs/latest/tutorial/process-model>
- OpenJS Foundation (s.f.). Why Electron. <https://www.electronjs.org/docs/latest/why-electron>.
- PLUX Biosignals (2021). OpenSignals user manual.
URL https://support.pluxbiosignals.com/wp-content/uploads/2021/11/OpenSignals_Manual.pdf

PLUX Biosignals (2025). OpenSignals (r)evolution (Download).

URL <https://support.pluxbiosignals.com/knowledge-base/introducing-opensignals-revolution/>

PLUX Biosignals (s.f.a). OpenSignals Real-time biosignals. In the lab on-the-go.

URL <https://www.pluxbiosignals.com/pages/opensignals>

PLUX Biosignals (s.f.b). Software Add-Ons.

URL <https://www.pluxbiosignals.com/collections/software-add-ons>

Polanco Berrios, D. I. (2019). Desarrollo de un módulo para asistentes destinados a la gestión y despliegue de protocolos de investigación en MO.

Preston-Werner, T. (s.f.). Versionado Semántico 2.0.0.

URL <https://semver.org/lang/es/>

Prismic (2024). What is extensibility?

URL <https://prismic.io/blog/what-is-extensibility>

Python (s.f.). Python FAQ – General Python Questions. <https://docs.python.org/3/faq/general.html>.

Ramírez, S. (s.f.). FastAPI.

URL <https://fastapi.tiangolo.com/>

SonarQube (2025). Understanding measures and metrics | SonarQube Server Documentation.

URL <https://docs.sonarsource.com/sonarqube-server/latest/user-guide/code-metrics/metrics-definition/>

Soto Piña, M. A. (2021). Marco de trabajo para la evaluación de validez y confiabilidad de Multimodal Observer (MO).

Spink, A. (2023). Five studies showing the power of multi-modal data in behavioral research.

URL <https://www.noldus.com/blog/multimodal-data>

Syal, S. (2023). Demystifying Software Versioning: Understanding Patch, Minor, and Major Versions. <https://medium.com/@shilpasyal55/demystifying-software-versioning-understanding-patch-minor-and-major-versions-11927673f3d3>.

Tauri (2024). What is Tauri? <https://v2.tauri.app/start/>.

Trylesinski, M. (s.f.). Uvicorn - An ASGI web server, for Python.

URL <https://uvicorn.dev/>

Vergara, S. (2024). El ciclo PDCA en el desarrollo de software.

URL <https://www.itdo.com/blog/el-ciclo-pdca-en-el-desarrollo-de-software/>

Vilches Céspedes, F. C. (2017). Módulo para la búsqueda, descarga, instalación y actualización de extensiones de MO.

VPN Unlimited (s.f.). ¿qué es la extensibilidad? - términos y definiciones de ciberseguridad.

URL <https://www.vpnunlimited.com/es/help/cybersecurity/extensibility>

VS Code (2025). Extension API.

URL <https://code.visualstudio.com/api>

VS Code (s.f.). Visual Studio Code - Your code editor.

URL <https://code.visualstudio.com/>

WordPress (2014a). Plugin Basics.

URL <https://developer.wordpress.org/plugins/plugin-basics/>

WordPress (2014b). What is a Plugin?

URL <https://developer.wordpress.org/plugins/intro/what-is-a-plugin/>

Yegulalp, S. (2024). Electron vs. Tauri: Which cross-platform framework is for you? <https://www.infoworld.com/article/3547072/electron-vs-tauri-which-cross-platform-framework-is-for-you.html>.

ANEXO A. LICENCIA

© **Matías Jacop Figueroa Contreras y Roberto González-Ibáñez** , 2025



• Algunos derechos reservados. Esta obra está bajo una Licencia Creative Commons Reconocimiento Internacional 4.0. Sus condiciones de uso pueden ser revisadas en:
<https://creativecommons.org/licenses/by/4.0/>.

APÉNDICE A. REQUERIMIENTOS FUNCIONALES

Tabla A.1: Requerimientos Funcionales con Historias de Usuario asociadas.

ID	Requerimiento Funcional	Historia de Usuario	Módulo
RF-01	Crear proyecto	Como usuario, quiero crear un proyecto, para comenzar un nuevo proyecto de investigación.	Organización
RF-02	Editar proyecto	Como usuario, quiero editar los datos de un proyecto, para corregir errores o mantener actualizada la información.	Organización
RF-03	Eliminar proyecto	Como usuario, quiero eliminar un proyecto cuando ya no sea relevante o necesario, para mantener organizados mis proyectos.	Organización
RF-04	Ver proyectos creados	Como usuario, quiero ver los proyectos que he creado, para acceder a sus detalles y gestionar sus componentes.	Organización
RF-05	Ver información de un proyecto	Como usuario, quiero ver la información de un proyecto específico , para revisar sus datos.	Organización
RF-06	Agregar participantes a un proyecto	Como usuario, quiero agregar participantes a un proyecto, para individualizar sesiones de estudio.	Organización
RF-07	Editar participante de un proyecto	Como usuario, quiero modificar la información de un participante, para corregir errores o actualizar sus datos personales.	Organización
RF-08	Eliminar participante de un proyecto	Como usuario, quiero eliminar a un participante cuando ya no formen parte del estudio para evitar datos innecesarios.	Organización
RF-09	Ver los participantes de un proyecto	Como usuario, quiero poder ver los participantes de un proyecto para acceder a detalles de algún participante y gestionar sus componentes	Organización
RF-10	Ver información de un participante	Como usuario, quiero ver información de un participante específico para revisar sus datos.	Organización
RF-11	Proteger proyecto para que no sea modificable ni se pueda eliminar	Como usuario, quiero poder bloquear/desbloquear un proyecto, para que no se pueda modificar ni eliminar sin desbloquearlo	Organización
RF-12	Proteger participante para que no sea modificable ni se pueda eliminar	Como usuario, quiero poder bloquear/desbloquear un participante, para que no se pueda modificar ni eliminar sin desbloquearlo	Organización
RF-13	Crear un protocolo de investigación sobre un proyecto	Como usuario, quiero crear un protocolo de investigación, para que el proyecto tenga una base metodológica clara y estructurada que guie su ejecución	Organización

ID	Requerimiento Funcional	Historia de Usuario	Módulo
RF-14	Eliminar un protocolo de investigación asociado a un proyecto	Como usuario, quiero eliminar un protocolo de investigación, para que pueda gestionar correctamente los protocolos activos y evitar confusiones con protocolos incorrectos o inutilizados	Organización
RF-15	Ver las actividades del protocolo de investigación asociado al proyecto	Como usuario, quiero ver las actividades del protocolo de investigación asociado al proyecto, para que pueda conocer el detalle de las tareas planificadas, su orden, duración y configuración	Organización
RF-16	Agregar actividades al protocolo de investigación	Como usuario, quiero agregar actividades al protocolo de investigación, para que pueda estructurar y automatizar tareas específicas, incluyendo la ejecución de aplicaciones externas y la gestión del tiempo asignado a cada actividad.	Organización
RF-17	Editar una actividad del protocolo de investigación	Como usuario, quiero editar una actividad del protocolo de investigación, para poder actualizar su configuración en caso de cambios en el protocolo a utilizar	Organización
RF-18	Eliminar una actividad en el protocolo de investigación	Como usuario, quiero eliminar una actividad del protocolo de investigación, para que pueda mantener actualizada la estructura del protocolo, eliminando actividades que no son necesarias	Organización
RF-19	Iniciar un protocolo de investigación	Como usuario, quiero iniciar un protocolo de investigación, para que se ejecuten las actividades definidas en el orden establecido.	Organización
RF-20	Proteger un protocolo de investigación para que no se puedan modificar sus actividades ni eliminar el protocolo	Como usuario, quiero poder bloquear/desbloquear un protocolo de investigación, para que no se puedan modificar sus actividades ni eliminar el protocolo una vez validado, asegurando su integridad	Organización
RF-21	Permitir que al iniciar una captura se inicie con ella el protocolo de investigación	Como usuario, quiero que al iniciar una captura también se inicie automáticamente el protocolo de investigación asociado, para que se haga uso real del protocolo dentro de una captura de datos	Organización
RF-22	Permitir agregar plugins de distintos tipos	Como usuario, quiero poder agregar plugins, para extender las funcionalidades de la aplicación	Núcleo

ID	Requerimiento Funcional	Historia de Usuario	Módulo
RF-23	Eliminar plugins agregados	Como usuario, quiero poder eliminar plugins agregados, para mantener limpia la aplicación y eliminar funcionalidades que ya no necesito.	Núcleo
RF-24	Ver los plugins agregados	Como usuario, quiero ver los plugins agregados en la aplicación, para tener una vista general y gestionar los plugins.	Núcleo
RF-25	Ver información de un plugin	Como usuario, quiero poder ver la información detallada de un plugin específico, para comprender su funcionalidad y ver los datos asociados a este como el autor, la versión y tipo.	Núcleo
RF-26	Agregar configuraciones para el uso de un plugin en un proyecto	Como usuario, quiero poder agregar y personalizar configuraciones específicas para un plugin dentro de un proyecto, para asegurar que su comportamiento se ajuste a las necesidades particulares del proyecto,	Núcleo
RF-27	Editar configuraciones de un plugin configurable en un proyecto	Como usuario, quiero poder editar las configuraciones de un plugin en un proyecto, para ajustar su comportamiento según nuevas necesidades en un proyecto.	Núcleo
RF-28	Eliminar configuraciones de un plugin configurable en un proyecto	Como usuario, quiero poder eliminar configuraciones establecidas de un plugin en un proyecto, para dejar de utilizar funcionalidades configuradas.	Núcleo
RF-29	Ver las configuraciones de un plugin configurable en un proyecto	Como usuario, quiero ver las configuraciones realizadas a los plugins configurables en un proyecto, para acceder a los detalles de las configuraciones o realizar acciones sobre estas.	Núcleo
RF-30	Ver la configuración realizada de un plugin configurable en un proyecto	Como usuario, quiero ver la configuración realizada de un plugin en un proyecto, para conocer los detalles específicos de su configuración y verificar que este correctamente ajustada a las necesidades del proyecto.	Núcleo
RF-31	Conservar la posición de las ventanas de la aplicación al cerrar y volver a abrir	Como usuario, quiero que la aplicación conserve la posición y el tamaño de las ventanas al cerrarla y volverla a abrir, para mantener mi espacio de trabajo y no tener que reorganizarlo cada vez que uso la aplicación.	Núcleo
RF-32	Cambiar el lenguaje de la aplicación	Como usuario, quiero poder cambiar el lenguaje de la aplicación, para utilizar la interfaz en el idioma de mi preferencia.	Núcleo

ID	Requerimiento Funcional	Historia de Usuario	Módulo
RF-33	Comenzar una captura de datos	Como usuario, quiero poder iniciar una captura de datos, para registrar datos de múltiples fuentes relevantes para un uso posterior.	Captura
RF-34	Pausar una captura de datos	Como usuario, quiero poder pausar una captura de datos en curso, para detener temporalmente la recolección de datos sin perder el progreso ni tener que reiniciar por completo.	Captura
RF-35	Reanudar una captura de datos	Como usuario, quiero poder reanudar una captura de datos que fue pausada, para continuar con la recolección sin perder el progreso.	Captura
RF-36	Detener una captura de datos	Como usuario, quiero poder detener una captura de datos en curso, para finalizar la recolección de datos cuando ya no sea necesaria.	Captura
RF-37	Listar las sesiones de captura realizada	Como usuario, quiero ver un listado de las sesiones de captura de datos realizadas, para revisar el historial de capturas y acceder a sus detalles.	Captura
RF-38	Ver información de una sesión de captura	Como usuario, quiero ver la información de una sesión de captura de datos, para revisar los metadatos de la sesión.	Captura
RF-39	Eliminar una sesión de captura realizada	Como usuario, quiero poder eliminar sesiones de captura, para poder gestionar correctamente las sesiones y evitar confusiones con sesiones incorrecta o que fueron de prueba.	Captura
RF-40	Agregar teclas de acceso rápido para las acciones de captura (comenzar, pausar, reanudar, detener)	Como usuario, quiero poder configurar teclas de acceso rápido para las acciones de captura (comenzar, pausar, reanudar y detener), para ejecutar estas acciones de forma rápida.	Captura
RF-41	Agregar la opción de minimizar la aplicación al comenzar una captura	Como usuario, quiero tener la opción de minimizar automáticamente la aplicación al comenzar una captura de datos, para poder continuar con las tareas a realizar sin que la aplicación interfiera visualmente.	Captura
RF-42	Reproducir una sesión capturada	Como usuario, quiero poder reproducir una sesión de captura previamente realizada, para revisar de forma visual los datos recolectados y analizar lo registrado durante esa sesión	Visualización
RF-43	Configurar los datos a reproducir en cada plugin	Como usuario, quiero poder configurar que datos se deben reproducir por cada plugin de visualización, para personalizar la reproducción de sesiones según mis necesidades.	Visualización

ID	Requerimiento Funcional	Historia de Usuario	Módulo
RF-44	Pausar la reproducción de una sesión capturada	Como usuario, quiero poder pausar la reproducción de una sesión capturada, para detener temporalmente la reproducción .	Visualización
RF-45	Reanudar la reproducción de una sesión capturada	Como usuario, quiero poder reanudar la reproducción de una sesión capturada que fue pausada, para continuar con la visualización de los datos recolectados desde el punto en que lo dejé.	Visualización
RF-46	Navegar en la sesión capturada mediante una barra de reproducción	Como usuario, quiero poder navegar a través de una sesión capturada utilizando una barra de reproducción, para acceder rápidamente a momentos específicos de la sesión.	Visualización
RF-47	Cambiar la posición de los plugins de visualización en el espacio establecido (guardándose la configuración)	Como usuario, quiero poder cambiar la posición de los plugins de visualización dentro del espacio de trabajo, para organizar la interfaz según mis preferencias y que esa configuración se conserve en futuras sesiones.	Visualización
RF-48	Ocultar y desocultar un plugin en la visualización	Como usuario, quiero poder ocultar un plugin de visualización del espacio de trabajo, para enfocarme en los elementos que necesito en determinado momento sin perder la configuración del plugin.	Visualización
RF-49	Pasar a ventana un plugin de visualización	Como usuario, quiero poder pasar un plugin de visualización del espacio de trabajo a una ventana independiente, para trabajar con él de forma más cómoda, sin afectar el resto del espacio de trabajo.	Visualización
RF-50	Capturar eventos del teclado	Como usuario, quiero disponer de un plugin que capture los eventos del teclado durante una sesión, para poder registrar y analizar los eventos ocurridos.	Plugin de captura
RF-51	Capturar eventos del mouse	Como usuario, quiero disponer de un plugin que capture los eventos del mouse durante una sesión, para poder registrar y analizar los eventos ocurridos.	Plugin de captura
RF-52	Capturar la pantalla del computador	Como usuario, quiero disponer de un plugin que capture la pantalla del computador durante una sesión, para poder obtener un registro visual de lo ocurrido.	Plugin de captura

ID	Requerimiento Funcional	Historia de Usuario	Módulo
RF-53	Visualizar los datos capturados del teclado	Como usuario, quiero disponer de un plugin que me permita visualizar el mensaje completo escrito a partir de los datos capturados por el plugin de teclado, para poder comprender el contenido de lo que se escribió durante la sesión.	Plugin de reproducción
RF-54	Visualizar los datos capturados del mouse	Como usuario, quiero que exista un plugin que me permita visualizar los datos capturados por el plugin de mouse, para poder revisar los movimientos realizados.	Plugin de reproducción
RF-55	Visualizar un video capturado	Como usuario, quiero que exista un plugin que me permita visualizar videos capturados, para poder reproducir y analizar visualmente lo grabado.	Plugin de reproducción

Fuente: Elaboración propia, 2025.

APÉNDICE B. DIAGRAMAS DE CLASES

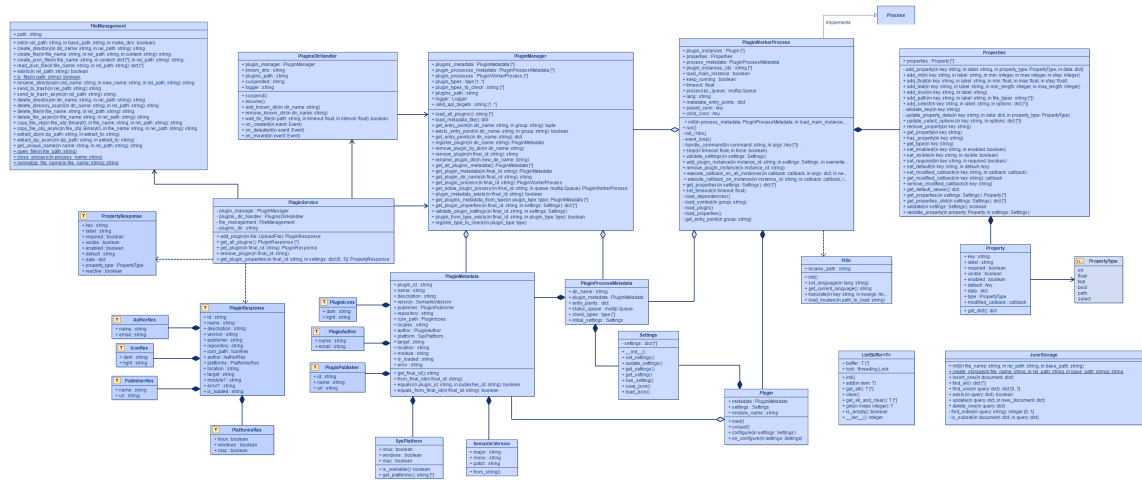


Figura B.1: Diagramas de clases del núcleo (backend).
Fuente: Elaboración propia, 2025.

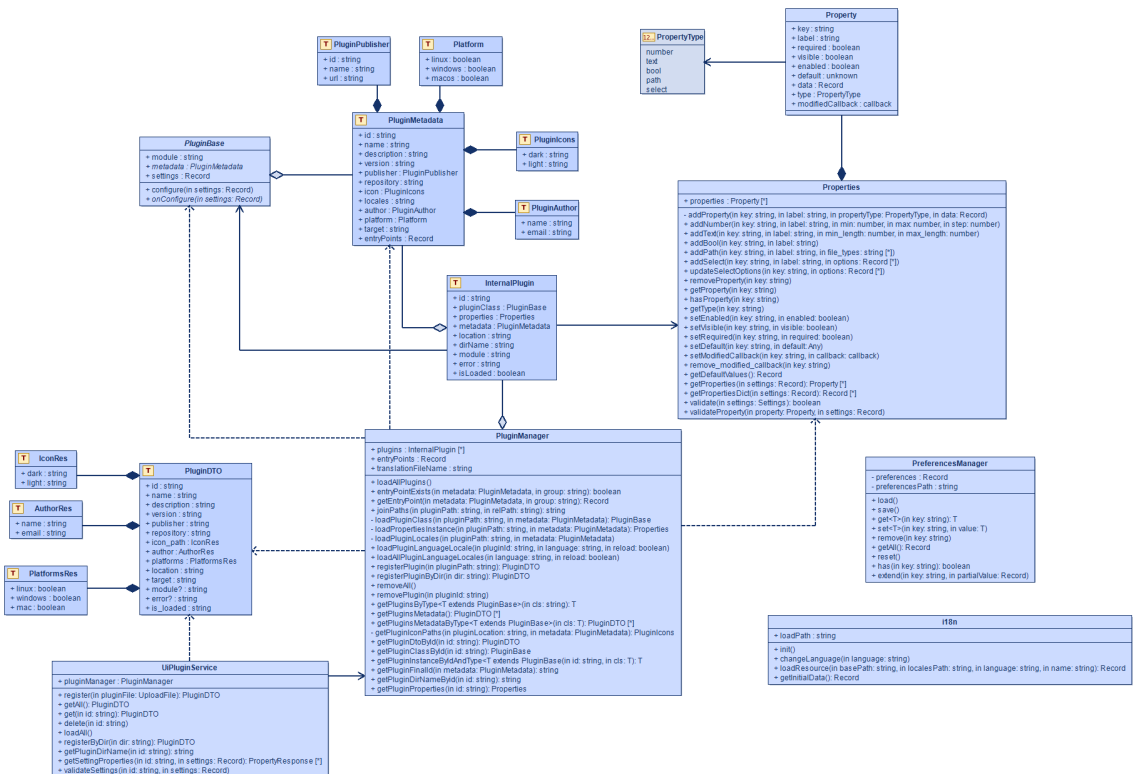


Figura B.2: Diagramas de clases del núcleo (frontend).
Fuente: Elaboración propia, 2025.

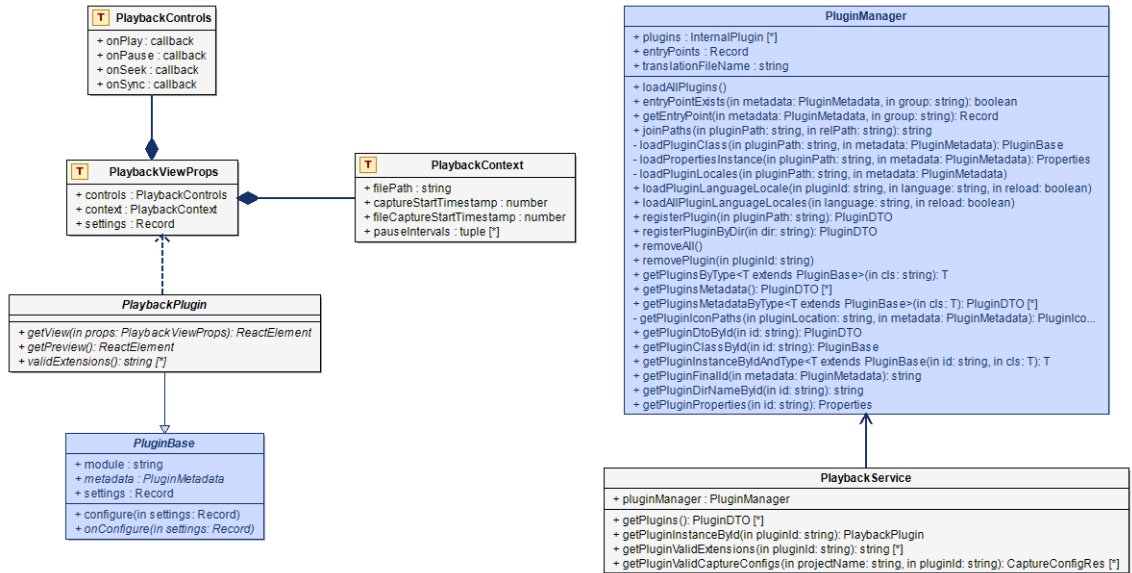


Figura B.6: Diagramas de clases del módulo de Visualización (frontend).

Fuente: Elaboración propia, 2025.

APÉNDICE C. PRUEBAS DE ACEPTACIÓN

Tabla C.1: Prueba de aceptación PA-01 - Operaciones con plugins.

Título	Operaciones con plugins
ID	PA-01
Prerrequisitos	Multimodal Observer, Plugins de captura, Plugins de visualización
Requisitos abordados	RF-22, RF-23, RF-24, RF-25, RNF-02, RNF-05
Pasos	1. Presionar en sección de plugins 2. Presionar en la opción de registrar 3. Examinar o soltar el archivo .zip de un plugin de captura 4. Presionar en el botón para registrar plugin 5. Examinar o soltar el archivo .zip de un plugin de visualización 6. Presionar en el botón para registrar plugin 7. Presionar la opción de plugins instalados 8. Verificar que se muestran correctamente los plugins instalados 9. Ver información de uno de los plugins instalados 10. Presionar en el botón para eliminar el plugin instalado 11. Presionar aceptar en la ventana de confirmación
Resultados esperados	• Los plugins se instalan correctamente y se ven en la sección de plugins instalados. • Se ve correctamente los datos asociados a un plugin. • Se elimina correctamente el plugin.
Evaluación	A
Observaciones	Esta es una observación general. Explorar la posibilidad de poner un Splash Window al inicio del programa donde aparezca el nombre del software, versión, nombre del lab, etc.

Fuente: Elaboración propia, 2025.

Tabla C.2: Prueba de aceptación PA-02 - Configuración inicial de un proyecto.

Título	Configuración inicial de un proyecto
ID	PA-02
Prerrequisitos	Multimodal Observer con plugins cargados
Requisitos abordados	RF-01, RF-04, RF-06, RF-09, RF-13, RF-15, RF-16, RF-26, RF-29, RNF-02

Pasos	1. Presionar en el icono “+” para crear un proyecto 2. Rellenar los campos del proyecto 3. Presionar el botón principal para crear el proyecto 4. Seleccionar el proyecto creado 5. Presionar en el icono “+” para crear un participante 6. Rellenar los campos del participante 7. Presionar en el botón principal para crear el participante 8. Presionar en el icono “+” para agregar una configuración de fuentes de captura 9. Seleccionar la fuente de captura y presionar en el botón principal para agregar fuente 10. Rellenar los campos de configuración del plugin 11. Presionar en el botón principal para agregar fuente 12. Presionar en el icono “+” para crear un protocolo 13. Rellenar el campo de nombre 14. Presionar en el icono “+” para agregar una actividad 15. Rellenar los campos de la actividad 16. Presionar en agregar actividad y cerrar 17. Presionar en el botón principal para crear el protocolo
Resultados esperados	Se observa el proyecto, el participante, la fuente de captura y el protocolo creado correctamente.
Evaluación	A
Observaciones	

Fuente: Elaboración propia, 2025.

Tabla C.3: Prueba de aceptación PA-03 - Proceso de captura y ejecución de protocolo.

Título	Proceso de captura y ejecución de protocolo
ID	PA-03
Prerrequisitos	Multimodal Observer, contar con un proyecto, participante, protocolo y configuración de plugins de captura de pantalla, teclado y mouse creados
Requisitos abordados	RF-19, RF-33, RF-34, RF-35, RF-36, RF-40, RF-41, RF-50, RF-51, RF-52
Pasos	1. Ir a la sección de ajustes 2. Habilitar las opciones de: • Iniciar protocolo al comenzar la captura • Detener protocolo al comenzar la captura • Minimizar la aplicación al iniciar la captura • Mostrar la aplicación al detener la captura 3. Configurar los atajos de teclado para la captura (iniciar, detener, pausar y reanudar) 4. Volver a la sección de proyectos 5. Seleccionar el proyecto, participante y protocolo 6. Iniciar la captura usando el atajo de teclado 7. Seguir los pasos del protocolo 8. Pausar y reanudar entre los pasos del protocolo la captura de datos usando los atajos de teclado 9. Al terminar el protocolo detener la captura con el atajo de teclado

Resultados esperados	• Funcionan correctamente las opciones seleccionadas • Funcionan correctamente los atajos de teclado • Se inicia, pausa, reanuda y detiene correctamente la captura • Se inicia y finaliza correctamente el protocolo
Evaluación	B
Observaciones	Aquí se provocan algunas confusiones. En el sentido que efectivamente al iniciar una captura esta graba sin problemas. Por otro lado uno puede iniciar un protocolo sin grabar, pero este no graba. Le veo utilidad, pues permite probar protocolo sin grabar datos. Por otro lado la vinculación de ambas funcionalidades solo se da cuando uno lo hace manualmente (iniciar protocolo y luego captura) o bien en configuraciones, indicando que la captura debe ir enlazada al protocolo. La observación aquí es que por un lado los botones del apartado de controles podrían ser solo iconos representativos de las acciones (record, play, start protocol) pero que estos botones al presionar tenga opciones subyacentes (ej: capture with protocol y capture without protocol) así el botón de inicio de protocolo podrá relegarse a una opción dentro de la ventana de protocolo en la línea de “probar protocolo”.

Fuente: Elaboración propia, 2025.

Tabla C.4: Prueba de aceptación PA-04 - Reproducción de sesión capturada.

Título	Reproducción de sesión capturada
ID	PA-04
Prerrequisitos	Multimodal Observer, contar con un participante con sesiones de captura disponible con datos válidos para los plugins de visualización
Requisitos abordados	RF-42, RF-43, RF-44, RF-45, RF-46, RF-47, RF-48, RF-49, RF-53, RF-54, RF-55, RNF-02, RNF-07
Pasos	1. Seleccionar el proyecto y participante 2. Presionar el botón de reproducir sesiones 3. Modificar la disposición del espacio de reproducción 4. Abrir en una ventana nueva una de las vistas 5. Presionar el botón de iniciar la reproducción 6. Presionar el botón de pausar reproducción 7. Navegar mediante el reproductor 8. En las configuraciones de vistas de reproducción ocultar y desocultar una vista
Resultados esperados	• Se modifica la disposición del espacio de reproducción • Se abre y reproduce correctamente en una ventana nueva una de las vistas de reproducción • Se oculta/desoculta correctamente una vista de reproducción • Se inicia, pausa y navega correctamente • Se reproduce de manera sincronizada la sesión capturada
Evaluación	B

Observaciones	Al iniciar un proceso de reproducción parte inmediatamente con la última y no hay un hint directo a donde se encuentran las anteriores. Luego de explorar, <i>llegue</i> al botón de lista de sesiones. La petición aquí es que cuando se proceda a reproducir sesiones, debería partir con el <i>menu</i> de sesiones grabadas expandido para que el usuario escoja una dentro del listado disponible. No fue intuitivo
----------------------	--

Fuente: Elaboración propia, 2025.

Tabla C.5: Prueba de aceptación PA-05 - Operaciones extra participante.

Título	Operaciones extra participante
ID	PA-05
Prerrequisitos	Multimodal Observer, contar con un proyecto y participante previamente creado
Requisitos abordados	RF-07, RF-08, RF-09, RF-10, RF-12
Pasos	1. Presionar el botón de información del participante 2. Presionar en el botón de bloquear el participante 3. Presionar el botón de actualizar y comprobar que no deja actualizar 4. Presionar el botón de eliminar y comprobar que no deja eliminar 5. Presionar el botón de desbloquear el participante 6. Presionar el botón de actualizar del participante 7. Cambiar los datos originales del participante 8. Presionar en el botón principal para actualizar el participante 9. Presionar el botón de información del participante para verificar que se cambió la información 10. Presionar el botón de eliminar el participante 11. Confirmar la eliminación del participante
Resultados esperados	• Se ve correctamente la información del participante en un inicio y al actualizar esta. • Se bloquea correctamente el participante prohibiendo actualizar y eliminar este. • Al desbloquear el participante se actualiza y elimina correctamente.
Evaluación	A
Observaciones	

Fuente: Elaboración propia, 2025.

Tabla C.6: Prueba de aceptación PA-06 - Operaciones extra protocolo.

Título	Operaciones extra protocolo
ID	PA-06
Prerrequisitos	Multimodal Observer, contar con un proyecto y protocolo previamente creado
Requisitos abordados	RF-13, RF-14, RF-15, RF-16, RF-17, RF-18

Pasos	1. Presionar el icono de información del protocolo 2. Presionar en el icono de bloquear el protocolo 3. Presionar el icono de actualizar y comprobar que no deja actualizar 4. Presionar el icono de eliminar y comprobar que no deja eliminar 5. Presionar el icono de desbloquear el protocolo 6. Presionar el icono de actualizar del protocolo 7. Cambiar el nombre original del protocolo 8. Presionar en el icono de actualizar actividad del protocolo 9. Cambiar los campos de la actividad y presionar en actualizar 10. Presionar en el botón principal para actualizar el protocolo 11. Presionar el icono de información del protocolo para verificar que se cambió la información 12. Presionar el icono de eliminar el protocolo 13. Confirmar la eliminación del protocolo
Resultados esperados	• Se ve correctamente la información del protocolo y sus actividades en un inicio y al actualizar esta. • Se bloquea correctamente el protocolo prohibiendo actualizar y eliminar este. • Al desbloquear el protocolo se actualiza y elimina correctamente.
Evaluación	B
Observaciones	• aparentemente (no si es algo general), el archivo ejecutable vinculado al proceso es el mismo que queda en la lista de tarea (al menos en Windows). De ser así, el sistema podría sugerir como base ese nombre de Proceso. • Luego al “testear” el protocolo (sin grabar), que es lo que indico anteriormente se puede comprobar si realiza o no la acción. Ayudaría en esta parte un hint que indique al usuario que significa “cerrar” al terminar captura. • nota: la opción de pausa de captura, no pausa el protocolo. esto produce inconsistencia, ya que se puede pausar una captura y el protocolo temporizado sigue avanzando.

Fuente: Elaboración propia, 2025.

Tabla C.7: Prueba de aceptación PA-07 - Operaciones extra plugin configurable.

Título	Operaciones extra plugin configurable
ID	PA-07
Prerrequisitos	Multimodal Observer, contar con un proyecto y configuración de plugin de captura creado para el proyecto
Requisitos abordados	RF-26, RF-27, RF-28, RF-29, RF-30

Pasos	1. Presionar el icono de ajustes en la configuración de fuente de captura 2. Modificar la configuración 3. Presionar en actualizar fuente 4. Revisar que se actualizó correctamente, viendo la configuración 5. Presionar en el icono de eliminar en la configuración de fuente de captura 6. Confirmar la eliminación
Resultados esperados	La configuración se actualiza y elimina correctamente
Evaluación	B
Observaciones	Funciona, pero queda pendiente que cada captura vinculada a un plugin tenga una metadata que lo relacione, esto para que al momento de añadir un plugin de reproducción solo muestre las fuentes de datos compatibles. Actualmente un plugin de reproducción de keyboard y mouse muestra fuentes de dato de mouse y teclado sin distinción lo que deriva en errores al escoger una fuente incorrecta (ej. mouse con plugin de teclado o viceversa)

Fuente: Elaboración propia, 2025.

Tabla C.8: Prueba de aceptación PA-08 - Operaciones extra proyecto.

Título	Operaciones extra proyecto
ID	PA-08
Prerrequisitos	Multimodal Observer, contar con un proyecto creado
Requisitos abordados	RF-02, RF-03, RF-04, RF-05, RF-11
Pasos	1. Presionar el icono de información del proyecto 2. Presionar en el icono de bloquear el proyecto 3. Presionar el icono de actualizar y comprobar que no deja actualizar 4. Presionar el icono de eliminar y comprobar que no deja eliminar 5. Presionar el icono de desbloquear el proyecto 6. Presionar el icono de actualizar del proyecto 7. Cambiar los datos originales del proyecto 8. Presionar en el botón principal para actualizar el proyecto 9. Presionar el icono de información del proyecto para verificar que se cambió la información 10. Presionar el icono de eliminar el proyecto 11. Confirmar la eliminación del proyecto
Resultados esperados	• Se ve correctamente la información del proyecto en un inicio y al actualizar esta. • Se bloquea correctamente el proyecto prohibiendo actualizar y eliminar este. • Al desbloquear el proyecto se actualiza y elimina correctamente.
Evaluación	A
Observaciones	

Fuente: Elaboración propia, 2025.